

MindMate

An AI-Powered Mental Health Assistant for Students

A Project Report submitted in partial fulfilment of the requirements for the award of
the degree of

Master of Science (Data Science)

Submitted by

Ganesh L

Enrolment No: [Enrolment Number]

Under the guidance of

Mr. Prashant Stephens (Mentor)

In association with

Plag Pro, Noida

Chandigarh University

2026

Certificate

This is to certify that the project report titled **MindMate, An AI-Powered Mental Health Assistant for Students** is a bona fide record of the project work carried out by **Ganesh L** (Enrolment No: [Enrolment Number]) in partial fulfilment of the requirements for the award of the degree of Master of Science (Data Science).

The work was carried out under my supervision during the academic year 2026 in association with Plag Pro, Noida, and has not been submitted previously for the award of any other degree or diploma.

Copy of the certificate received from Qollabb to be pasted here.

Mr. Prashant Stephens

(Mentor / Guide)

Declaration

I, **Ganesh L**, hereby solemnly declare that the project report titled **MindMate, An AI-Powered Mental Health Assistant for Students**, submitted in partial fulfilment of the requirements for the award of the degree of Master of Science (Data Science), is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2026 under the supervision of Mr. Prashant Stephens (Mentor).
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge; the datasets used for model training and evaluation are clearly identified as synthetically generated for demonstration and no real personal data has been collected or used.

Place: _____ Date: _____

Student Signature: _____

Student Name: Ganesh L Enrolment No: [Enrolment Number]

Acknowledgement

The successful completion of this project would not have been possible without the guidance and support of several individuals, and I take this opportunity to express my sincere gratitude to them.

I am deeply grateful to my mentor, Mr. Prashant Stephens, for his valuable guidance, constructive feedback, and continuous encouragement throughout the course of this project. His insights shaped both the technical direction and the ethical grounding of this work.

I extend my sincere thanks to Plag Pro, Noida, for providing the opportunity and platform to undertake this project, and to the faculty of Chandigarh University for their academic support.

Finally, I thank my family and friends for their patience and unwavering support during the development of MindMate.

Abstract

This report presents **MindMate, An AI-Powered Mental Health Assistant for Students**, a privacy-respecting web application designed to offer accessible, stigma-free, first-line wellbeing support to a student population that faces rising psychological strain and limited access to professional counselling. The objective of the project was to design and develop an AI mental-health assistant, to personalise its support using machine learning, and to evaluate its effect on mental-health and academic outcomes. The system was built in **Python** with a **Streamlit** multipage interface, **scikit-learn** for a K-Means student segmenter and a Logistic-Regression stress classifier, the **VADER** lexicon for sentiment analysis, and a conversational layer that uses the **OpenAI Chat Completions API** when available and a fully **offline rule-based engine** otherwise, with all records held in **SQLite**. A rule-based crisis-safety layer screens every message before any reply is generated. Development followed an **iterative, Agile-influenced methodology** of incremental builds and continuous testing. On synthetic but reproducible data, the segmenter achieved a silhouette score of **0.209** across four meaningful segments, while the stress classifier reached **0.900** accuracy and a **macro-F1 of 0.889**. A simulated four-week pilot comparing treatment and control groups produced large effect sizes, including a Cohen's d of **-1.62** for PHQ-9 reduction. These results, obtained on simulated data, demonstrate a sound evaluation methodology rather than clinical efficacy, and position MindMate as a transparent, safety-first complement to existing student support services.

Table of Contents

Chapter 1: Introduction	14
1.1 Background of the Project	14
1.2 Problem Statement	15
1.3 Objectives	16
1.4 Scope of the Project	16
1.5 Existing System Overview	17
1.6 Proposed System Overview	18
1.7 Technologies Used	19
1.8 Organisation of the Report	20
 Chapter 2: Literature Review and System Study	 21
2.1 Introduction	21
2.2 Mental Health Among Students	22
2.3 Conversational Agents in Mental Health	23
2.4 Machine Learning in Mental Health	25
2.5 Sentiment Analysis and NLP	26
2.6 Screening Instruments (PHQ-9 and GAD-7)	27
2.7 Personalisation and Recommender Systems	28
2.8 Comparative Analysis of Existing Systems	29
2.9 Software Development Models	31
2.10 Research Gap	32
 Chapter 3: System Analysis	 34
3.1 Introduction	34
3.2 Requirement Analysis	35
3.3 Functional Requirements	35
3.4 Non-Functional Requirements	38
3.5 User Requirements	39
3.6 Feasibility Study	40
3.6.1 Technical Feasibility	40

3.6.2 Economic Feasibility	41
3.6.3 Operational Feasibility	41
3.7 Use-Case Analysis	41
3.8 Data-Flow Analysis	43
3.9 System Requirements (Hardware/Software)	45
Chapter 4: System Design	46
4.1 Introduction	46
4.2 Design Objectives and Principles	46
4.3 System Architecture	47
4.4 Class Design	49
4.5 Database Design	51
4.5.1 Entity-Relationship Model	51
4.5.2 Schema and Table Structures	53
4.6 Behavioural Design (Sequence and Activity)	56
4.7 User-Interface Design	58
4.8 Algorithm and Model Design	59
Chapter 5: System Implementation	61
5.1 Introduction	61
5.2 Development Environment and Tools	61
5.3 Project Structure	62
5.4 Database Implementation	63
5.5 Sentiment and Emotion Analysis Module	64
5.6 Student Segmentation Module	65
5.7 Stress Classification Module	67
5.8 Recommendation Engine	68
5.9 Conversational Assistant and OpenAI Integration	69
5.10 Safety and Crisis-Detection Layer	70
5.11 User Interface Implementation	71
5.12 Version Control	73

Chapter 6: Testing	74
6.1 Introduction	74
6.2 Testing Strategy	74
6.3 Unit Testing	75
6.4 Integration Testing	76
6.5 System Testing	77
6.6 Test Cases	77
6.7 Safety-Critical Testing	79
6.8 Model Validation	80
6.9 Results and Defects	82
 Chapter 7: Results and Discussion	 84
7.1 Introduction	84
7.2 Dataset Overview	85
7.3 Student Segmentation Results	86
7.4 Stress Classification Results	89
7.5 Sentiment Analysis Results	91
7.6 Pilot Evaluation of Effectiveness	92
7.7 Discussion	95
7.8 Comparison with Existing Systems	96
7.9 Limitations	96
 Chapter 8: Conclusion and Future Scope	 98
8.1 Conclusion	98
8.2 Achievement of Objectives	99
8.3 Key Contributions	101
8.4 Technical Learning Outcomes	101
8.5 Limitations	102
8.6 Future Scope	103
 Chapter 9: References	 105

Chapter 10: Appendices	108
Appendix A: Database Schema	108
Appendix B: Sentiment Analysis Module (src/ml/sentiment.py)	109
Appendix C: Student Segmentation Module (src/ml/personalization.py)	111
Appendix D: Safety / Crisis-Detection Layer (src/assistant/safety.py)	113
Appendix E: Conversational Engine (src/assistant/chat_engine.py)	115
Appendix F: Running the Project	117

List of Figures

Figure 3.1: Use-case diagram of MindMate	42
Figure 3.2: Data-flow diagram (Level 0 / context)	43
Figure 3.3: Data-flow diagram (Level 1)	44
Figure 4.1: MindMate layered system architecture	48
Figure 4.2: Core class diagram	49
Figure 4.3: Entity-relationship diagram	52
Figure 4.4: Sequence diagram for sending a chat message	56
Figure 4.5: Activity diagram for message handling with the safety branch	57
Figure 6.1: Confusion matrix of the stress classifier on the test set	81
Figure 7.1: Correlation matrix of the six wellbeing features	85
Figure 7.2: Distribution of students across the four support segments	87
Figure 7.3: PCA projection of the student segments	88
Figure 7.4: Confusion matrix of the stress classifier	90
Figure 7.5: Sentiment of a student's journal notes over time	91
Figure 7.6: Pre/post comparison of treatment vs control	93
Figure 7.7: Effect sizes by outcome measure	94

List of Tables

Table 2.1: Comparative Analysis of Existing Systems	30
Table 3.1: Functional Requirements	36
Table 4.1: Schema and Table Structures	53
Table 4.2: Schema and Table Structures	54
Table 4.3: Schema and Table Structures	54
Table 4.4: Schema and Table Structures	55
Table 4.5: Schema and Table Structures	55
Table 6.1: Test Cases	78
Table 7.1: Stress Classification Results	89
Table 7.2: Pilot Evaluation of Effectiveness	93
Table 8.1: Achievement of Objectives	100

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
VADER	Valence Aware Dictionary and sEntiment Reasoner
PHQ-9	Patient Health Questionnaire (9-item)
GAD-7	Generalised Anxiety Disorder scale (7-item)
CBT	Cognitive Behavioural Therapy
API	Application Programming Interface
UI / UX	User Interface / User Experience
DFD	Data-Flow Diagram
ER	Entity-Relationship
UML	Unified Modelling Language
PCA	Principal Component Analysis
CSV	Comma-Separated Values
SQL	Structured Query Language

Chapter 1: Introduction

1.1 Background of the Project

Mental ill-health among young people has become one of the defining public-health challenges of the present decade. The World Health Organization estimates that mental disorders affect a substantial share of the global population and that the majority of lifetime conditions begin before early adulthood, yet most affected individuals never receive adequate care (World Health Organization, 2022). University students sit precisely within this high-risk window. The transition into higher education combines academic pressure, financial uncertainty, social upheaval, separation from established support networks, and, frequently, the first experience of living independently. These stressors arrive at the same developmental stage at which many serious conditions first emerge, making the student years a period of unusual vulnerability.

The scale of the problem is well documented. The World Health Organization World Mental Health Surveys International College Student Project found that roughly a third of first-year university students across multiple countries screened positive for at least one common mental disorder, with anxiety and mood disorders especially prevalent (Auerbach et al., 2018). Campus-based research in the United States reaches similar conclusions, reporting high and persistent rates of depression and anxiety that vary across student subgroups and institutions but remain consistently elevated relative to the general population (Eisenberg et al., 2013). These conditions are not merely matters of personal distress; they are strongly associated with reduced concentration, impaired academic performance, lower retention, and, in the most serious cases, self-harm and suicidal ideation. The academic and human costs are therefore deeply intertwined.

Against this rising demand stands a chronic shortage of supply. University counselling services are frequently overstretched, with waiting lists that can extend for weeks during peak periods such as examinations. Even where services exist, they operate within fixed office hours and finite staffing, which means that the moment a student most needs support, late at night, during an anxiety spike, or in the hours before a deadline, is often precisely the moment when no human counsellor is available. Compounding the capacity gap is the persistent stigma surrounding mental health. Many students hesitate to disclose distress to a counsellor, a tutor, or a peer for fear of being judged, labelled, or treated differently, and this reluctance can delay help-seeking until a problem has escalated.

Digital tools offer a pragmatic response to this mismatch between need and capacity. The field of digital psychiatry has matured rapidly, and a growing evidence base indicates that smartphone applications, chatbots, and other digital interventions can extend the reach of

mental-health support, lower barriers to entry, and provide continuous, on-demand contact at a scale that human services alone cannot match (Torous et al., 2021). Such tools are well suited to students, a population that is digitally fluent, accustomed to managing aspects of life through applications, and often more willing to engage candidly with a private interface than with a person. Digital support is not a substitute for clinical care, but it can serve as an accessible first point of contact, a means of self-monitoring, and a bridge that helps students recognise when professional help is warranted. It is within this context, high need, constrained capacity, persistent stigma, and a digitally engaged audience, that MindMate is situated.

1.2 Problem Statement

The core problem that MindMate addresses is the gap between the volume of student mental-health need and the availability of timely, low-barrier support. Students who experience stress, low mood, or anxiety frequently lack an immediate, private, and non-judgemental avenue through which to express what they are feeling, to make sense of it, and to take a small constructive step. Traditional counselling, while essential, cannot be available to every student at every hour, and the stigma attached to seeking help further suppresses early engagement. The result is that distress often goes unspoken until it becomes severe.

MindMate is designed to provide **accessible, stigma-free, always-available, and personalised first-line support**. It offers a supportive conversational space that a student can use at any time and from any device, without an appointment and without disclosing their identity to another person. Rather than treating every user identically, the system personalises its tone and its recommendations using machine learning, so that the support offered reflects the individual's recent mood, screening scores, sentiment, and engagement patterns. Crucially, the system is built around a safety-first principle: every message is screened for indications of crisis before any reply is produced, and where serious risk is detected the application steps back from conversation and directs the student to qualified human help and verified helplines.

It is equally important to state clearly what the problem is *not*. MindMate is **not** a clinical instrument and does not attempt to replace professional diagnosis or treatment. It does not deliver therapy, prescribe interventions, or manage acute emergencies. The problem it solves is the absence of an accessible, supportive, and intelligently personalised first layer of care, one that can listen, signpost, encourage healthy coping, and, when necessary, escalate to human services. Positioning the tool honestly in this way is itself part of solving the problem responsibly.

1.3 Objectives

The project is organised around three objectives drawn directly from the project brief.

Objective 1: Design and develop an AI mental-health assistant for students. The first aim is to build a complete, working application that students can actually use, rather than a research prototype confined to a notebook. This entails designing an intuitive multipage interface, implementing a supportive conversational engine, providing structured features for mood tracking and validated self-screening, and ensuring that the whole system operates reliably, including a guarantee that it continues to function even without an external language-model service. The assistant must be approachable, private, and safe by design.

Objective 2: Personalise its support using machine-learning algorithms. The second aim is to move beyond a one-size-fits-all assistant by embedding machine learning at the heart of the experience. This involves segmenting students into meaningful support groups using unsupervised clustering, predicting stress level using a supervised classifier, analysing the sentiment of free-text input, and feeding these signals into a transparent recommender that selects appropriate coping strategies. Personalisation is intended to be explainable: a student should be able to see why a given recommendation was made.

Objective 3: Evaluate its effectiveness on mental-health and academic outcomes. The third aim is to assess, in a methodologically rigorous manner, whether the assistant plausibly improves outcomes. Because real-participant trials lie beyond the scope of an academic project, evaluation is conducted on carefully constructed synthetic data that encodes realistic relationships. The evaluation measures changes in standardised mental-health indicators and in academic focus, applies appropriate statistical tests, and reports effect sizes, thereby demonstrating a complete and credible evaluation pipeline.

1.4 Scope of the Project

The scope of MindMate is defined deliberately so that the system remains both useful and responsible.

In scope. The project delivers the following capabilities:

- **Conversational wellbeing support** through an empathetic chat interface that adapts its tone to the user's state.
- **Mood tracking**, allowing students to log daily mood, energy, and sleep, with sentiment analysis of free-text notes and trend visualisation.
- **Validated self-screening** using the PHQ-9 and GAD-7 questionnaires with their published scoring and severity bands (Kroenke et al., 2001; Spitzer et al., 2006).

- **Machine-learning personalisation**, including student segmentation, stress prediction, and an explainable coping-strategy recommender.
- **A transparent insights view** that exposes the features and reasoning behind the system's outputs.
- **A curated resources library** of psychoeducational material, coping strategies, and verified crisis helplines covering both Indian and global services.

Out of scope. The project explicitly excludes:

- **Clinical diagnosis** of any mental-health condition. Screening tools indicate severity for self-awareness and signposting only.
- **Treatment or therapy.** MindMate does not deliver a structured clinical intervention and is not a medical device.
- **Emergency response.** The system cannot dispatch help or guarantee a human response; in crisis situations it signposts to qualified services rather than attempting to manage the situation itself.

This boundary is not a limitation imposed reluctantly but a core design decision. A first-line support tool that overstates its competence would be ethically unacceptable, so the scope is drawn to keep the system helpful, honest, and safe.

1.5 Existing System Overview

Several conversational mental-health applications already exist and have informed the design of MindMate. The earliest demonstration of conversational interaction with a computer, ELIZA, showed decades ago that people will engage emotionally with a text-based agent even when its responses are generated by simple pattern matching (Weizenbaum, 1966). Contemporary systems are considerably more sophisticated.

Woebot is a fully automated conversational agent that delivers principles of cognitive behavioural therapy through scripted dialogue. In a randomised controlled trial with young adults, it was associated with a significant reduction in symptoms of depression over a two-week period, providing some of the strongest early evidence that a chatbot can produce measurable benefit (Fitzpatrick et al., 2017). **Wysa** is an empathy-driven agent that combines conversational support with self-help tools; a real-world, mixed-methods evaluation found that users who engaged more heavily with the application reported greater improvements in mood, suggesting a dose-response relationship between engagement and benefit (Inkster et al., 2018). **Tess** is a psychological AI agent designed to deliver on-demand emotional support; a randomised controlled trial reported reductions in self-identified symptoms of depression and anxiety among students who interacted with it

(Fulmer et al., 2018).

Alongside these products sits conventional, human-delivered university counselling, which remains the gold standard for depth of care but is constrained by capacity, cost, opening hours, and the stigma that deters some students from attending. Reviews of the wider landscape note that while conversational agents show promise, many are closed, commercial systems whose internal logic is opaque to the user, and that few are tailored to the specific pressures of student life or designed with transparent, inspectable personalisation. These observations directly motivate the design choices behind the proposed system.

1.6 Proposed System Overview

MindMate is proposed as a supportive conversational wellbeing assistant for students that combines the accessibility of existing chatbots with three differentiating commitments: transparent machine-learning personalisation, a safety-first architecture, and guaranteed offline capability.

The system is delivered as a multipage Streamlit web application. A student can converse with the assistant, log and review their mood over time, complete validated PHQ-9 and GAD-7 self-checks, inspect the reasoning behind the system's outputs, and access a library of coping strategies and helplines. Beneath this interface, six canonical features, average mood, average sleep, PHQ-9 score, GAD-7 score, average sentiment, and engagement, are computed for each user and passed to the machine-learning layer. A K-Means segmenter assigns the student to one of four meaningful support groups, and a Logistic-Regression classifier predicts their stress level. These outputs, together with the emotion detected in the student's own words, drive a hybrid recommender that selects coping strategies and, importantly, returns a plain-language rationale for each suggestion.

The first differentiator is **transparency**. Where many existing applications present personalisation as a black box, MindMate exposes the features it uses, the segment it has inferred, the stress probabilities it has computed, and the reasoning behind each recommendation, so that the student can understand and challenge the system's view of them. The second is **safety-first design**: a rule-based crisis-detection layer screens every message before any reply is generated, classifying risk as none, elevated, or crisis, and responding conservatively so that a false alarm is always preferred to a missed signal. In a crisis case the system declines to counsel and instead provides a validating message and verified helplines. The third is **offline capability**: although the assistant can use a hosted language model when one is configured, it ships with a complete rule-based fallback engine, ensuring that the application remains fully functional, private, and free to run

without any external dependency. Together these properties distinguish MindMate from comparable tools and align it with the responsible-use principles emphasised in the digital-psychiatry literature (Torous et al., 2021).

1.7 Technologies Used

The implementation draws on a focused, open stack of well-established technologies, each chosen for a specific role.

- **Python 3.12** is the implementation language, providing a mature ecosystem for data science and rapid application development.
- **Streamlit** provides the user interface, with multipage navigation, enabling a complete interactive web application to be built directly in Python.
- **scikit-learn** supplies the machine-learning components, K-Means clustering, Logistic Regression, standardisation, pipelines, and PCA, and is the foundation of the personalisation layer (Pedregosa et al., 2011).
- **VADER**, together with a custom emotion-keyword lexicon, performs lexicon-based sentiment analysis on free-text input, which is robust on short, informal messages (Hutto & Gilbert, 2014).
- The **OpenAI Chat Completions API** powers the conversational replies when a key is available, while a fully **offline rule-based fallback engine** guarantees that the assistant always works without one.
- **pandas**, **NumPy**, and **SciPy** handle data manipulation and statistical analysis, and **plotly** and **matplotlib** provide in-app and report visualisations respectively.
- **SQLite**, accessed through Python's `sqlite3` module, provides lightweight local persistence, while **joblib** serialises trained models and **python-dotenv** manages configuration.
- **pytest** underpins the testing strategy, with a suite of automated tests covering safety, sentiment, screening, persistence, and the machine-learning components.

1.8 Organisation of the Report

The remainder of this report is organised as follows. **Chapter 2** reviews the relevant literature on student mental health, conversational agents, sentiment analysis, and machine learning in mental health, situating the project within existing research. **Chapter 3** sets out the system analysis, including the development methodology, functional and non-functional requirements, and feasibility considerations. **Chapter 4** presents the system design, covering the architecture, database schema, data-flow and class structures, and the machine-learning models, together with their experimental results. **Chapter 5** describes the implementation, detailing the major modules, the safety layer, and the personalisation pipeline. **Chapter 6** explains the testing strategy and reports the outcomes of the test suite. **Chapter 7** presents the evaluation study, including the synthetic pilot, the statistical analysis, and the effect sizes obtained. **Chapter 8** discusses the findings, their significance, the ethical considerations, and the limitations of the work. **Chapter 9** outlines directions for future enhancement. **Chapter 10** concludes the report by summarising the contributions of the project against its stated objectives.

Chapter 2: Literature Review and System Study

2.1 Introduction

The purpose of this chapter is to situate MindMate within the existing body of scholarly and practical work on student mental health, digital interventions, and the data-science techniques that underpin them. A literature review in an applied data-science project serves two complementary functions. First, it establishes the clinical and social need for the system by surveying the evidence on the scale and consequences of psychological distress among students. Second, it audits the technical landscape, conversational agents, machine-learning methods, sentiment analysis, validated screening instruments, and recommender systems, so that the design choices made later in the report can be defended against, or distinguished from, prior art. A review that only celebrated previous achievements would be of little use; the more valuable contribution is a critical reading that exposes the strengths and weaknesses of existing systems and, in doing so, isolates the precise gap that the present work attempts to fill.

Accordingly, this chapter is organised to move from the problem to the methods and finally to the gap. Sections 2.2 establishes the human problem. Sections 2.3 to 2.7 examine the four methodological pillars on which MindMate rests: conversational agents, machine learning in mental health, sentiment analysis and natural language processing, validated screening instruments, and personalisation through recommender systems and clustering. Section 2.8 draws the threads together in a comparative analysis of representative existing systems, presented in a structured table. Section 2.9 reviews the software-engineering process models that informed how the system was actually built, and Section 2.10 articulates the research gap that motivates the entire project. Throughout, the discussion is deliberately critical: every technique surveyed is examined not only for what it makes possible but also for the limitations and risks it carries, because a mental-health system that ignores those risks would be irresponsible regardless of its technical elegance.

It is worth stating at the outset that MindMate is conceived as a *first-line wellbeing support tool* for students and not as a diagnostic or therapeutic instrument. This framing matters for the review, because much of the literature concerns clinical populations and clinically validated interventions, whereas the present system occupies the lighter-touch, preventive and self-management space. Reading the clinical evidence with that distinction in mind allows the project to borrow what is transferable, validated questionnaires, evidence-based coping strategies, safety practices, without overclaiming a clinical status it does not possess.

2.2 Mental Health Among Students

The case for a student-focused mental-health tool begins with the scale of the problem. The World Health Organization's *World mental health report* (World Health Organization, 2022) documents that mental-health conditions remain among the leading contributors to the global burden of disease, that a substantial proportion of these conditions emerge before early adulthood, and that the gap between the number of people who need care and the number who actually receive it remains wide across most of the world. The report is particularly emphatic that early intervention and accessible, community-based support are more effective and more humane than waiting for conditions to escalate to the point of crisis. For a population of students, who are typically in the very age band in which disorders first appear, this argues strongly for low-barrier, preventive support delivered where young people already are, namely on their phones and laptops.

The student population is not merely a smaller version of the general adult population; it carries a distinct and elevated risk profile. The WHO World Mental Health Surveys International College Student Project, reported by Auerbach et al. (2018), gathered data across multiple countries and found that a large fraction of first-year university students screened positive for at least one common mental disorder, with anxiety and mood disorders especially prevalent. Crucially, the same study found that only a minority of affected students sought treatment, a treatment gap that the authors attributed to a combination of stigma, a preference for self-reliance, low perceived need, and practical barriers such as cost and availability. The picture that emerges is of a population that is simultaneously high in need and low in help-seeking, precisely the combination that a discreet, self-directed digital tool is well placed to address.

These international findings are corroborated and enriched by campus-level research. Eisenberg et al. (2013), drawing on survey data from numerous American colleges and universities, documented substantial variation in the prevalence of depression, anxiety, and suicidal ideation both across student subgroups and across institutions. Their work is important for two reasons. First, it confirms that distress is common rather than exceptional within student bodies. Second, by demonstrating that prevalence varies systematically with characteristics such as financial stress, identity, and campus context, it implies that one-size-fits-all support is likely to be suboptimal. A student carrying acute financial pressure and poor sleep is not in the same situation as one experiencing transient examination anxiety, and an effective tool should be able to recognise and respond to that difference. This insight, that heterogeneity within the student population is the norm, directly motivates the personalisation component of MindMate described in later chapters.

Taken together, the literature establishes three facts that frame the entire project. Psychological distress among students is highly prevalent; most affected students do not access formal care; and the needs within the population are heterogeneous. A system that is accessible, non-stigmatising, and adaptive to individual circumstances therefore addresses a genuine and well-evidenced need rather than a hypothetical one. At the same time, the same literature counsels humility: the consequences of distress can be severe, including self-harm and suicide, which means that any tool entering this space must treat user safety as its first responsibility rather than an afterthought.

2.3 Conversational Agents in Mental Health

The idea of a computer program that converses about a person's emotional state is far older than the modern smartphone. The lineage begins with ELIZA, the program described by Weizenbaum (1966), which simulated a Rogerian psychotherapist by means of simple pattern-matching and template substitution. ELIZA had no understanding of language in any meaningful sense; it detected keywords and reflected the user's own statements back as questions. Yet Weizenbaum observed, to his own discomfort, that people readily attributed understanding and empathy to the program and were willing to disclose personal feelings to it. This phenomenon, later called the ELIZA effect, is foundational for the whole field, because it demonstrates both the promise and the peril of conversational mental-health software. The promise is that even a simple, transparent, rule-based system can elicit disclosure and provide a sense of being heard. The peril is that users may over-trust a system that does not, and cannot, understand them, which places a heavy ethical burden on designers to avoid deception and to build in safeguards.

The decades since ELIZA have seen conversational agents grow from curiosities into clinically evaluated interventions, and the contemporary evidence base, though still young, is encouraging. Fitzpatrick et al. (2017) conducted a randomised controlled trial of Woebot, a text-based agent that delivers principles of cognitive behavioural therapy (CBT) to young adults. Over a two-week period, participants who used Woebot showed significantly greater reductions in symptoms of depression than a control group given an information resource, and engagement with the agent was high. Around the same time, Fulmer et al. (2018) reported a randomised controlled trial of Tess, an integrative agent, and likewise found significant reductions in symptoms of depression and anxiety among college students relative to a control condition. Inkster et al. (2018), studying the empathy-driven agent Wysa with real-world usage data rather than a controlled trial, found that users who engaged more heavily with the agent reported greater improvements in mood, while also cautioning that observational data of this kind cannot establish causation. These three studies are frequently cited together because they represent some of the first credible

evidence that automated conversational agents can produce measurable, if modest, benefits in non-clinical and sub-clinical populations.

Broader reviews place these individual results in context and temper any uncritical enthusiasm. Vaidyam et al. (2019) reviewed the psychiatric landscape of chatbots and concluded that, although early findings are promising and patient satisfaction is often high, the evidence base remains small, the trials are typically short, and standardised outcome measurement and long-term follow-up are largely absent. Abd-Alrazaq et al. (2019), in a scoping review of the features of mental-health chatbots, mapped the field and found wide variation in design, in the underlying dialogue technology, the therapeutic framework employed, the platforms used, and, importantly, the extent to which safety and crisis handling were addressed. Provoost et al. (2017), reviewing embodied conversational agents in clinical psychology, similarly observed that while such agents are technically feasible and often well received, rigorous effectiveness evidence is thin and methodological quality is uneven. The consistent message across these reviews is that conversational agents are a promising but immature technology whose benefits are real but bounded, and whose risks, particularly around safety, privacy, and the management of user expectations, are not yet uniformly well handled.

Two limitations recur in this literature and are directly relevant to MindMate. The first is the question of safety: a conversational agent will inevitably encounter users in acute distress, and the reviews note that not all systems handle disclosures of self-harm or suicidal ideation adequately. The second is opacity. Many modern agents are built on large, proprietary language models whose internal reasoning is neither transparent to the user nor inspectable by clinicians or researchers, which complicates trust, accountability, and clinical governance. MindMate responds to both concerns deliberately: it screens every message for crisis risk before generating any reply, and it favours transparent, rule-based and inspectable components, using a generative language model only as an optional enhancement with a fully functional offline fallback. In this sense the project is informed not only by what the conversational-agent literature has achieved but, more importantly, by the gaps and cautions that that literature itself identifies.

2.4 Machine Learning in Mental Health

Beyond dialogue, the second technical pillar of the project is machine learning, which has become increasingly central to digital mental health over the past decade. Shatte et al. (2019) conducted a scoping review of machine-learning methods and applications in mental health and identified four broad clusters of use: detection and diagnosis, prognosis and treatment prediction, public-health and surveillance applications, and clinical administration. Their review found that supervised learning dominates the field, that a wide range of data sources is exploited, from questionnaire responses and electronic health records to social-media text and smartphone sensor streams, and that the most common goal is the prediction of a mental-health status or outcome from observable features. This taxonomy is helpful for positioning MindMate, whose stress classifier is a straightforward instance of supervised prediction from a small set of interpretable features, and whose student segmenter is an example of the unsupervised pattern-discovery that Shatte et al. note is comparatively less common but valuable for stratifying populations.

D'Alfonso (2020) offers a more reflective overview of artificial intelligence in mental health, emphasising that the techniques are tools in service of care rather than replacements for human judgement. The paper highlights both the opportunities, scalability, continuous monitoring, the ability to surface patterns that clinicians might miss, and the hazards, including the risk of biased or unrepresentative training data, the danger of over-reliance on automated outputs, and the ethical importance of transparency and informed consent. For a student project this is a salutary reminder that a model achieving high accuracy is not, by itself, evidence of a good system; the way the model's outputs are framed, explained, and acted upon matters at least as much.

Torous et al. (2021) survey the broader field of digital psychiatry, encompassing apps, social media, chatbots, and virtual reality, and provide perhaps the most balanced account of where the field stands. They note genuine progress and a growing evidence base, but they are forthright about persistent problems: many consumer mental-health apps lack any evidence of effectiveness, data-privacy practices are frequently weak, engagement tends to decay rapidly over time, and the gap between research prototypes and clinically governed products remains wide. Their call is for digital tools that are evidence-informed, transparent, privacy-respecting, and designed with the realities of sustained engagement in mind. These criteria read almost as a specification, and MindMate is consciously designed against them: it uses evidence-based instruments and strategies, exposes its reasoning rather than hiding it, stores data locally to respect privacy, and treats safety as foundational. The literature on machine learning in mental health, in short, supplies not only the methods the project uses but also the normative standards against which the project should be

judged.

2.5 Sentiment Analysis and NLP

A central requirement of MindMate is the ability to read the emotional tone of free text, a chat message or a mood-tracker note, and respond appropriately. This capability falls within sentiment analysis, a subfield of natural language processing concerned with the computational treatment of opinion, affect, and subjectivity in text. Liu (2012) provides the canonical treatment of the area, framing sentiment analysis as the task of identifying and quantifying the polarity (positive, negative, or neutral) and, in richer formulations, the specific emotions expressed in language. Liu distinguishes between document-level, sentence-level, and aspect-level analysis, and surveys the two dominant methodological families: lexicon-based approaches, which score text by reference to dictionaries of affect-bearing words, and machine-learning approaches, which learn to classify sentiment from labelled examples. Liu is careful to enumerate the difficulties that make the task hard in practice, negation, intensification, sarcasm, domain dependence, and the brevity and informality of much user-generated text.

For short, informal, emotionally charged text of the kind a student might type into a wellbeing app, a particularly well-suited tool is VADER (Valence Aware Dictionary and sEntiment Reasoner), introduced by Hutto and Gilbert (2014). VADER is a lexicon-and-rule-based model tuned specifically for social-media language. Beyond a curated lexicon of words rated for valence, it incorporates heuristics that handle several of the very phenomena Liu identifies as challenging: it accounts for punctuation (an exclamation mark intensifies sentiment), capitalisation, degree modifiers such as *very* or *slightly*, contrastive conjunctions such as *but*, and negation. Importantly for this project, VADER requires no training data and produces a transparent, deterministic compound score in the range from minus one to plus one. Hutto and Gilbert reported that VADER's performance on social-media text was competitive with, and in some cases exceeded, that of individual human raters, while remaining computationally trivial and fully interpretable.

The choice of a lexicon-based method over a trained or neural classifier is a deliberate one that the literature supports for this context. A rule-based model is transparent, one can always explain why a given sentence received its score, it runs instantly and offline without any external service, and it does not require a labelled training corpus that a small student project could not realistically assemble. These properties align with the transparency and offline-capability goals that recur throughout the present work. The limitations are equally clear and are acknowledged honestly: a lexicon model cannot capture deep context, can be misled by sarcasm or unusual phrasing, and offers only coarse polarity rather than fine-grained clinical insight. For this reason MindMate treats sentiment as one signal

among several and supplements VADER with a small custom emotion-keyword lexicon to detect specific affective states such as anxiety, sadness, or anger, which feed the personalisation logic. Sentiment analysis, in this design, is a lightweight, explainable sensor rather than a clinical judgement.

2.6 Screening Instruments (PHQ-9 and GAD-7)

Whereas sentiment analysis offers a soft, indirect reading of emotional state, structured self-report questionnaires provide a standardised, validated, and clinically meaningful measure. MindMate incorporates two of the most widely used brief screening instruments in the world: the Patient Health Questionnaire-9 (PHQ-9) for depressive symptoms and the Generalized Anxiety Disorder-7 (GAD-7) for anxiety. The PHQ-9 was validated by Kroenke et al. (2001), who showed that its nine items, each mapping onto a diagnostic criterion for major depressive disorder, form a reliable and valid measure of depression severity. Respondents rate how often they have been bothered by each symptom over the preceding two weeks on a four-point scale from zero (*not at all*) to three (*nearly every day*), yielding a total score from zero to twenty-seven. Kroenke et al. proposed and validated severity bands, minimal, mild, moderate, moderately severe, and severe, that allow a raw score to be interpreted meaningfully. Of particular relevance to a safety-conscious system, the ninth item of the PHQ-9 directly asks about thoughts of self-harm or being better off dead, which is why MindMate treats any non-zero response to that item as a trigger for a self-harm safeguard rather than a mere number contributing to a total.

The GAD-7 was developed and validated by Spitzer et al. (2006) as a brief, parallel measure for generalized anxiety. Its seven items use the same two-week reference period and the same zero-to-three response scale, producing a total from zero to twenty-one, again with published cut-points distinguishing minimal, mild, moderate, and severe anxiety. Spitzer et al. demonstrated good reliability, validity, and the ability of the instrument to discriminate anxiety from other conditions, and the GAD-7 has since become a standard screening tool in both research and routine care. The two instruments are frequently administered together because depression and anxiety commonly co-occur, and using both gives a more rounded picture of a person's psychological state than either alone.

It is worth noting that these are not the only validated instruments available. The Depression Anxiety Stress Scales (DASS), described by Lovibond and Lovibond (1995), offer an alternative that separately measures depression, anxiety, and stress and that was constructed to distinguish these related but distinct negative emotional states. The DASS is a respected instrument and its three-factor structure is conceptually attractive for a project that explicitly models stress. The decision to adopt the PHQ-9 and GAD-7 rather than the DASS rests on several practical considerations: the PHQ-9 and GAD-7 are shorter and

quicker to complete, which matters for engagement; they are extremely widely recognised, easing comparison with the chatbot trials reviewed earlier; and the PHQ-9's explicit self-harm item provides a built-in safety hook that aligns directly with the project's safety-first philosophy. Two important caveats apply to the use of any such instrument in MindMate. First, these are *screening* tools, not diagnostic ones; an elevated score indicates that further assessment by a qualified professional may be warranted, not that a disorder is present. Second, self-report scores must always be interpreted with care, since they can be influenced by mood at the moment of completion, social desirability, and honest self-knowledge. MindMate therefore presents screening results as informative indicators framed by appropriate guidance, not as verdicts.

2.7 Personalisation and Recommender Systems

The third objective of the project is to personalise support, and the relevant literature here is that of recommender systems together with the clustering and clinical-psychology foundations that underpin a meaningful, individualised response. Ricci et al. (2015), in the standard handbook on the subject, define recommender systems as software that suggests items likely to be of interest or value to a particular user, and they survey the principal families of technique: content-based filtering, which recommends items similar to those a user has liked before; collaborative filtering, which exploits patterns of preference across many users; and knowledge-based and hybrid approaches that combine multiple signals and explicit domain rules. The handbook also stresses two themes that are central to MindMate's design. The first is the importance of context: the *right* recommendation depends not only on who the user is but on their current situation. The second is the value of explanation; a recommendation that the user understands and trusts is more likely to be acted upon than an opaque one, and the literature treats explainability as a feature that improves both effectiveness and user confidence.

In a mental-health context, however, classic collaborative filtering is poorly suited, because the small user base of a student project cannot supply the dense preference data such methods require, and because borrowing one person's coping preferences for another raises ethical and clinical concerns. MindMate therefore adopts a transparent hybrid approach in which coping strategies are scored against several explicit, interpretable signals, the emotion detected in the user's text, the support segment to which the user belongs, and the level of stress predicted by the classifier, with each recommendation accompanied by a human-readable rationale. This design directly reflects the handbook's emphasis on context-awareness and explanation while sidestepping the data and ethical limitations of collaborative methods.

The segmentation that feeds the personalisation rests on clustering, and specifically on the K-Means algorithm. The method originates with MacQueen (1967), who introduced a procedure for partitioning a set of multivariate observations into a fixed number of groups by iteratively assigning points to the nearest cluster centre and recomputing those centres to minimise within-cluster variance. K-Means is unsupervised, requiring no labelled data, which suits the discovery of natural groupings among students based on their wellbeing features; it is also efficient and widely understood, though it requires the number of clusters to be chosen in advance and is sensitive to feature scaling, both of which the implementation addresses. The clinical substance of the recommendations, finally, draws on cognitive therapy as articulated by Beck (1979), whose foundational work established that emotional disorders are maintained by patterns of distorted thinking and that systematically identifying and reframing those patterns can relieve distress. Several of the coping strategies MindMate offers, most explicitly the thought-reframing exercise, but also behavioural activation and structured worry time, are direct descendants of this cognitive-behavioural tradition. Personalisation in MindMate is therefore not merely a matter of statistical matching; it connects an interpretable model of the individual to evidence-based psychological techniques, with the chain of reasoning kept visible throughout.

2.8 Comparative Analysis of Existing Systems

Having surveyed the methods individually, it is instructive to compare a representative set of conversational mental-health systems directly, in order to see where MindMate sits relative to prior art. The systems chosen are ELIZA, as the historical baseline; Woebot, Wysa, and Tess, as the most prominent and best-evidenced contemporary agents; and MindMate itself. They are compared along four dimensions that the preceding review identified as critical: the underlying technical approach, the degree of personalisation, the handling of user safety, and the openness or transparency of the system. The comparison is summarised in the table below.

Table 2.1: Comparative Analysis of Existing Systems

System	Approach	Personalisation	Safety handling	Open/Transparent
ELIZA	Pattern-matching and template substitution; no real language understanding	None, identical behaviour for all users	None, no crisis detection or escalation	Conceptually transparent but never designed for real-world support
Woebot	Scripted, CBT-based dialogue trees; RCT-evidenced	Limited, adapts within scripted flows	Built-in crisis signposting to external resources	Proprietary; underlying logic not openly inspectable
Wysa	Empathy-driven AI agent with CBT and mindfulness content	Moderate, tailors content to expressed mood	Crisis detection with signposting	Proprietary; commercial product
Tess	Integrative psychological AI delivering multiple modalities	Moderate, adapts modality to user input	Includes escalation pathways	Proprietary; commercial product
MindMate	Hybrid: transparent ML (sentiment, K-Means segmentation, stress classification) plus optional generative model with offline rule-based fallback	High and explainable, emotion, segment, and predicted stress drive scored recommendations with rationale	Rule-based crisis screening before every reply; three risk levels; conservative by design	Fully transparent and open methodology; inspectable components; offline-capable

Several patterns are visible in this comparison. ELIZA, for all its historical importance, offers no personalisation, no safety handling, and was never intended as a genuine support tool; it is included to mark the distance the field has travelled. The three modern commercial agents, Woebot, Wysa, and Tess, represent a clear advance: each has some empirical evidence behind it, each personalises to a degree, and each incorporates at least basic crisis signposting. Their common limitation, from the standpoint of this project, is opacity. They are proprietary products whose internal logic, data practices, and decision rules are not openly inspectable, which is precisely the concern that Torous et al. (2021) raise about much of the digital-mental-health marketplace. A student, a clinician, or a researcher cannot easily examine why one of these systems said what it said.

MindMate's distinctive position is therefore not that it does something the commercial systems cannot, but that it combines a set of properties that are rarely found together. It treats safety as the first action on every message rather than as an occasional signpost; it makes its personalisation fully explainable, exposing the segment, the predicted stress, the six features, and the recommendation rationale on a dedicated insights page; and it is designed to work entirely offline through a rule-based fallback, so that it never depends on the availability or expense of an external service. None of these properties is individually

novel, but their combination in a transparent, open, student-focused tool is uncommon, and it is this combination that the comparative analysis brings into focus and that Section 2.10 frames as the research gap.

2.9 Software Development Models

A literature review for an engineering project would be incomplete without considering how the system is built, not merely what it does. Software-engineering scholarship offers several process models, and the two most relevant poles are the plan-driven waterfall model and the family of agile, iterative approaches. Sommerville (2016) describes the waterfall model as a sequential process in which requirements, design, implementation, integration, and maintenance follow one another in fixed phases, with each phase signed off before the next begins. Its strengths are discipline, documentation, and predictability, which make it suitable for large projects with stable, well-understood requirements. Its well-known weakness is rigidity: because real requirements rarely freeze at the outset, the cost of accommodating change late in a waterfall project is high, and problems discovered during implementation can force expensive backtracking.

Agile and iterative models were developed in response to precisely this rigidity. Sommerville (2016) and Pressman and Maxim (2014) both characterise iterative development as a process of building the system in small increments, each delivering working functionality, with frequent opportunities to review, test, and adjust. Rather than attempting to specify everything in advance, an iterative team evolves the requirements and the design together through repeated cycles. The advantages are responsiveness to change, early and continuous validation, and the early visibility of working software; the trade-offs are a lighter documentation burden and a need for discipline to prevent scope drift. Pressman and Maxim emphasise that the choice of process should be governed by the characteristics of the project, its size, the stability of its requirements, the experience of the team, and the consequences of error, rather than by dogma.

MindMate was developed using an iterative, incremental approach rather than a strict waterfall, and the reasons follow directly from these characteristics. The project is the work of a single student exploring a domain in which requirements could not be fully known in advance: it was not clear at the outset exactly how the sentiment signal, the clustering, the stress classifier, and the recommender would best fit together, nor how the safety layer should be tuned, and these questions could only be answered by building and testing successive versions. An iterative approach allowed each component, sentiment analysis, segmentation, classification, the recommender, the safety screen, and the user interface, to be implemented, validated against tests, and refined before the next was layered on. This is reflected in the project's substantial automated test suite, which embodies the

continuous-validation ethos that iterative development depends upon, and in the layered architecture that emerged from successive refinement rather than from a single up-front design. At the same time, certain plan-driven practices were retained where they added value, most notably clear documentation of requirements and design and the use of validated, pre-specified instruments and metrics, producing in effect a pragmatic hybrid. This balance, iterative in construction, disciplined in evaluation and documentation, matches the guidance of both Sommerville (2016) and Pressman and Maxim (2014) that the process should fit the project rather than the reverse.

2.10 Research Gap

The preceding review supports a clear and specific conclusion about what is missing from the current landscape, and it is this gap that MindMate is designed to address. The need is not in doubt: distress among students is highly prevalent, most affected students do not seek formal help, and the consequences of unaddressed distress can be severe (World Health Organization, 2022; Auerbach et al., 2018; Eisenberg et al., 2013). The methods are largely available: conversational agents can elicit disclosure and deliver evidence-based content (Fitzpatrick et al., 2017; Fulmer et al., 2018), machine learning can stratify populations and predict outcomes from interpretable features (Shatte et al., 2019; D'Alfonso, 2020), lightweight sentiment analysis can read emotional tone transparently (Hutto & Gilbert, 2014), validated questionnaires can quantify symptom severity (Kroenke et al., 2001; Spitzer et al., 2006), and hybrid recommender techniques grounded in cognitive therapy can deliver explainable, individualised support (Ricci et al., 2015; Beck, 1979).

What the review also makes clear, however, is that these strengths rarely come packaged together in a way that is simultaneously transparent, safety-first, offline-capable, and genuinely personalised for students. The reviews of the field repeatedly flag opacity, weak data-privacy practices, inconsistent safety handling, and dependence on external services as recurring shortcomings of existing tools (Vaidyam et al., 2019; Torous et al., 2021; Abd-Alrazaq et al., 2019). The most prominent commercial agents are effective but proprietary and cloud-dependent; the historical and experimental systems are transparent but unsafe or unpersonalised; and very few tools expose their reasoning to the people they serve.

The research gap that MindMate addresses can therefore be stated precisely. It is the absence of a *transparent, safety-first, offline-capable, and personalised first-line wellbeing tool for students*, one that screens every interaction for crisis risk before responding, that explains its personalisation rather than hiding it behind a proprietary model, that functions fully without any external service so that it respects privacy and remains available

regardless of connectivity or cost, and that grounds its recommendations in validated instruments and evidence-based psychological techniques. MindMate does not claim to be a clinical instrument, nor to replace professional care, and its evaluation in this report rests on synthetic data that demonstrates a methodology rather than proving an effect on real people. Within those honest limits, the system occupies the under-served space that the literature exposes: accessible, explainable, and safe self-directed support for a population that is large, at risk, and reluctant to seek help through conventional channels. The chapters that follow set out how this gap is translated into a concrete system design, implementation, and evaluation.

Chapter 3: System Analysis

3.1 Introduction

System analysis is the phase in which the broad aims of a software project are translated into a precise, verifiable description of what the system must do and the constraints under which it must operate. Before any line of application code is written, it is essential to establish a shared understanding of the problem, the people the system serves, and the qualities the finished product must exhibit. This chapter performs that work for MindMate, the AI-powered mental health assistant for students described in the preceding chapters. It elaborates the requirements gathered from the project brief and the wider literature, classifies them into functional, non-functional and user requirements, examines the feasibility of building the system within the available resources, and models its behaviour through use-case and data-flow analysis. The chapter closes with a statement of the hardware and software needed to develop and run the application.

The analysis follows the structured approach to requirements engineering advocated in the software engineering literature, in which requirements are first elicited, then specified in a form that is unambiguous and traceable, and finally validated against the goals of the project (Sommerville, 2016; Pressman & Maxim, 2014). MindMate is a postgraduate dissertation project rather than a commercial product, so the analysis is scoped accordingly: the emphasis is on demonstrating a complete, internally consistent and reproducible system rather than on satisfying the full demands of a regulated clinical deployment. Throughout, the project's three objectives act as the anchor for the requirements, to design and develop an AI mental-health assistant for students, to personalise its support using machine-learning algorithms, and to evaluate its effectiveness on mental-health and academic outcomes. Every requirement recorded below can be traced back to one of these objectives.

A second concern that shapes the analysis is safety. Because the application addresses psychological wellbeing, the system must behave responsibly when a user expresses distress. The need for a conservative, rule-based crisis-detection layer that runs before any conversational reply is therefore treated not as an optional feature but as a first-class requirement. This reflects the cautionary findings of reviews of mental-health chatbots, which note that conversational agents can offer genuine support but must be designed with explicit safeguards and clear boundaries on what they claim to do (Vaidyam et al., 2019; Abd-Alrazaq et al., 2019).

3.2 Requirement Analysis

Requirement analysis for MindMate drew on three complementary sources. The first was the project brief supplied by the industry partner, which fixed the high-level objectives, the student-focused scope and the expectation of a privacy-respecting, locally runnable application. The second was a study of comparable digital mental-health tools and the academic literature surrounding them; systems such as Woebot, Wysa and Tess illustrate how conversational agents deliver structured, evidence-informed support to young adults (Fitzpatrick et al., 2017; Inkster et al., 2018; Fulmer et al., 2018), while broader reviews map the features that such tools typically provide and the safeguards they require (D'Alfonso, 2020; Torous et al., 2021). The third source was the body of clinical instruments and computational methods the system was to incorporate, namely the PHQ-9 and GAD-7 screening questionnaires (Kroenke et al., 2001; Spitzer et al., 2006), rule-based sentiment analysis (Hutto & Gilbert, 2014), and the machine-learning techniques provided by scikit-learn (Pedregosa et al., 2011).

From these sources a set of needs emerged. Students require a confidential, always-available space to express how they feel and to receive a supportive, non-judgemental response. They benefit from being able to record their mood over time and to see patterns in it. They need access to validated self-assessment of depression and anxiety symptoms, presented with appropriate care. They are better served when the support offered is tailored to their individual situation rather than generic, which motivates the machine-learning personalisation layer. They must be protected by an explicit safety mechanism for moments of crisis. Finally, the project itself requires the ability to evaluate whether the assistant makes a measurable difference, which introduces a requirement for structured data capture and an evaluation methodology.

These needs were then separated into functional requirements, which describe the concrete services the system delivers; non-functional requirements, which describe the quality attributes the system must possess; and user requirements, which describe the experience from the perspective of the student who uses it. The remaining sections of this chapter present each category in turn.

3.3 Functional Requirements

The functional requirements specify the observable behaviour of MindMate, the inputs it accepts, the processing it performs and the outputs it returns. They are enumerated in the table below and elaborated in the prose that follows. Each requirement carries a unique identifier (FR-1 to FR-12) so that it can be referenced from the design and testing chapters and traced to the use cases of Section 3.7.

Table 3.1: Functional Requirements

ID	Requirement	Description
FR-1	Conversational support	The system shall provide an empathetic chat interface in which a student can type messages and receive supportive, context-aware replies, using an external language model when an API key is present and a fully offline rule-based engine otherwise.
FR-2	Crisis-safety screening	The system shall screen every incoming user message for crisis risk before any reply is generated, classify the risk as none, elevated or crisis, and respond to a crisis with a validating safe message and helpline information rather than attempting counselling.
FR-3	Mood logging	The system shall allow a student to record a daily entry comprising mood (1-5), energy (1-5), sleep hours and an optional free-text note, and to persist these entries for later review.
FR-4	Sentiment analysis	The system shall compute a sentiment score for free-text notes and chat messages using VADER together with an emotion-keyword lexicon, and store the result alongside the text.
FR-5	PHQ-9 depression screening	The system shall administer the nine-item PHQ-9 questionnaire on the standard 0-3 response scale, compute the total score and map it to the published severity band.
FR-6	GAD-7 anxiety screening	The system shall administer the seven-item GAD-7 questionnaire on the standard 0-3 response scale, compute the total score and map it to the published severity band.
FR-7	Self-harm safeguard	The system shall flag a non-zero response to PHQ-9 item 9 as a self-harm risk and surface a supportive safeguard message and helplines.
FR-8	Student segmentation	The system shall assign a student to one of four support segments using a trained K-Means model over the six canonical wellbeing features.

ID	Requirement	Description
FR-9	Stress prediction	The system shall predict a student's stress level (Low, Moderate, High) and its class probabilities using a trained logistic-regression classifier over the six features.
FR-10	Personalised recommendations	The system shall rank and present coping strategies using a transparent hybrid recommender that combines detected emotion, segment preference and stress level, and shall return an explanatory rationale for each recommendation.
FR-11	Resources and psychoeducation	The system shall provide a resources page listing Indian and global helplines, psychoeducational content and the full coping-strategy library.
FR-12	Data persistence	The system shall persist users, mood logs, chat messages, screening results and recommendation logs in a local SQLite database, with referential integrity between a user and their related records.

The conversational requirement (FR-1) is the heart of the application. The chat must feel responsive and human, yet remain useful even with no internet connection or paid API access; the dual-engine design satisfies this by defaulting to a deterministic offline engine and upgrading to the external model only when a key is configured. Crucially, FR-2 takes precedence over FR-1 at runtime: no reply is composed until the crisis screen has classified the message, so a student in danger is never met with an ordinary chatbot answer. The screening is deliberately conservative, treating a false alarm as far less costly than a missed warning, in keeping with the duty-of-care principles emphasised in reviews of mental-health chatbots (Vaidyam et al., 2019).

The self-tracking and assessment requirements (FR-3 to FR-7) give the system its clinical grounding. Mood logging produces the longitudinal record that drives the trend charts and the engagement feature, while sentiment analysis enriches both notes and chat with an affective signal (Hutto & Gilbert, 2014). The PHQ-9 and GAD-7 instruments are implemented exactly to their published scoring rules and severity thresholds (Kroenke et al., 2001; Spitzer et al., 2006), and the item-9 safeguard (FR-7) mirrors the crisis layer's protective philosophy at the point of screening.

The machine-learning requirements (FR-8 to FR-10) deliver the personalisation that distinguishes MindMate from a generic chatbot. Segmentation and stress prediction operate over the six canonical features, average mood, average sleep hours, PHQ-9 score, GAD-7

score, average sentiment and engagement, and their outputs feed the recommender, which must remain explainable so that the support offered can always be justified to the student. Finally, the resources and persistence requirements (FR-11 and FR-12) ensure that help is one click away at all times and that every interaction is recorded in a structured form suitable for the project's evaluation.

3.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes that determine how well the system performs its functions rather than which functions it performs. For a sensitive application of this kind they are as important as the functional behaviour itself.

Performance. The application must feel immediate during interactive use. Screening of a chat message, computation of sentiment, segment assignment and stress prediction must all complete within a fraction of a second on a standard laptop, since the underlying operations are lightweight rule evaluations and small scikit-learn model inferences over six features. The only operation with variable latency is a call to the external language model, which depends on network conditions; the offline engine therefore acts as a predictable fallback that keeps the interface responsive at all times.

Usability. The interface is intended for students who are not technical specialists and who may be using the tool while distressed. It must therefore be clean, calm and self-explanatory, with a small number of clearly labelled pages, plain-language prompts, and supportive rather than clinical tone. Streamlit's multipage navigation provides a consistent layout, and visualisations are used to make trends and machine-learning outputs intuitive rather than intimidating.

Privacy and security. Because the data is psychologically sensitive, privacy is a defining constraint. The system is designed to run locally, storing all personal data in a SQLite database on the user's own machine rather than transmitting it to a central server. The optional external language model is the only component that can send data off the device, and the application functions fully without it, so a privacy-conscious user can disable it entirely. No real identifying information is required to use the system, and all model development and evaluation in this project use clearly labelled synthetic data, avoiding any handling of real personal records.

Reliability. The system must behave predictably and must never fail in a way that leaves a distressed user without support. The offline engine guarantees that the chat always produces a reply; the crisis layer is rule-based and deterministic, so its protective behaviour does not depend on a probabilistic model; and a suite of automated tests verifies the critical paths. Reliability is reinforced by the safety design principle that errors should fail safe, for

example, ambiguous messages err toward flagging rather than ignoring risk.

Maintainability. The codebase is organised into clear layers, presentation, a service layer, an assistant package and a machine-learning package over a database, so that each concern can be modified in isolation. Trained models are serialised with joblib and loaded at runtime, decoupling model training from application logic. A body of automated unit and integration tests (described in the testing methodology) protects against regressions and documents the intended behaviour, supporting the maintainable-design guidance of the software engineering literature (Sommerville, 2016).

Portability. The application depends only on Python 3.12 and a set of open-source, cross-platform libraries, with no platform-specific components. It therefore runs unchanged on Windows, macOS and Linux, and can be hosted on any environment that supports a Python web process. SQLite, being a single self-contained file, makes the data layer trivially portable as well.

3.5 User Requirements

User requirements describe what the system must offer from the standpoint of the people who use it, expressed in their own terms rather than in technical language. The primary user of MindMate is a higher-education student, and a representative persona helps to keep the analysis grounded.

Persona. Consider "Aditi", a twenty-one-year-old undergraduate in her second year. She is managing a demanding course alongside part-time work, sleeps irregularly during assessment periods, and notices that her mood and concentration suffer when deadlines cluster. She is reluctant to approach campus counselling because of stigma and waiting times, and she values privacy. She owns a laptop and is comfortable with everyday apps but is not technical. From Aditi's perspective the system must let her: talk to a supportive listener at any hour without being judged; quickly note how she is feeling and later see whether things are improving or worsening; check in a structured way whether her low mood or worry has reached a level worth taking seriously; receive concrete, manageable suggestions tailored to her situation rather than generic advice; and reach professional help immediately if she ever feels unsafe. The prevalence of mental-health difficulties among students reported in international surveys underlines that Aditi is far from unusual (Auerbach et al., 2018; Eisenberg et al., 2013), and the World Health Organization's call to widen access to support (World Health Organization, 2022) motivates a tool that meets students where they already are.

From this persona the following user requirements are drawn. The student requires a private and confidential environment that does not demand personal identification. They

require an interface that is calming and easy to navigate even under stress. They require honesty about the tool's nature, it is a supportive companion and self-help aid, not a substitute for professional care or a diagnostic instrument, so that expectations are appropriately set. They require their data to remain under their control. And they require unambiguous, prominent routes to emergency help. These user requirements are deliberately satisfied by the functional and non-functional requirements above, ensuring traceability from human need to system behaviour.

3.6 Feasibility Study

A feasibility study examines whether the proposed system can realistically be built and operated given the available technology, budget and operating environment (Pressman & Maxim, 2014). For MindMate the study considers three dimensions.

3.6.1 Technical Feasibility

The project is technically feasible because every capability it requires is provided by mature, well-documented, open-source technology. The application is built in Python 3.12 with Streamlit for the user interface, scikit-learn for the machine-learning models, and VADER for sentiment analysis (Pedregosa et al., 2011; Hutto & Gilbert, 2014). These libraries are widely used, actively maintained and supported by extensive community knowledge, which substantially lowers the risk of encountering an insurmountable technical obstacle. The machine-learning components, K-Means clustering for segmentation (MacQueen, 1967; Lloyd, 1982) and multinomial logistic regression for stress classification, are standard algorithms with efficient implementations and modest computational demands, so they train and run comfortably on ordinary hardware. Data handling relies on pandas and numpy, persistence on the SQLite engine built into Python's standard library, and model serialisation on joblib, none of which introduces unusual complexity. The optional integration with an external language model is the only component that depends on a third-party service, and because a complete offline engine provides the same interface, the system carries no hard dependency on it. The presence of a passing automated test suite confirms that the intended behaviour is achievable in practice rather than only in principle. The project is therefore technically sound and well within the capability of the development environment.

3.6.2 Economic Feasibility

Economically the project is highly favourable, which is one of its deliberate design goals. The entire technology stack is free and open-source, so there are no licensing costs for the language, the web framework, the machine-learning libraries or the database. Development was carried out on hardware already available to the author, requiring no capital expenditure. The most significant potential running cost, fees for calls to a commercial language-model API, is avoided entirely by the offline rule-based engine, which allows the application to deliver its core conversational function at zero marginal cost. Where an external model is used, the lightweight default model keeps any expenditure minimal and entirely optional. Because the data is stored locally in a single SQLite file, there is no requirement for paid database hosting or cloud storage. The result is a system whose total cost of ownership is effectively nil for a student or institution, which is significant given that cost and access are recognised barriers to mental-health support (World Health Organization, 2022). The project is thus economically feasible with a wide margin.

3.6.3 Operational Feasibility

Operationally the system is straightforward to deploy and to use. Running it requires only a Python environment and the installation of the listed packages, after which a single command starts the Streamlit server and opens the application in a web browser. Because it runs locally and stores its own data, it needs no server administration, database management or network configuration for individual use; equally, because it is a standard Python web application, it can be hosted on any platform that supports such processes should wider access be desired. For the end user the operational burden is minimal: the interface is a familiar web page navigated through clearly labelled pages, demanding no training. The supportive, low-friction design and the privacy of local operation make adoption realistic for the student persona described earlier, who values confidentiality and ease of use. The system therefore fits comfortably into its intended operating context and is operationally feasible.

3.7 Use-Case Analysis

Use-case analysis models the system from the perspective of the goals that an external actor wishes to achieve through it. The single primary actor in MindMate is the Student, who interacts with all of the application's functions. A secondary, supporting actor is the external Language-Model Service, which the chat function may call upon when configured but on which the system does not depend. The use-case diagram below summarises the goals the student can pursue and their relationships to the system.

Figure: Use-Case Diagram

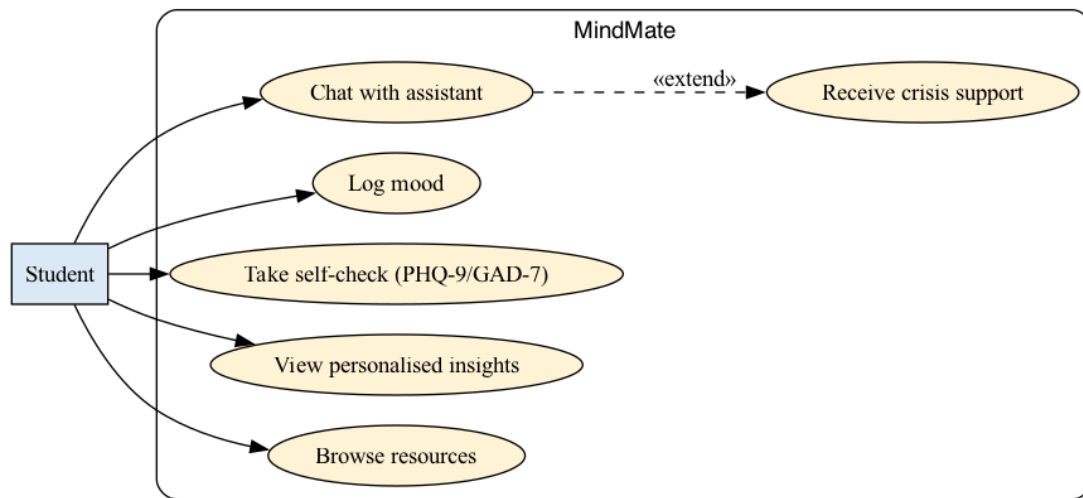


Figure 3.1: Use-case diagram of MindMate

The principal use cases are as follows.

- 1 **Chat with the assistant.** The student opens the chat page and exchanges messages with MindMate to obtain emotional support. Every message first passes through the crisis-screening use case; only then is a reply produced, either by the external service or by the offline engine. This is the most frequently exercised use case and the one that most directly delivers conversational support.
- 1 **Screen for crisis risk.** Invoked automatically for each chat message, this use case classifies risk as none, elevated or crisis. It is an included behaviour of the chat use case rather than something the student triggers explicitly, reflecting the safety-first runtime ordering. A crisis classification short-circuits the normal reply and presents a validating message together with helplines.
- 1 **Log mood.** The student records mood, energy, sleep hours and an optional note. The note is passed to the sentiment-analysis behaviour, and the entry is persisted for later review and for use as machine-learning features.
- 1 **Take a self-check.** The student completes the PHQ-9 or GAD-7 questionnaire, receives a total score and severity band, and, in the case of PHQ-9 item 9, may trigger the self-harm safeguard. The resulting scores are stored and feed into segmentation and stress prediction.
- 1 **View insights.** The student views their support segment, predicted stress level and its probabilities, the six feature values driving these outputs, and the rationale behind the recommendations. This use case makes the machine-learning reasoning transparent.

- 1 **Receive recommendations.** Drawing on detected emotion, segment and stress level, the system ranks coping strategies and presents them with an explanation. This behaviour is included by the chat and insights use cases.
- 1 **Access resources.** The student browses helplines, psychoeducation and the coping-strategy library. This use case is always available and is the destination to which the crisis and safeguard behaviours direct a user in need.

Together these use cases cover every functional requirement of Section 3.3, with FR-2 and FR-7 appearing as protective behaviours woven through the others rather than as standalone goals, an arrangement that captures the system's safety-first character.

3.8 Data-Flow Analysis

Data-flow analysis describes how information moves through the system, independent of the control structures that implement it. It complements the use-case view by focusing on the transformation of data rather than on user goals. The analysis is presented at two levels of abstraction.

The context, or Level 0, diagram treats the whole of MindMate as a single process and shows its boundary with the outside world. The sole external entity that initiates interaction is the Student, who supplies inputs, chat messages, mood entries and questionnaire responses, and receives outputs in the form of supportive replies, trends, scores, recommendations and resources. A second external entity, the optional Language-Model Service, exchanges prompt and completion data with the system only when configured. All persistent information is held in the local data store. The context diagram thus establishes the scope of the system and confirms that, apart from the optional model service, MindMate is a self-contained application that does not communicate with any external party.

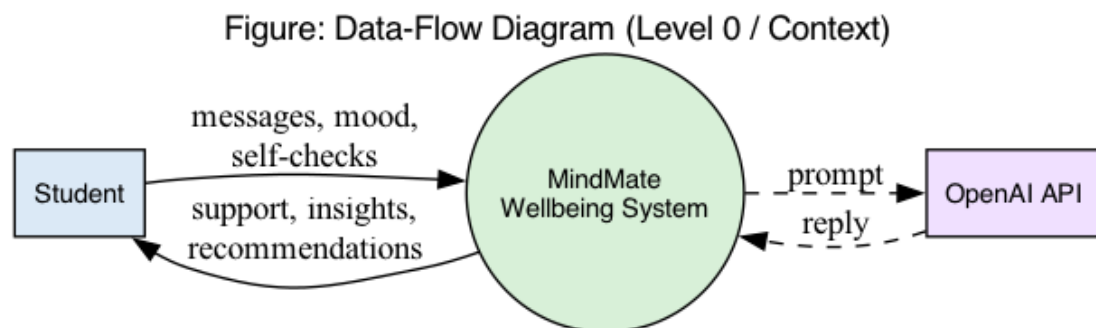


Figure 3.2: Data-flow diagram (Level 0 / context)

The Level 1 diagram decomposes the single process into the principal internal processes and shows the data stores that connect them. A chat message entering the system is first

directed to the safety-screening process, which produces a risk level; depending on that level, the message is either answered by the crisis-response process or passed to the reply-generation process, which composes a supportive answer and consults the recommendation process. A mood entry flows into the mood-logging process, where the sentiment-analysis process derives an affective score before the entry is written to the mood-log store. Questionnaire responses flow into the screening process, which computes scores and severity bands and writes them to the screening-results store. The personalisation processes, segmentation and stress prediction, read the accumulated features from the data stores and produce a segment and a stress prediction, which the insights and recommendation processes consume. Each process writes its outputs back to the appropriate store, so that the database accumulates a complete, structured record of the interaction. The mapping between these processes and the database tables, users, mood logs, chat messages, screening results and recommendation logs, is direct, and the cascade relationships between a user and their records preserve integrity as data flows in over time.

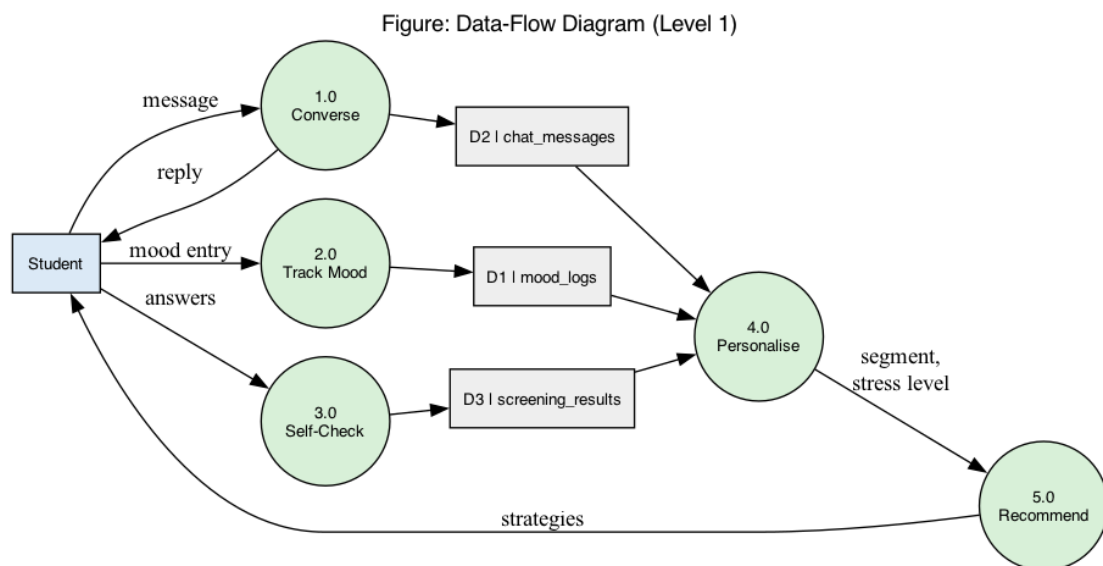


Figure 3.3: Data-flow diagram (Level 1)

Taken together, the two data-flow diagrams demonstrate that information passes through the system in a disciplined and traceable manner: every input is processed by an identifiable transformation, every output is derived from stored or computed data, and the safety screen sits squarely on the path of every chat message. This orderly flow underpins both the reliability of the application and the credibility of the evaluation that depends on the data it records.

3.9 System Requirements (Hardware/Software)

The final element of the analysis is a statement of the resources needed to develop, run and host MindMate. Because the system is deliberately lightweight, these requirements are modest.

Hardware. Development and execution require only a standard personal computer. A laptop with a dual-core processor, 8 GB of RAM and a few hundred megabytes of free disk space is more than sufficient, since the machine-learning models are small and operate over six features and a few hundred synthetic records. No graphics accelerator or specialised hardware is needed for either training or inference, and the SQLite database occupies negligible space. An internet connection is required only if the optional external language model is used; all other functions, including the offline chat engine, operate without one.

Software. The core software requirement is Python 3.12 together with the project's open-source dependencies: Streamlit for the web interface; scikit-learn for the K-Means and logistic-regression models, together with StandardScaler, Pipeline and PCA; vaderSentiment for sentiment analysis; pandas, numpy and scipy for data handling and statistics; plotly for in-application charts and matplotlib for report figures; joblib for model serialisation; python-dotenv for configuration; and SQLite, provided through Python's standard library, for persistence. The pytest framework supports the automated test suite and graphviz is used to render the structural diagrams. The application runs on any operating system that supports Python 3.12, Windows, macOS or Linux, and is accessed through a standard modern web browser. A code editor and the Git version-control system complete the development environment. This compact and entirely open-source software footprint is what makes the system portable, inexpensive and easy to reproduce, and it is consistent with the feasibility conclusions reached earlier in this chapter.

Chapter 4: System Design

4.1 Introduction

This chapter translates the requirements established in the preceding chapters into a concrete technical blueprint for MindMate, the AI-powered mental-health assistant for students. Where the earlier analysis described *what* the system must do, the present chapter describes *how* the system is organised internally so that those obligations can be met reliably, safely, and in a way that remains comprehensible to both maintainers and examiners. Design, in the sense used here, is the bridge between abstract requirement statements and the source code that ultimately realises them (Sommerville, 2016).

The design is presented through a series of complementary views. The structural view explains how the software is decomposed into layers and packages and how the principal classes relate to one another. The data view documents the persistent storage as a normalised relational schema together with its entity-relationship model. The behavioural view captures the dynamic interactions between components when a student sends a chat message, including the crisis-safety branch that is central to the application's responsible operation. Finally, the chapter addresses the user-interface design and the design of the machine-learning algorithms that personalise the assistant's behaviour. Taken together these views provide a single coherent account of the system that can be implemented, tested, and evaluated.

Throughout the chapter the description is kept faithful to the implemented system rather than to an idealised one. MindMate is a Streamlit web application written in Python 3.12, persisting data in SQLite and drawing on scikit-learn for its machine-learning components. The crisis-safety layer, the offline fallback chat engine, and the transparent recommender are emphasised because they embody the design priorities that distinguish a responsible student wellbeing tool from a generic chatbot.

4.2 Design Objectives and Principles

The design of MindMate is governed by a small set of objectives that follow directly from the project's three aims: to build an assistant for students, to personalise its support with machine learning, and to evaluate its effectiveness. From these aims the following design objectives were derived.

- **Safety first.** No component may generate or display a reply to a potentially distressed student before that message has been screened for crisis risk. The architecture must make it structurally difficult to bypass this check.

- **Separation of concerns.** Presentation, application logic, domain intelligence, and persistence must be cleanly separated so that each can be developed, reasoned about, and tested in isolation.
- **Testability without the user interface.** The core behaviour of the system must be exercisable by automated tests that do not depend on the Streamlit runtime, which is why application logic is concentrated in a thin service layer rather than scattered across page scripts.
- **Transparency.** Because the system makes inferences about a student's psychological state, every machine-derived output, including segment assignment, predicted stress level, and recommendations, must be explainable rather than opaque.
- **Graceful degradation.** The application must remain fully usable when optional external dependencies, in particular the language-model API, are unavailable; an offline rule-based engine guarantees this.
- **Privacy by design.** Data is stored locally in a single SQLite database, the schema is minimal, and deletion of a user cascades to all of their records.

These objectives are realised through established design principles. A layered architecture enforces separation of concerns and constrains the direction of dependencies, a recognised structural style for information systems of this kind (Sommerville, 2016; Pressman & Maxim, 2014). High cohesion within modules and low coupling between them are pursued so that, for example, the sentiment component can be modified without disturbing the database layer. Information hiding is applied so that no part of the application issues raw SQL; all persistence flows through small, well-named helper functions. Finally, a conservative bias is deliberately encoded into the safety design: a false positive, in which a benign message is treated as concerning, is considered far less costly than a false negative, in which genuine distress is missed.

4.3 System Architecture

MindMate adopts a layered architecture in which each layer offers services to the layer above and depends only on the layer below. This style localises change, clarifies responsibilities, and supports incremental testing, and it is widely recommended for applications that combine a user interface with persistent data and analytical logic (Sommerville, 2016). Figure 4.1 shows the four principal layers and the optional external dependency.

Figure: MindMate system architecture

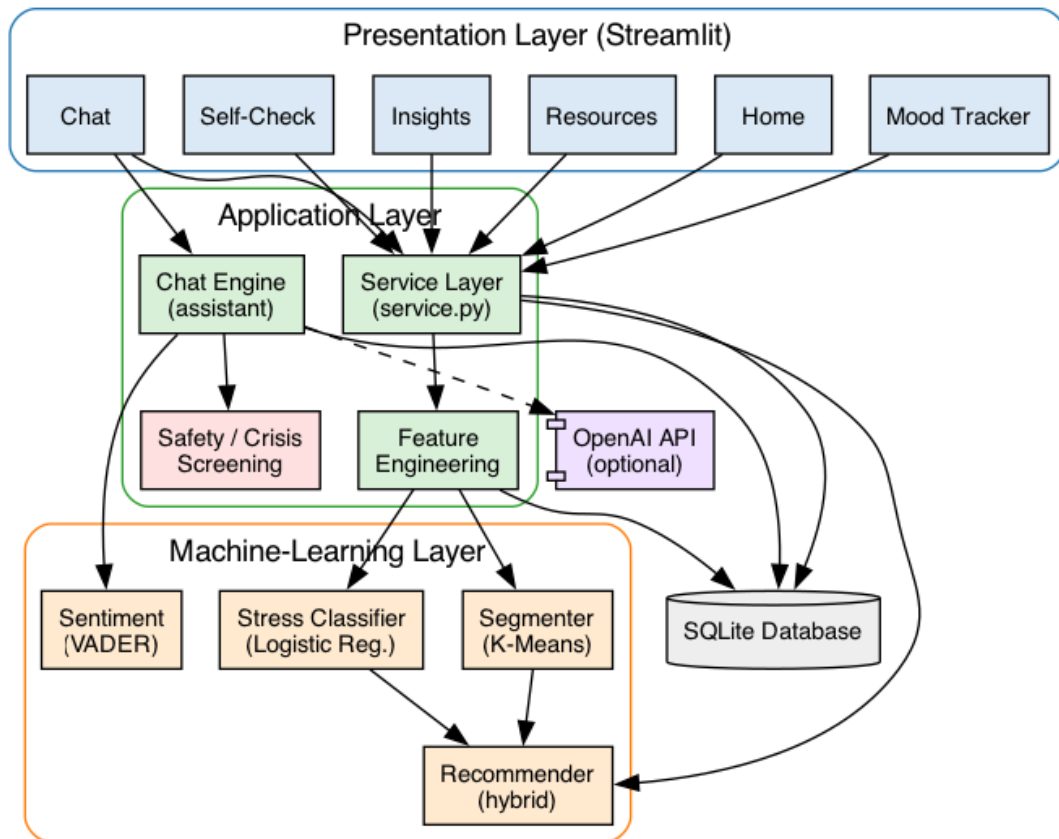


Figure 4.1: MindMate layered system architecture

The **presentation layer** comprises the Streamlit user interface. It is organised as a multipage application assembled through `st.navigation`, with six pages: Home, Talk to MindMate, Mood Tracker, Self-Check, Insights, and Resources. The pages are deliberately thin; they are responsible for rendering widgets, collecting input, and displaying results, but they delegate all decision-making to the layer beneath. Keeping the pages thin is what allows the application's behaviour to be tested independently of the browser-based runtime.

The **service layer** is implemented in `service.py` and acts as the single point of contact between the user interface and the underlying intelligence and storage. It exposes a compact set of operations such as computing a user's live machine-learning profile, calculating the 0-100 wellness index for the dashboard headline, and producing personalised recommendations. By centralising orchestration here, the design ensures that the various lower-level modules are composed in one place and that the user interface never needs to know how a recommendation or a stress prediction is actually produced.

Beneath the service layer sit two sibling packages that together constitute the **domain-intelligence layer**. The *assistant* package contains the conversational components: the chat engine, the crisis-safety module, and the prompt templates. The *machine-learning*

package contains the sentiment analyser, the K-Means student segmenter (personalization.py), the Logistic-Regression stress classifier (stress.py), and the hybrid recommender (recommender.py). These packages encapsulate the analytical heart of the system. Crucially, the safety module within the assistant package is invoked before the chat engine, so that risk screening cannot be skipped by the normal control flow.

The **data layer** is implemented in database.py and wraps a single SQLite database accessed through Python's sqlite3 module. It defines the schema, manages connections with foreign-key enforcement enabled, and provides typed helper functions for every read and write. No higher layer issues SQL directly.

Finally, the **OpenAI Chat Completions API** is shown as an optional external dependency rather than a core layer. When an API key is configured, the chat engine uses the language model (by default gpt-4o-mini) to generate empathetic replies; when no key is present, an offline rule-based engine produces a reply instead. This arrangement satisfies the graceful-degradation objective: the loss of an external service downgrades the quality of conversation but never disables the application.

4.4 Class Design

The class design refines the layered architecture into the concrete types that carry the system's state and behaviour. The principal classes are small, focused, and, where they represent the result of a computation, implemented as immutable data carriers so that results can be passed safely between layers. Figure 4.2 depicts the core classes and their associations.

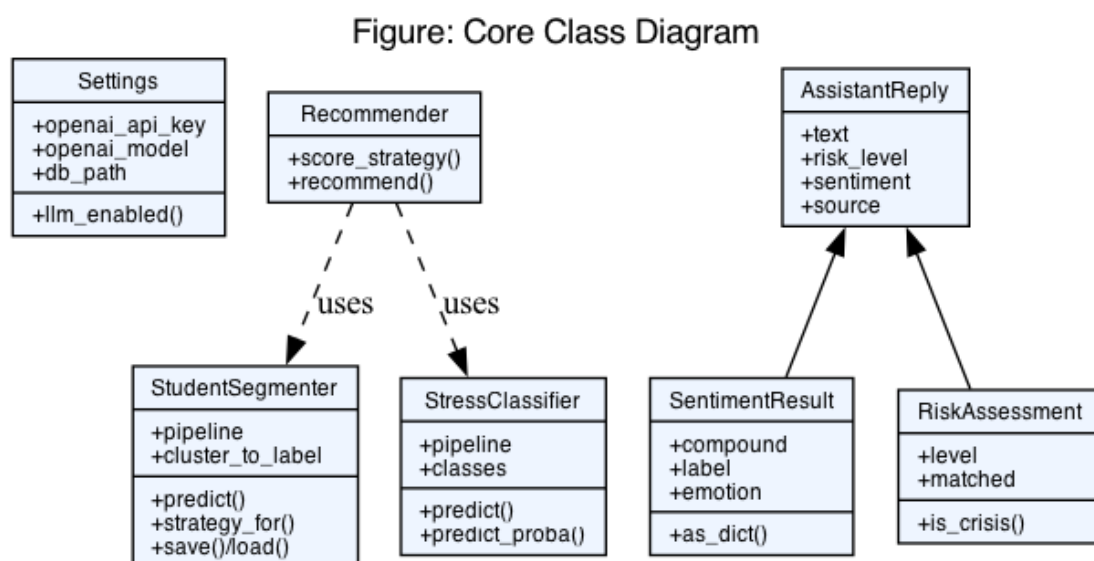


Figure 4.2: Core class diagram

Settings is the central configuration object. It holds runtime parameters such as the database path, the model file locations, and the optional API key, and it is loaded once from the environment using python-dotenv. A single shared instance (SETTINGS) is consumed by the database and machine-learning modules, which keeps configuration in one place and avoids hard-coded paths.

StudentSegmenter wraps a fitted scikit-learn pipeline together with a mapping from raw cluster identifiers to human-readable labels. Its `predict` method accepts a dictionary of the six canonical features, assembles them into the fixed feature order, runs the pipeline, and returns the segment label associated with the resulting cluster. The class also offers `predict_many` for batch prediction, `strategy_for` to retrieve the support strategy attached to a label, and `save/load` methods that serialise the pipeline and its label mapping with joblib. By bundling the pipeline and the mapping into one serialisable object, the design guarantees that a loaded model always carries its interpretation with it.

StressClassifier plays the analogous role for supervised prediction. It encapsulates a multinomial Logistic-Regression model and exposes `predict`, which returns a categorical stress level (Low, Moderate, or High), and `predict_proba`, which returns the class probability distribution used by the Insights page to display the model's confidence. Like the segmenter, it supports joblib persistence so that training and inference are cleanly separated.

SentimentResult is an immutable value object returned by the sentiment analyser. It captures the VADER compound score together with the dominant emotion inferred from a custom emotion-keyword lexicon (Hutto & Gilbert, 2014). Representing the analysis as a single typed result, rather than as a loose tuple, makes the downstream code that consumes it self-documenting.

RiskAssessment is the output of the crisis-safety module. It records the detected risk level, one of none, elevated, or crisis, and any safe message and helpline information that must accompany the reply. Because risk screening governs whether a normal reply is generated at all, this small object has outsized importance in the control flow.

AssistantReply is the value object returned by the chat engine to the service and presentation layers. It bundles the textual reply with the metadata needed for storage and display, including the risk level and sentiment, so that a single object fully describes the assistant's turn.

Recommender is realised through the recommender module, which scores the nine coping strategies against the current emotional and segment signals and returns ranked recommendations, each accompanied by a human-readable rationale. The accompanying

recommendation objects carry a title, a category, and the explanatory rationale string, which together support the transparency objective.

The associations among these classes follow the dependency direction of the architecture. The service layer holds references to a segmenter and a classifier, obtains a `SentimentResult` from the sentiment analyser and a `RiskAssessment` from the safety module, and assembles an `AssistantReply`. None of the value objects depend on the service or presentation layers, which keeps the lower layers reusable and independently testable.

4.5 Database Design

4.5.1 Entity-Relationship Model

Persistent data is modelled relationally and stored in a single SQLite database. The schema is deliberately normalised so that it can be documented cleanly as an entity-relationship model and so that each fact is recorded in exactly one place. There are five entities: a central **User** entity and four dependent entities that record the different kinds of activity a student generates over time, namely **Mood Log**, **Chat Message**, **Screening Result**, and **Recommendation Log**.

The model is organised around a single one-to-many relationship pattern. Each user may own many mood logs, many chat messages, many screening results, and many recommendation logs, while each of those records belongs to exactly one user. The User entity is therefore the parent in four separate one-to-many associations, and the four activity entities are the children. Figure 4.3 presents this structure.

Figure: Entity-Relationship diagram

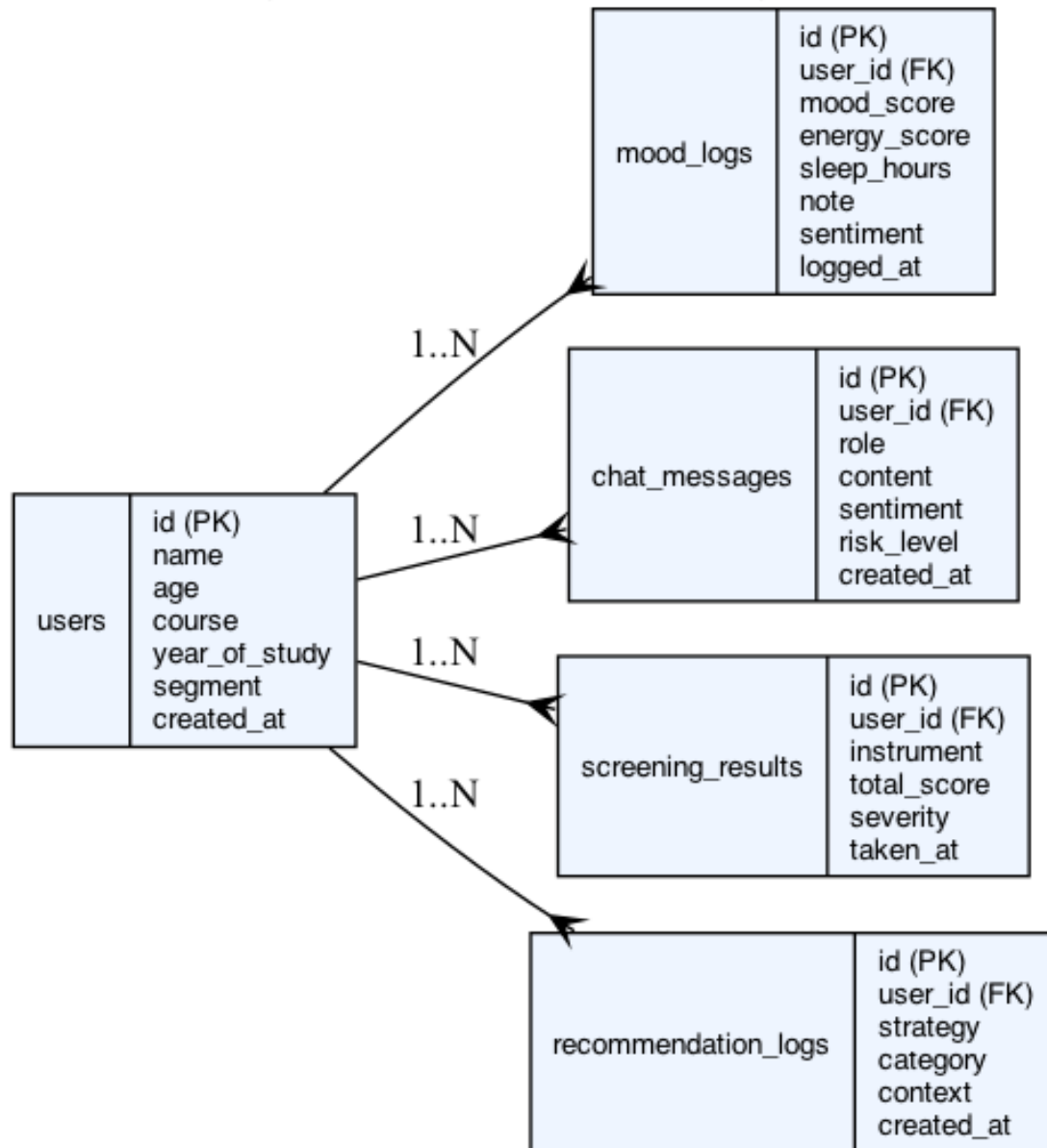


Figure 4.3: Entity-relationship diagram

Referential integrity is enforced at the database level. Every child table declares a foreign key referencing `users(id)` with an `ON DELETE CASCADE` rule, and foreign-key checking is switched on for every connection through the `PRAGMA foreign_keys = ON` statement. The practical consequence is that deleting a user automatically and atomically removes all of that user's mood logs, chat messages, screening results, and recommendation logs. This supports the privacy-by-design objective directly: a single deletion erases an individual's entire footprint, leaving no orphaned records behind.

4.5.2 Schema and Table Structures

The five tables are created by an idempotent schema script that uses `CREATE TABLE IF NOT EXISTS`, so initialising an existing database is harmless. Every table uses an auto-incrementing integer primary key, and every record carries an ISO-8601 timestamp recorded at insertion time. Timestamps are stored as text, which keeps the schema portable and chronologically sortable. The following tables document each relation as implemented in `database.py`.

users

Table 4.1: Schema and Table Structures

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for the student
name	TEXT	NOT NULL	Display name of the student
age	INTEGER	,	Age in years (optional)
course	TEXT	,	Programme of study (optional)
year_of_study	INTEGER	,	Current year of study (optional)
segment	TEXT	,	Most recently assigned ML segment label
created_at	TEXT	NOT NULL	ISO-8601 timestamp of account creation

mood_logs

Table 4.2: Schema and Table Structures

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for the mood entry
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Owning student
mood_score	INTEGER	NOT NULL	Self-reported mood, 1 (very low) to 5 (very good)
energy_score	INTEGER	,	Self-reported energy, 1 to 5
sleep_hours	REAL	,	Reported hours of sleep
note	TEXT	,	Free-text note for the day
sentiment	REAL	,	VADER compound sentiment of the note
logged_at	TEXT	NOT NULL	ISO-8601 timestamp of the log

chat_messages

Table 4.3: Schema and Table Structures

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for the message
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Owning student
role	TEXT	NOT NULL	Author of the message, user or assistant
content	TEXT	NOT NULL	Message text
sentiment	REAL	,	Sentiment score of the message
risk_level	TEXT	,	Screening outcome: none , elevated , or crisis
created_at	TEXT	NOT NULL	ISO-8601 timestamp of the message

screening_results

Table 4.4: Schema and Table Structures

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for the screening
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Owning student
instrument	TEXT	NOT NULL	Questionnaire used, PHQ-9 or GAD-7
total_score	INTEGER	NOT NULL	Summed item score
severity	TEXT	NOT NULL	Severity band derived from published cut-offs
taken_at	TEXT	NOT NULL	ISO-8601 timestamp of completion

recommendation_logs

Table 4.5: Schema and Table Structures

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for the log entry
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE	Owning student
strategy	TEXT	NOT NULL	Title of the recommended coping strategy
category	TEXT	,	Category of the strategy
context	TEXT	,	Signals behind the recommendation (emotion, segment)
created_at	TEXT	NOT NULL	ISO-8601 timestamp of the recommendation

The screening table reflects the standard instruments used by the Self-Check page, the nine-item PHQ-9 for depressive symptoms (Kroenke et al., 2001) and the seven-item GAD-7 for anxiety (Spitzer et al., 2006), with each result reduced to a total score and a severity band. The context column of the recommendation log preserves the signals that produced a recommendation, which is what later allows the Insights page to reconstruct and display the rationale behind past suggestions. Access to every table is mediated by dedicated helper functions, for instance `add_mood_log`, `add_chat_message`, and `get_screening_results`, so that the rest of the application interacts with the database through a small, audited surface rather than ad-hoc queries.

4.6 Behavioural Design (Sequence and Activity)

The structural and data designs describe the static shape of the system; the behavioural design describes how the components collaborate over time. The most important interaction in MindMate is the handling of a chat message, because it is here that the safety obligation, the sentiment analysis, the personalisation, and the persistence all come together in a single ordered flow.

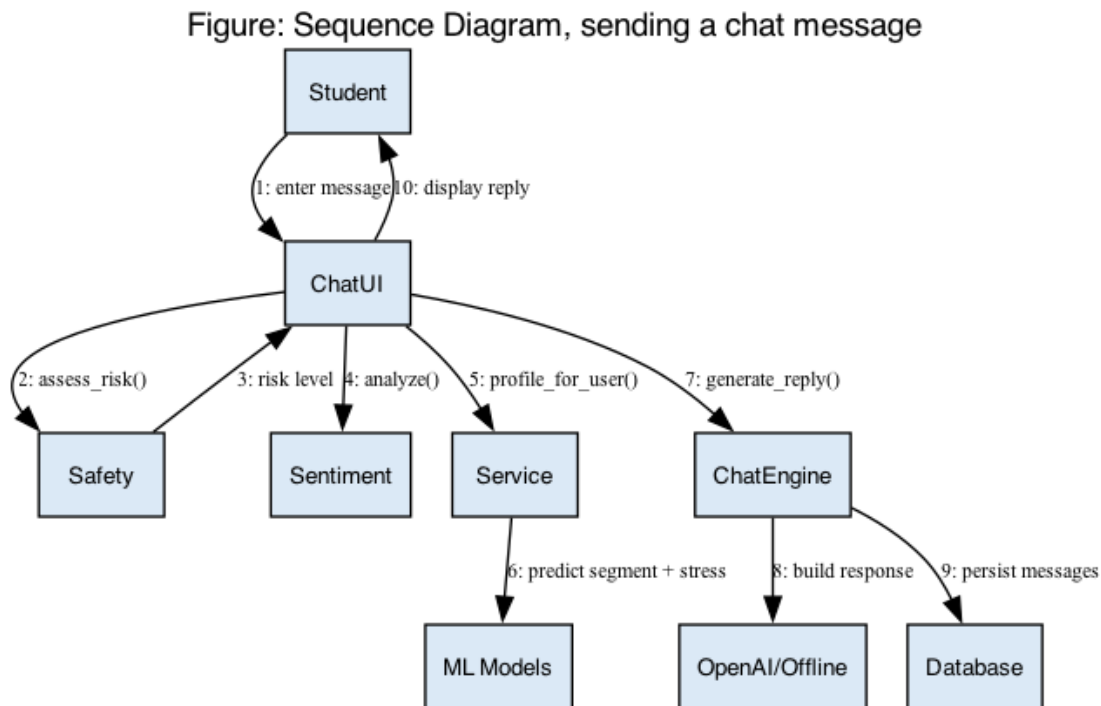


Figure 4.4: Sequence diagram for sending a chat message

Figure 4.4 shows the sequence of method calls. The interaction begins when the student submits text on the Talk to MindMate page. The page passes the message to the service layer, which first invokes the crisis-safety module to obtain a RiskAssessment. This screening happens before any reply is generated, which is the structural guarantee that distress cannot be answered with a casual response. The service then requests a SentimentResult from the sentiment analyser, both to inform the reply and to be stored with the message. With the risk and sentiment known, the service delegates reply generation to the chat engine. When an API key is configured, the engine calls the OpenAI Chat Completions API; otherwise it falls back to the offline rule-based engine. The engine returns an AssistantReply, which the service persists by writing both the student's message and the assistant's response to the chat_messages table, each annotated with its sentiment and risk level. The completed reply is finally returned to the page for display.

The same behaviour is captured from a control-flow perspective in the activity diagram of Figure 4.5, which makes the crisis branch explicit.

Figure: Activity Diagram, message handling with safety branch

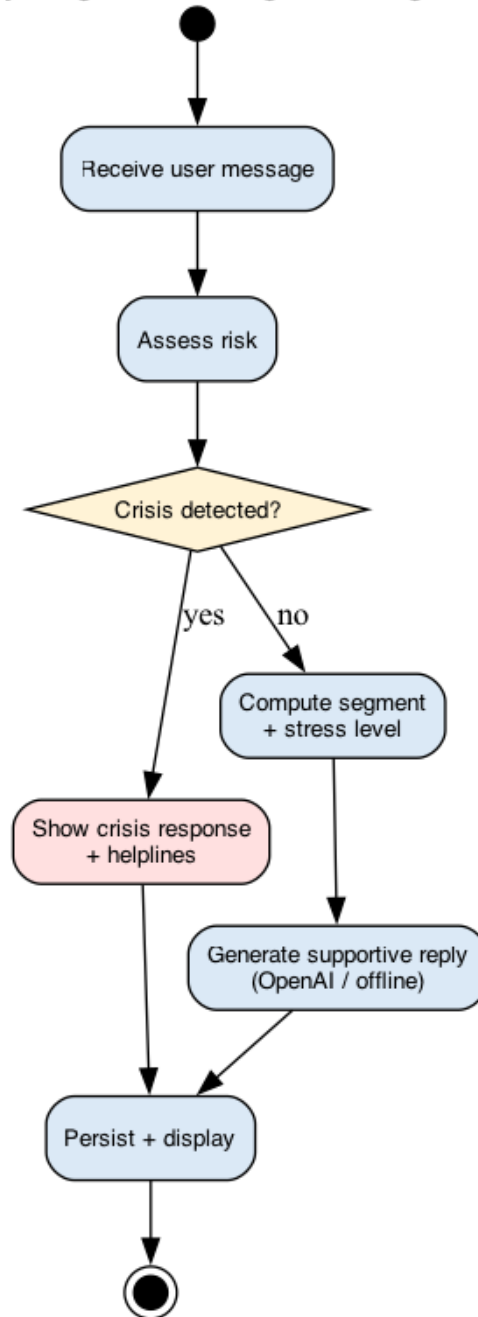


Figure 4.5: Activity diagram for message handling with the safety branch

After the message is received, the safety module classifies its risk level, and the flow branches on the result. If the level is **crisis**, the system does not attempt to counsel. Instead it returns a brief, validating message accompanied by crisis helplines, including iCall, the Vandrevalla Foundation, Tele-MANAS, and AASRA, and the interaction terminates

without invoking the conversational engine. If the level is **elevated**, a normal empathetic reply is generated and then augmented with a gentle nudge toward supporting resources. If the level is **none**, the engine produces an ordinary supportive reply. In every branch the messages and their metadata are written to the database before the reply is shown. This conservative branching, in which any sign of acute risk short-circuits the ordinary conversational path, embodies the principle that a missed crisis is a far worse outcome than an over-cautious one. The rule-based nature of this safeguard is acknowledged as a limitation: it is intentionally simple and explainable, not a clinical risk-assessment instrument.

4.7 User-Interface Design

The user interface is built with Streamlit and organised as a multipage application assembled through `st.navigation`. Six pages divide the application's functionality into clear, task-oriented areas, each accessible from a persistent sidebar.

- **Home** presents the dashboard headline: a 0-100 wellness index, the student's current support segment, the predicted stress level, a concise support plan, and the top personalised recommendations. It is designed to give an at-a-glance, encouraging summary rather than to overwhelm the visitor with detail.
- **Talk to MindMate** is the conversational page where the safety-screened chat takes place. It is the most carefully designed screen because every message passes through the crisis-safety layer before a reply appears.
- **Mood Tracker** allows daily logging of mood and energy on a one-to-five scale, sleep hours, and a free-text note, and it renders trend charts so that the student can observe patterns over time.
- **Self-Check** administers the PHQ-9 and GAD-7 questionnaires using the standard 0-3 response scale and published severity bands, with the ninth PHQ-9 item triggering a self-harm safeguard.
- **Insights** exposes the machine-learning internals transparently, showing the assigned segment, the stress probabilities, the six underlying features, and the rationale behind recommendations.
- **Resources** gathers Indian and global helplines, psychoeducation, and the full coping-strategy library in one reliably reachable place.

The visual design adopts a calming tone appropriate to a wellbeing context: a restrained, soft colour palette, generous spacing, and plain, supportive language rather than clinical jargon. Interactive charts are rendered with Plotly so that students can explore their own trends. Accessibility was a design consideration throughout. The interface relies on simple,

linear layouts that read well on smaller screens, uses descriptive labels on every input widget, avoids conveying meaning through colour alone by pairing colour with text labels for severity and risk, and keeps reading level low so that the content is approachable during periods of distress. Crisis information, in particular, is presented in clear, high-contrast text and is never hidden behind additional interaction.

4.8 Algorithm and Model Design

The personalisation that distinguishes MindMate rests on three designed algorithms: an unsupervised segmenter, a supervised stress classifier, and a transparent hybrid recommender. All three operate on the same six canonical features, computed for each student in a fixed order: average mood, average sleep hours, PHQ-9 score, GAD-7 score, average sentiment, and engagement. Fixing this feature order is a deliberate design choice, because the order must remain stable for the serialised models to be applied correctly at inference time.

Student segmentation with K-Means. Segmentation is performed by a scikit-learn Pipeline (Pedregosa et al., 2011) that chains a StandardScaler to a KMeans estimator. Standardisation is essential here because the six features occupy very different numerical ranges, from a one-to-five mood scale to a 0-27 PHQ-9 score; without scaling, the distance computation at the heart of K-Means would be dominated by the widest-ranged feature (Géron, 2019). The clustering uses $k = 4$, with ten random initialisations and a fixed random seed of 42 to guarantee reproducibility (MacQueen, 1967; Lloyd, 1982). A central difficulty with K-Means is that the cluster identifiers it produces are arbitrary, so a raw cluster number carries no meaning. The design solves this with a labelling step that, after fitting, inverts the scaling to bring each cluster centroid back into original feature units and then ranks the centroids on a composite wellbeing index in which higher mood, sleep, and sentiment are favourable while higher PHQ-9 and GAD-7 scores are unfavourable. The clusters are then assigned ordered, meaningful labels, from **Thriving** at the top of the wellbeing ranking through **Coping** and **At-Risk** to **High-Need** at the bottom. On the synthetic training population of 600 students the four segments contain 198 At-Risk, 176 Coping, 119 Thriving, and 107 High-Need students, with a silhouette score of 0.209, a modest but positive indication of structure (Rousseeuw, 1987). Each label carries a tailored support strategy that the recommender later consults.

Stress classification with Logistic Regression. The supervised component is a multinomial Logistic-Regression classifier that maps the same six features to one of three stress levels: Low, Moderate, or High. Logistic Regression was chosen for its interpretability and its native ability to output calibrated class probabilities, which the Insights page surfaces directly so that students see not only a predicted level but the

confidence behind it. The model is evaluated on a stratified 25% hold-out of 150 samples, on which it attains an accuracy of 0.900 and a macro-averaged F1 score of 0.889, with the strongest per-class performance on the High class ($F1 = 0.945$) and the weakest on the Moderate class ($F1 = 0.819$), the latter being the hardest because it lies between the two extremes. The trained model is persisted with joblib so that inference incurs no retraining cost.

Hybrid transparent recommender. Recommendations are produced by a hybrid scoring algorithm rather than an opaque model, in keeping with the transparency objective and the broader literature on recommender design (Ricci et al., 2015). Each of the nine coping strategies, which range from 4-7-8 Breathing and 5-4-3-2-1 Grounding to Thought Reframing drawn from cognitive behavioural therapy (Beck, 1979), is scored on three combined signals. A match between the strategy and the student's detected emotion contributes the largest weight (2.0); alignment with the student's segment preference contributes a moderate weight (1.0); and a stress-driven de-escalation boost of up to 1.5 raises calming strategies for students under high stress. The strategies are then ranked by total score and the top suggestions returned. Critically, every recommendation is accompanied by a rationale string explaining why it was chosen, and the context behind each suggestion is recorded in the recommendation log, so that no recommendation is ever presented as an unexplained verdict. This design makes the system's personalisation legible to the student, which is both an ethical requirement in a mental-health setting and a practical aid to evaluation.

Together these three algorithms convert a student's recent activity into a segment, a stress estimate, and an ordered set of explainable suggestions, and they do so deterministically and reproducibly. Their design reflects the project's guiding commitments throughout: personalisation that is genuinely tailored, intelligence that remains transparent, and an honest acknowledgement that the models are trained on synthetic data and are intended to demonstrate a methodology rather than to serve as a clinical instrument.

Chapter 5: System Implementation

5.1 Introduction

This chapter documents how the design presented in Chapter 4 was translated into working software. Where the previous chapter dealt with architecture, data flow and the entity-relationship model in the abstract, the focus here is concrete: the development environment, the layout of the codebase, the database, and the individual machine-learning and conversational modules that together make up MindMate. For each module I describe the key algorithm, quote a short, representative excerpt of the real source code, and then explain the logic in prose so that the reader can follow the reasoning behind each design decision rather than simply reading the listing.

Two themes recur throughout the implementation and are worth stating at the outset. The first is **graceful degradation**: the application is designed to work fully even when no external service is available. The conversational core can call the OpenAI Chat Completions API when a key is supplied, but it falls back to an entirely local, rule-based engine otherwise, so the system never breaks and never incurs a cost simply to demonstrate it. The second is **privacy by design**: all user data is held in a local SQLite database, no secrets are written into source code, and the large-language-model integration is strictly optional. These principles shaped many of the smaller choices described below.

A further commitment that runs through the whole project is **reproducibility**. Every stochastic step in model training is seeded with the fixed value 42, the dependency versions are pinned, and the synthetic datasets are regenerated deterministically by scripts under version control. Anyone who clones the repository and runs the training script should obtain exactly the metrics reported in Chapter 6.

5.2 Development Environment and Tools

MindMate was developed in **Python 3.12**, chosen for its mature data-science ecosystem and first-class support across all the libraries the project depends on. Development followed standard Python practice: an isolated virtual environment created with the built-in `venv` module, with all third-party packages installed through `pip` from a pinned `requirements.txt`. Pinning exact versions (for example `scikit-learn==1.5.2`, `streamlit==1.39.0`, `vaderSentiment==3.3.2`) is deliberate, it guarantees that the numerical results and the user interface behave identically on any machine, which is essential for an academic submission that must be reproducible.

The principal tools and their roles are as follows:

- **Streamlit (1.39.0)**, the web-application framework that renders the multipage user interface, using the modern `st.navigation/st.Page` API.
- **scikit-learn (1.5.2)**, the machine-learning library providing `KMeans`, `LogisticRegression`, `StandardScaler`, `Pipeline` and `PCA` (Pedregosa et al., 2011).
- **vaderSentiment (3.3.2)**, the VADER rule-based sentiment analyser used to score the affect of free text (Hutto & Gilbert, 2014).
- **pandas, numpy and scipy**, for data manipulation, numerical arrays and the statistical tests used in the evaluation.
- **plotly** for interactive in-app charts and **matplotlib** for the static figures included in this report.
- **SQLite**, accessed through Python's standard-library `sqlite3` module, for persistence.
- **joblib (1.4.2)** for serialising the trained model pipelines to disk.
- **python-dotenv (1.0.1)** for loading configuration from a local `.env` file into environment variables.
- **pytest (8.3.3)** for the automated test suite (34 tests, all passing).
- **graphviz** for generating the structural diagrams referenced in earlier chapters.

The choice of Python and scikit-learn for the analytical core reflects the conventional, well-documented workflow recommended in standard texts on applied machine learning (Géron, 2019). Keeping the model-building stack identical to the one used everywhere in the data-science community also lowers the barrier for future maintainers.

5.3 Project Structure

The codebase is organised into clearly separated packages, each corresponding to one architectural layer described in Chapter 4. The top level contains the Streamlit entry point `app.py`, the pinned `requirements.txt`, a `.env.example` template, configuration files, and directories for data, models, scripts, tests and the report. The application logic lives under `src/`:

- `src/config.py`, central configuration resolved from environment variables.
- `src/database.py`, the SQLite persistence layer.
- `src/content.py`, static content (coping strategies, helpline resources).
- `src/features.py`, `src/screening.py`, `src/service.py`, feature assembly, PHQ-9/GAD-7 scoring, and the service layer that mediates between the UI and the lower layers.

- src/ml/, the machine-learning package: sentiment.py, personalization.py, stress.py, recommender.py.
- src/assistant/, the conversational package: chat_engine.py, safety.py, prompts.py.
- src/ui/, one module per Streamlit page (home, chat, mood, screening_page, insights, resources).

Offline tooling lives in scripts/ (generate_data.py, train_models.py, make_figures.py, make_diagrams.py, analyze_evaluation.py), trained artefacts in models/artifacts/, and the test suite in tests/. This layered separation means the presentation code never imports machine-learning internals directly; everything passes through the service layer, which keeps the modules independently testable (Sommerville, 2016).

5.4 Database Implementation

Persistence uses SQLite through Python's sqlite3 module. SQLite was selected because it is a serverless, file-based database requiring no installation or running daemon, which suits a self-contained student wellbeing application and keeps all data on the user's own machine, a meaningful privacy property given the sensitivity of mental-health information. The schema is normalised into five tables (users, mood_logs, chat_messages, screening_results, recommendation_logs) with a one-to-many relationship from users to each of the others.

The schema is declared as a single SQL string and applied idempotently at start-up. A representative fragment is shown below:

```
CREATE TABLE IF NOT EXISTS chat_messages (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  role        TEXT     NOT NULL,          -- 'user' | 'assistant'
  content      TEXT     NOT NULL,
  sentiment    REAL,
  risk_level   TEXT,                    -- 'none' | 'elevated' | 'crisis'
  created_at   TEXT     NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);
```

Each child table carries a FOREIGN KEY ... ON DELETE CASCADE constraint, so removing a user automatically removes all of their logs, messages, screenings and recommendations, important for honouring a deletion request without orphaning data. Note that chat_messages stores both the computed sentiment and the assessed risk_level alongside the text, so the safety and analytics behaviour is auditable after the fact.

All connections are obtained through a single context manager that enforces referential integrity and guarantees clean-up:

```
@contextmanager
def get_connection(db_path: Optional[Path] = None):
    """Yield a SQLite connection with foreign keys enabled."""
    path = Path(db_path) if db_path else SETTINGS.db_path
    path.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(str(path))
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON;")
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

The `PRAGMA foreign_keys = ON` statement is required because SQLite does not enforce foreign keys by default. Setting `row_factory` to `sqlite3.Row` lets rows be addressed by column name and converted to dictionaries, so the rest of the application works with plain Python objects rather than tuples. Every public function, `create_user`, `add_chat_message`, `add_mood_log`, and the rest, wraps its query inside this context manager, meaning no raw SQL ever leaks into the higher layers. The database file path itself is configurable through the `MINDMATE_DB_PATH` environment variable, and the live database is excluded from version control via `.gitignore`.

5.5 Sentiment and Emotion Analysis Module

The sentiment module in `src/ml/sentiment.py` provides the affect signal used across the application, on mood-journal notes, on chat messages, and as one of the six features driving the machine-learning models. The primary scorer is **VADER** (Valence Aware Dictionary and sEntiment Reasoner), a lexicon- and rule-based model that is particularly well suited to the short, informal, emoji-laden text that students actually write (Hutto & Gilbert, 2014). VADER requires no training, is deterministic, and runs locally, all of which fit the project's reproducibility and privacy goals.

On top of the raw VADER output, a lightweight emotion layer maps text onto a small set of student-relevant states. A coarse keyword lexicon defines each emotion:

```
EMOTION_LEXICON: Dict[str, list[str]] = {
    "anxiety": [
        "anxious", "anxiety", "panic", "nervous", "worried", "worry", "scared",
        "afraid", "overwhelm", "overwhelmed", "tense", "restless", "dread",
    ],
    "sadness": [
        "sad", "down", "depressed", "hopeless", "lonely", "empty", "cry",
        "crying", "miserable", "worthless", "numb", "unmotivated",
    ],
    "stress": [
        "stress", "stressed", "pressure", "deadline", "exam", "exams",
        "assignment", "burnout", "burnt out", "workload", "overworked",
    ],
}
```

```
    ...
}
```

The dominant emotion is chosen by counting keyword hits per group and returning the highest-scoring group, defaulting to "neutral" when nothing matches. The core scoring routine then combines both signals:

```
def analyze(text: str) -> SentimentResult:
    """Run sentiment + emotion analysis on a single piece of text."""
    if not text or not text.strip():
        return SentimentResult(0.0, 0.0, 1.0, 0.0, "neutral", "neutral")

    scores = _ANALYZER.polarity_scores(text)
    compound = float(scores["compound"])
    return SentimentResult(
        compound=compound,
        positive=float(scores["pos"]),
        neutral=float(scores["neu"]),
        negative=float(scores["neg"]),
        label=_label_from_compound(compound),
        emotion=detect_emotion(text),
    )
```

The function returns a structured `SentimentResult` dataclass carrying the compound score (in the range -1 to +1), the positive/neutral/negative proportions, a three-way label, and the detected emotion. Empty input short-circuits to a neutral result, avoiding meaningless scores on blank notes. The label is derived from the compound score using VADER's conventional thresholds (≥ 0.05 positive, ≤ -0.05 negative, otherwise neutral). This dual output is deliberate: the continuous compound value feeds the numerical feature vector, while the categorical emotion is consumed directly by the recommender and the offline reply engine, which need a discrete label to select an appropriate strategy or opening line (Liu, 2012).

5.6 Student Segmentation Module

The segmentation module in `src/ml/personalization.py` groups students into behavioural **segments** using unsupervised **K-Means** clustering (MacQueen, 1967; Lloyd, 1982). Clustering operates on the six canonical features, average mood, average sleep hours, PHQ-9 score, GAD-7 score, average sentiment and engagement, whose order is fixed in a `FEATURE_COLUMNS` constant shared with the stress classifier to guarantee consistency between training and inference.

Training fits a two-step pipeline of standardisation followed by clustering:

```
def train_segmenter(matrix: np.ndarray, n_clusters: int = 4,
                    random_state: int = 42) -> StudentSegmenter:
    """Fit the standardisation + K-Means pipeline on a feature matrix."""
    pipeline = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("kmeans", KMeans(n_clusters=n_clusters, n_init=10,
                             random_state=random_state)),
        ]
    )
```

```

    ]
)
pipeline.fit(matrix)
mapping = label_clusters(pipeline, n_clusters)
return StudentSegmenter(pipeline=pipeline, cluster_to_label=mapping)

```

Wrapping the `StandardScaler` and `KMeans` in a scikit-learn Pipeline ensures the identical scaling is applied at inference time, which matters because K-Means is distance-based and therefore sensitive to feature magnitude, without standardisation, the PHQ-9 range of 0-27 would dominate the 1-5 mood range. The `random_state=42` and `n_init=10` arguments make the result reproducible while running ten initialisations and keeping the best, mitigating K-Means' sensitivity to starting centroids.

A subtlety of K-Means is that cluster identifiers are arbitrary: cluster 0 carries no meaning. To produce labels that are genuinely interpretable, *Thriving*, *Coping*, *At-Risk*, *High-Need*, the centroids are ranked on a composite wellbeing index:

```

def _wellbeing_index(centroid: np.ndarray) -> float:
    """Composite score: higher = better wellbeing."""
    avg_mood, avg_sleep, phq9, gad7, sentiment, _engagement = centroid
    return (
        (avg_mood / 5.0)
        + (min(avg_sleep, 9.0) / 9.0)
        + ((sentiment + 1.0) / 2.0)
        - (phq9 / 27.0)
        - (gad7 / 21.0)
    )

```

Before computing this index, the centroids are passed back through `scaler.inverse_transform` so they are expressed in their original units. The index rewards higher mood, sleep and sentiment and penalises higher PHQ-9 and GAD-7 scores, each term normalised to a comparable scale. Clusters are then sorted by this index and the ordered labels assigned in descending order of wellbeing, so the names remain meaningful across re-trainings even though the raw cluster ids change. On the synthetic dataset this segmenter achieves a silhouette score of 0.209 with segment sizes of At-Risk 198, Coping 176, Thriving 119 and High-Need 107, figures discussed in Chapter 6 and validated using the silhouette criterion of Rousseeuw (1987). At runtime the fitted pipeline and its cluster-to-label mapping are serialised with `joblib` and reloaded by `StudentSegmenter.load`.

5.7 Stress Classification Module

The stress module in `src/ml/stress.py` adds a supervised layer: a **multinomial Logistic Regression** that predicts a student's stress level (Low, Moderate or High) from the same six features. Logistic Regression was chosen over more opaque models because its outputs are well-calibrated probabilities and its behaviour is interpretable, a property that matters in a wellbeing setting, where a student deserves a transparent rather than a black-box judgement (Shatte et al., 2019).

```
def train_classifier(matrix: np.ndarray, labels: np.ndarray,
                    random_state: int = 42) -> StressClassifier:
    pipeline = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("clf", LogisticRegression(max_iter=1000, random_state=random_state)),
        ]
    )
    pipeline.fit(matrix, labels)
    return StressClassifier(pipeline=pipeline, classes=STRESS_LEVELS)
```

As with the segmenter, standardisation is bundled into the pipeline so that the gradient-based optimiser converges reliably and the same transformation is reapplied at prediction time; `max_iter=1000` gives the solver ample iterations to converge and the fixed `random_state` keeps results reproducible. The classifier exposes two prediction methods. `predict` returns the single most likely stress label, while `predict_proba` returns the full probability distribution:

```
def predict_proba(self, features: Dict[str, float]) -> Dict[str, float]:
    vector = np.array([[features.get(c, 0.0) for c in FEATURE_COLUMNS]])
    probs = self.pipeline.predict_proba(vector)[0]
    ordered = list(self.pipeline.classes_)
    return {str(cls): float(p) for cls, p in zip(ordered, probs)}
```

The probability output is what the Insights page surfaces to the user, turning an opaque label into a transparent "how confident is the model" view. Both methods read features through `features.get(c, 0.0)`, so a missing field degrades gracefully to zero rather than raising. On a stratified 25% hold-out of 150 samples the classifier attains an accuracy of 0.900 and a macro-F1 of 0.889, with full per-class metrics reported in Chapter 6 (Pedregosa et al., 2011). The trained pipeline is persisted with joblib and reloaded at runtime, exactly mirroring the segmenter so that both models share a single, predictable lifecycle.

5.8 Recommendation Engine

The recommender in `src/ml/recommender.py` decides which coping strategies a student sees. It is a **hybrid, content-based and rule-weighted** engine that combines three signals, the detected emotion, the student's segment, and the predicted stress level, and crucially produces an explanation for every score. A black-box recommender would be inappropriate in a wellbeing context, so transparency was treated as a first-class requirement (Ricci et al., 2015).

The scoring of a single strategy makes the weighting scheme explicit:

```
def _score_strategy(strategy: Dict, ctx: RecommendationContext) -> tuple[float, str]:
    """Return (score, human-readable rationale) for one strategy."""
    score = 0.0
    reasons: List[str] = []

    # 1. Emotion match, the strongest single signal.
    if ctx.emotion in strategy["emotions"]:
        score += 2.0
        reasons.append(f"matches your current feeling ({ctx.emotion})")

    # 2. Segment preference.
    preferred = _SEGMENT_PREFERENCES.get(ctx.segment, set())
    if strategy["category"] in preferred:
        score += 1.0
        reasons.append(f"suits the '{ctx.segment}' support plan")

    # 3. Stress-driven de-escalation boost.
    stress_w = _STRESS_WEIGHT.get(ctx.stress_level, 0.5)
    if strategy["category"] in _DEESCALATION_CATEGORIES:
        score += 1.5 * stress_w
        if stress_w > 0:
            reasons.append("helps settle high stress quickly")

    rationale = "; ".join(reasons) if reasons else "a generally helpful technique"
    return score, rationale
```

Three weighted rules combine additively. A match between the detected emotion and a strategy's target emotions contributes the strongest signal (weight 2.0). A match between the strategy's category and the student's segment preferences adds 1.0. Finally, when a student is stressed, calming and grounding categories receive a de-escalation boost of up to 1.5, scaled by a stress weight that is 0.0 for *Low*, 0.5 for *Moderate* and 1.0 for *High*, so a highly stressed student is steered firmly toward settling exercises, while a relaxed one is not. As each rule fires it appends a plain-language reason, and these are joined into a rationale string returned alongside the score.

The public `recommend` function scores every strategy in the catalogue, sorts them by descending score and returns the top *k*. Because the score is a simple sum of named contributions, the ranking is fully explainable: the Insights page can show not just *what* was recommended but *why*. The same engine is reused by the chat layer through the convenience wrapper `recommend_from_signals`, ensuring that the advice given in

conversation is consistent with the advice shown on the dashboard.

5.9 Conversational Assistant and OpenAI Integration

The conversational core lives in `src/assistant/chat_engine.py` and exposes a single public function, `generate_reply`. Its design embodies the graceful-degradation principle: it can produce a reply using the **OpenAI Chat Completions API** when a key is configured, or fall back to a fully **offline, rule-based engine** when one is not. Because the fallback is always available, the application runs end-to-end with no external dependency and no API cost, vital for a reproducible submission and a dependable live demo.

When a key is present, the engine assembles a message list and calls the API:

```
def _openai_reply(message: str, history: List[Dict], note: str) -> Optional[str]:
    """Call the OpenAI API; return None on any failure so we can fall back."""
    try:
        from openai import OpenAI

        client = OpenAI(api_key=SETTINGS.openai_api_key)
        messages = [{"role": "system", "content": prompts.SYSTEM_PROMPT},
                    {"role": "system", "content": note}]
        # Include a trimmed window of prior turns for continuity.
        for turn in history[-8:]:
            role = "assistant" if turn["role"] == "assistant" else "user"
            messages.append({"role": role, "content": turn["content"]})
        messages.append({"role": "user", "content": message})

        response = client.chat.completions.create(
            model=SETTINGS.openai_model,
            messages=messages,
            temperature=0.7,
            max_tokens=400,
        )
        return response.choices[0].message.content.strip()
    except Exception:
        # Any error (missing package, network, quota) -> use offline engine.
        return None
```

Two design choices deserve emphasis. First, the persona-defining `SYSTEM_PROMPT` and a personalisation note (built from the current emotion, segment and stress level) are injected as system messages, so the model adopts MindMate's empathetic, non-clinical voice and tailors its tone without ever naming the internal labels to the student. Only the last eight turns of history are sent, bounding token use and cost. Second, and most importantly, the entire call is wrapped in a broad try/except that returns `None` on *any* failure: a missing package, no network, an exhausted quota, or an invalid key. Returning `None` rather than raising is what lets the fallback take over without the student ever noticing an interruption.

When the API path is unavailable, the offline engine constructs a supportive reply from a small set of emotion-specific openers plus a single recommendation drawn from the same recommender used elsewhere:

```
def _offline_reply(message: str, emotion: str, segment: str, stress_level: str) -> str:
```

```

"""Construct a supportive, personalised reply with no external calls."""
opener = _EMOTION_OPENERS.get(emotion, _EMOTION_OPENERS["neutral"])
recs = recommend_from_signals(emotion=emotion, segment=segment,
                             stress_level=stress_level, top_k=1)
parts = [opener]
...

```

The public `generate_reply` function ties the layers together. It first assesses risk and analyses sentiment on every message, regardless of mode. If a crisis is detected it returns the safe crisis response immediately, never reaching the language model. Otherwise it tries the OpenAI path when `SETTINGS.llm_enabled` is true, falls back to the offline reply when that returns None, and finally appends a gentle resource nudge for elevated distress. The result is wrapped in an `AssistantReply` dataclass that records the text, risk level, sentiment, emotion and source ('openai', 'offline' or 'safety'), so the system always knows, and can store, which path produced each reply.

5.10 Safety and Crisis-Detection Layer

Safety is implemented in `src/assistant/safety.py` and is the most consequential module in the system. Following the precedent of earlier therapeutic conversational agents (Fitzpatrick et al., 2017; Inkster et al., 2018), risk screening runs *before* any reply is generated, on every single message. The layer is deliberately **rule-based and conservative**: in a wellbeing product a false positive (showing helplines unnecessarily) is far cheaper than a false negative (missing a genuine crisis), and a transparent set of patterns is easier to audit and reason about than a statistical classifier whose mistakes are hard to predict.

Risk is detected with two ordered lists of regular-expression patterns matched on word boundaries:

```

CRISIS_PATTERNS: List[str] = [
    r"\bkill myself\b",
    r"\bend my life\b",
    r"\btake my (own)?life\b",
    r"\bsuicid(e|al)\b",
    r"\bwant to die\b",
    r"\bwish i (was|were) dead\b",
    r"\bdon't want to (be alive|live)\b",
    r"\bno reason to live\b",
    r"\bharm(ing)? myself\b",
    r"\bself[- ]harm\b",
    r"\boverdose\b",
    r"\bcan't go on\b",
    r"\bbetter off without me\b",
]

```

The `\b` word-boundary anchors are important: they let the patterns catch genuine expressions of self-harm while avoiding false hits on innocuous phrasing such as "killing it in exams." A second, separate list of `ELEVATED_PATTERNS` (for example `\bhopeless\b`,

\bcan'?t cope\b, \bgiving up\b, \bnumb\b) captures distress that is serious but not acute. The two lists are compiled into case-insensitive regular expressions, and `assess_risk` evaluates them in strict priority order:

```
def assess_risk(text: str) -> RiskAssessment:
    """Classify the risk level of a user message."""
    if not text:
        return RiskAssessment("none", [])

    crisis_hits = _CRISIS_RE.findall(text)
    if crisis_hits:
        flat = [h if isinstance(h, str) else next(filter(None, h), "") for h in crisis_hits]
        return RiskAssessment("crisis", [m for m in flat if m])

    elevated_hits = _ELEVATED_RE.findall(text)
    if elevated_hits:
        flat = [h if isinstance(h, str) else next(filter(None, h), "") for h in elevated_hits]
        return RiskAssessment("elevated", [m for m in flat if m])

    return RiskAssessment("none", [])
```

Crisis patterns are checked first, so any acute signal takes precedence over a merely elevated one. When a crisis is matched the chat engine bypasses both the language model and the offline generator entirely and returns `crisis_response()`, which provides a brief validating message and prominently lists crisis and suicide-prevention helplines (iCall, Vandrevalla Foundation, Tele-MANAS, AASRA, and the IASP directory) drawn from the resources catalogue. The assistant explicitly does *not* attempt to counsel a student in crisis on its own. For elevated risk, a normal reply is generated and a gentle nudge toward professional support is appended. The assessed level is stored on every chat message so the behaviour is fully auditable. It must be stated plainly that this keyword-based approach has limits, it cannot understand sarcasm, indirect expression or unanticipated phrasing, and MindMate is therefore explicitly a self-help tool, not a clinical or emergency service.

5.11 User Interface Implementation

The user interface is built with **Streamlit** and organised as a multipage application. The entry point `app.py` configures the page, initialises the database schema on first launch, and assembles the navigation using Streamlit's modern `st.Page/st.navigation` API:

```
def main() -> None:
    _ensure_user()
    _sidebar()

    pages = [
        st.Page(home.render, title="Home", icon="🏠", default=True),
        st.Page(chat.render, title="Talk to MindMate", icon="💬"),
        st.Page(mood.render, title="Mood Tracker", icon="😊"),
        st.Page(screening_page.render, title="Self-Check", icon="📋"),
        st.Page(insights.render, title="Insights", icon="📊"),
        st.Page(resources.render, title="Resources", icon="📖"),
    ]
    st.navigation(pages).run()
```

Each page is a separate module under `src/ui/` exposing a render function, which keeps the presentation code modular and independently testable. Before any page renders, `_ensure_user` performs a lightweight onboarding flow: if no user is active in session state it shows a consent form (including an explicit disclaimer that MindMate is not a medical service) and, once submitted, creates a user record and stores the new `user_id` in `st.session_state`. The sidebar transparently reports whether the assistant is running in live OpenAI mode or the offline engine, so the user always knows which path is active.

Streamlit's **session state** is used throughout to bridge the framework's re-run-on-every-interaction execution model. The chat page is the clearest example. On first load it lazily pulls stored history from the database into `st.session_state.chat_messages`, then drives the conversation with `st.chat_message` for rendering and `st.chat_input` for the prompt box:

```
def render() -> None:
    user_id = st.session_state.user_id
    ...
    messages = _load_history(user_id)
    ...
    prompt = st.chat_input("Type how you're feeling...")
    if not prompt:
        return

    with st.chat_message("user"):
        st.markdown(prompt)
        messages.append({"role": "user", "content": prompt})

    profile = service.profile_for_user(user_id)
    with st.chat_message("assistant"):
        with st.spinner("MindMate is thinking..."):
            reply = chat_engine.generate_reply(
                prompt,
                history=messages[:-1],
                segment=profile["segment"],
                stress_level=profile["stress_level"],
            )
        st.markdown(reply.text)
```

When the user submits a message, the page displays it, retrieves the user's personalisation profile (segment and stress level) through the service layer, and calls `generate_reply` with the conversation history. A spinner gives feedback while the reply is produced. If the assessed risk level is `crisis`, an additional prominent error banner directs the student to the Resources page. Finally both turns are persisted to the database together with their sentiment and risk metadata, so the conversation survives a page refresh or a return visit. The other pages follow the same pattern, the Mood Tracker captures daily mood, energy, sleep and a free-text note (scored with VADER), the Self-Check page administers the PHQ-9 and GAD-7 instruments (Kroenke et al., 2001; Spitzer et al., 2006), and the Insights page renders the segment, the stress probabilities, the six features and the recommendation rationale to keep the machine learning transparent to the user.

5.12 Version Control

The project was developed under **Git** for version control, following standard practice in modern software engineering (Sommerville, 2016). Git provided an incremental history of the implementation, made it safe to experiment on feature changes, and supports the reproducibility goals of the project by recording exactly which version of the code produced any given result.

A carefully curated `.gitignore` enforces the project's security and reproducibility principles directly at the version-control boundary. Several categories are deliberately excluded:

- **Secrets and environment files**, `.env` is ignored so that no API key is ever committed. Only the `.env.example` template, which contains empty placeholders, is tracked.
- **The live database**, `data/mindmate.db` is ignored so that no real user data is checked in, while the schema and data-generation scripts remain under version control.
- **Trained model artefacts**, the `*.joblib` files in `models/artifacts/` are excluded to avoid committing large binaries; they are regenerated deterministically after cloning by running `scripts/train_models.py`.
- **Generated and environment clutter**, virtual-environment directories, `__pycache__`, the pytest cache and OS/IDE files.

This arrangement reflects a clear separation between *source* (tracked) and *generated or sensitive artefacts* (ignored). Because configuration is read entirely from environment variables through `src/config.py`, no secret ever appears in tracked source, and because the models are reproducible from seeded scripts, omitting the binaries costs nothing. Together with the pinned dependency versions and the fixed random seed of 42, this version-control discipline ensures that a reviewer can clone the repository, recreate the virtual environment, regenerate the data and models, and obtain precisely the results reported in the next chapter.

Chapter 6: Testing

6.1 Introduction

Testing is the discipline through which a software product is examined for the presence of defects and through which confidence is built that the system behaves as its specification requires. For a wellbeing assistant aimed at students, this activity carries a weight that goes beyond ordinary correctness: an error in the conversational interface is an inconvenience, but an error in the crisis-safety layer could allow a genuine cry for help to pass unnoticed. The testing effort for MindMate was therefore designed not only to confirm that features work, but to demonstrate that the parts of the system on which a vulnerable user might depend are robust, predictable, and conservative in the face of ambiguity.

This chapter documents the verification and validation activities carried out on MindMate. It explains the overall testing strategy, the structure of the automated test suite, and the way responsibilities were divided across unit, integration, and system levels. It then presents a consolidated table of representative test cases, describes the special treatment given to safety-critical behaviour, and reports the methodology used to validate the machine-learning models. The chapter closes with an honest account of the results obtained and of the defects and limitations that remain. Throughout, the distinction between verification, building the product correctly, and validation, building the correct product, is observed, following the conventional framing of software engineering practice (Sommerville, 2016; Pressman & Maxim, 2014).

In total, the project ships with a suite of **34 automated tests**, all of which pass. These are written using the **pytest** framework and are executed as part of the ordinary development workflow, so that any regression introduced by a later change is caught immediately rather than discovered in use.

6.2 Testing Strategy

The testing strategy for MindMate followed a layered, risk-driven model. Rather than allocating effort uniformly across the codebase, I directed the most rigorous testing toward the components whose failure carries the greatest consequence. The crisis-detection layer, the screening scorers, and the persistence layer therefore received the densest coverage, while purely presentational Streamlit code, which is difficult to test automatically and low in risk, was verified manually.

Several principles guided the strategy:

- **Automation first.** Every behaviour that can be expressed as a deterministic input-output relationship is captured as an automated pytest case, so that the full suite can be re-run cheaply after any change. This supports the regression-testing discipline recommended for evolving systems (Sommerville, 2016).
- **Isolation of side effects.** Database tests run against a temporary, throwaway SQLite file supplied by a pytest fixture, so they never touch production data and leave no residue. The chat-engine tests run with no API key configured, which forces the deterministic offline path and removes any dependency on a remote service.
- **Conservatism in safety tests.** Where the safety layer is concerned, the tests deliberately probe both the cases that must be flagged and the cases that must *not* be flagged, so that sensitivity is verified without ignoring the cost of over-flagging.
- **Reproducibility of model validation.** Machine-learning evaluation is anchored to fixed random seeds and a stratified hold-out split, so that the reported metrics can be regenerated exactly.

The test suite is organised by component into six files, each targeting one area of the system: `test_safety.py` (crisis detection), `test_sentiment.py` (sentiment and emotion analysis), `test_screening.py` (PHQ-9 and GAD-7 scoring), `test_database.py` (the SQLite persistence layer), `test_ml.py` (segmentation, the stress classifier, and the recommender), and `test_chat_engine.py` (the assistant's offline reply path and safety routing). This mapping of tests onto architectural layers mirrors the structure described in earlier chapters and makes it straightforward to locate the cause of any failure.

6.3 Unit Testing

Unit testing exercises the smallest independently meaningful units of behaviour in isolation. In MindMate these units are the pure functions and small classes that make up the assistant and ML packages, chosen because they are deterministic and free of external state.

The **safety** unit tests confirm that `safety.assess_risk` returns the correct risk level for a representative range of inputs. Explicit crisis phrases such as "I want to kill myself", "Sometimes I think about ending my life", and "there is no reason to live anymore" are required to yield the `crisis` level. Phrases expressing distress without imminent danger, "I feel so hopeless", "I just can't cope with all this", "I feel worthless lately", are required to yield the `elevated` level. These cases are parameterised so that each phrase is reported as an individual result.

The **sentiment** unit tests verify the VADER-based analyser and the custom emotion lexicon (Hutto & Gilbert, 2014). Clearly positive text is required to produce a positive compound score and a positive label; clearly negative text the reverse. Separate cases

confirm that the emotion classifier recognises anxiety ("I am so anxious and worried about exams") and stress ("the workload and deadlines are too much"), and that empty input is treated as neutral in both label and emotion.

The **screening** unit tests check the scoring logic for the two standardised instruments. PHQ-9 with all-zero responses must total zero and be classified as "Minimal"; with all responses at the maximum it must total 27 and be classified as "Severe" (Kroenke et al., 2001). GAD-7 is checked at both extremes of its severity range (Spitzer et al., 2006). A guard test confirms that passing a response vector of the wrong length raises a `ValueError`, protecting the scorers from malformed input.

The **machine-learning** unit tests confirm that each model behaves sensibly on small, controlled fixtures. The segmenter, trained on a synthetic two-group matrix, must place a clearly thriving profile in a healthy segment and a clearly struggling profile in an at-risk segment. The stress classifier, trained on a generated signal, must label an unambiguously high-stress feature vector as "High". The recommender must return strategies in non-increasing order of score and, for an anxious high-stress context, must surface calming or grounding work such as 4-7-8 Breathing or 5-4-3-2-1 Grounding.

6.4 Integration Testing

Integration testing checks that units cooperate correctly once assembled, including the contracts between layers. Two areas of MindMate are inherently integrative and are tested at this level: the persistence layer and the chat engine.

The **database** tests treat the SQLite layer as an integrated whole, writing through the public API and reading back through it to confirm that data survives a complete round-trip. A user is created and re-fetched, with name and course verified. A mood log is written with mood, energy, sleep hours, a note, and a sentiment value, then read back to confirm the stored fields. Chat messages are written in sequence and retrieved to confirm that ordering is preserved. A user's support segment is updated and re-read, and a screening result is persisted and recovered. Because these tests exercise the schema, the SQL, and the row-to-dictionary mapping together, they catch faults that no single unit test would expose. Each test runs against an isolated temporary database supplied by a fixture, so the tests are independent of one another and of any real data.

The **chat-engine** tests are integrative because a single call to `chat_engine.generate_reply` orchestrates the safety layer, the personalisation context, and the reply generator. With no API key present, an ordinary message such as "I'm really stressed about my exams" must be answered through the offline engine, with the reply marked `offline`, a none risk level, and a substantive body of text. This confirms that the

offline fallback engine and the safety screening are wired together correctly and that the system degrades gracefully when no external language model is available.

6.5 System Testing

System testing evaluates the complete application against its intended use, treating it as the user would encounter it. For MindMate this combined automated end-to-end checks of the most critical user journey with manual walkthroughs of the Streamlit interface.

The most important system-level behaviour is **crisis routing**: when a user types a message expressing suicidal intent anywhere in the chat, the system must abandon ordinary reply generation and respond from the safety layer. The chat-engine test "crisis message routes to safety" verifies exactly this path: the input "I want to kill myself" produces a reply whose source is safety, whose risk level is *crisis*, and whose text contains a helpline reference. The complementary "elevated reply appends resource nudge" test confirms that a distressed but non-crisis message ("I feel hopeless about everything") is classified as *elevated* and that the reply gently points the user toward resources. Together these establish that the safety policy holds at the level of the assembled system, not merely in the isolated detector.

Beyond the automated checks, each Streamlit page was exercised manually across a typical session: registering a profile, logging several days of mood data and observing the trend charts, completing PHQ-9 and GAD-7 self-checks and confirming that the reported severity bands matched the scored totals, reviewing the Insights page to confirm that the six features, the segment, and the stress probabilities were displayed consistently, and confirming that the Resources page presented the correct Indian and global helplines. This manual layer covers presentation and navigation behaviour that is impractical to automate but essential to validate before use.

6.6 Test Cases

The following table presents a representative selection of the executed test cases drawn from across the suite. Each derives from an automated pytest case described above, and every case passed when the suite was last executed. Inputs are shown in an abbreviated form for readability.

Table 6.1: Test Cases

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Crisis detection on explicit intent	"I want to kill myself"	risk level = crisis	crisis	Pass
TC-02	Non-crisis idiom not over-flagged	"I'm killing it in my exams!"	risk level $\neq$ crisis	none	Pass
TC-03	Elevated distress detected	"I feel so hopeless"	risk level = elevated	elevated	Pass
TC-04	Positive sentiment classification	"I feel great and really hopeful today"	compound > 0, label = positive	positive	Pass
TC-05	Negative sentiment classification	"I am miserable and everything feels hopeless"	compound < 0, label = negative	negative	Pass
TC-06	Emotion detection (anxiety)	"I am so anxious and worried about exams"	emotion = anxiety	anxiety	Pass
TC-07	PHQ-9 minimal severity	nine responses all 0	total = 0, severity = Minimal, no self-harm flag	0 / Minimal / False	Pass
TC-08	PHQ-9 severe + self-harm flag	nine responses all 3	total = 27, severity = Severe, self-harm flag = True	27 / Severe / True	Pass
TC-09	GAD-7 severity thresholds	all 0, then all 3	Minimal, then Severe	Minimal / Severe	Pass
TC-10	Database user round-trip	create user "Test Student", MSc DS	stored name and course returned	matched	Pass
TC-11	Mood-log round-trip	mood 4, sleep 7.5, note, sentiment 0.2	one log with mood 4, sleep 7.5	matched	Pass
TC-12	Segmenter prediction	clearly thriving profile	healthy segment (Thriving/Coping)	Coping	Pass
TC-13	Stress-classifier prediction	high-stress feature vector	stress level = High	High	Pass
TC-14	Recommender ordering	emotion anxiety, segment At-Risk, stress High	scores non-increasing; calming/grounding present	ordered; 4-7-8 Breathing	Pass

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-15	Offline chat reply	"I'm really stressed about my exams"	source offline, risk none, substantive text	offline / none	Pass
TC-16	Crisis routing through engine	"I want to kill myself"	source safety, risk crisis, helpline in text	safety / crisis	Pass
TC-17	Self-harm flag isolated to item 9	items 1-5 = 3, items 6-9 = 0	self-harm flag = False	False	Pass
TC-18	Malformed screening input rejected	five PHQ-9 responses	ValueError raised	ValueError	Pass

6.7 Safety-Critical Testing

The safety layer is the part of MindMate that most clearly distinguishes it from an ordinary chatbot, and its testing was approached with a correspondingly different mindset. The governing observation is the **asymmetry of error cost**. A false positive, flagging a message as a crisis when the user is in no danger, produces, at worst, an unnecessary helpline message and a moment of friction. A false negative, failing to recognise genuine suicidal intent, could mean that a student who reached out received an ordinary, counselling-style reply instead of being directed to immediate help. Because the human cost of these two errors is so unequal, the detector is deliberately tuned to be conservative, preferring to over-flag rather than to miss, and the tests are written to enforce that bias.

Safety-critical testing in MindMate has three components:

- **Sensitivity to true crises.** A parameterised set of phrases expressing suicidal ideation or self-harm, direct statements of wanting to die, thoughts of ending one's life, intentions to hurt oneself, and statements that there is no reason to live, must each be classified as `crisis`. These verify that the detector responds to varied phrasings of the same underlying intent rather than to a single fixed wording.
- **Resistance to over-flagging.** The detector must not raise a crisis on everyday figurative language that happens to contain alarming words. "I'm killing it in my exams!" and "This deadline is killing me but I'll manage" both contain the word "killing" yet describe success and ordinary stress; both, along with neutral and empty input, are required to return a level other than `crisis`. This is the practical counterweight to conservatism: a detector that fired on every occurrence of a keyword would erode trust and quickly be ignored.

- **Correctness of the crisis response.** When a crisis is detected, the response itself is tested. The `crisis_response` output must contain a recognised helpline reference such as Tele-MANAS (14416), the Vandrevalla Foundation, or AASRA, ensuring that the user is given an actionable route to support rather than an empty acknowledgement.

In addition to the conversational detector, the screening pathway contains its own safeguard. PHQ-9 item 9 asks specifically about thoughts of self-harm, and the scorer raises a dedicated `flags_self_harm` indicator whenever that item is answered with any non-zero response (Kroenke et al., 2001). The tests confirm both directions of this rule: the flag is raised when item 9 is non-zero and, critically, is *not* raised when other items are elevated but item 9 is zero. This guarantees that a high overall depression score never silently masks, nor falsely manufactures, a self-harm signal. Treating this behaviour as safety-critical, and testing it from both sides, reflects the principle that the components carrying the greatest risk deserve the most explicit verification (Pressman & Maxim, 2014).

6.8 Model Validation

The machine-learning components were validated separately from the functional tests, because their quality is measured statistically rather than as pass-or-fail behaviour. Validation followed a conventional hold-out methodology and was conducted on the synthetic datasets described in earlier chapters; all results below should therefore be read as a demonstration of the evaluation method on simulated data rather than as evidence about real students.

The **stress classifier**, a multinomial logistic-regression model, was evaluated on a stratified 25% hold-out test set of 150 samples drawn from the 600-student synthetic dataset. On this unseen set it achieved an **accuracy of 0.900** and a **macro-averaged F1 score of 0.889**. The macro average is reported alongside accuracy because it weights each stress class equally and so is not flattered by the larger High class. Per-class performance was strongest for the High class (precision 0.932, recall 0.958) and weakest for the Moderate class (precision 0.810, recall 0.829), which is expected, since the moderate band lies between the two extremes and is the most easily confused. The use of a held-out split, rather than evaluation on the training data, follows standard machine-learning practice for obtaining an unbiased estimate of generalisation performance (Géron, 2019; Pedregosa et al., 2011).

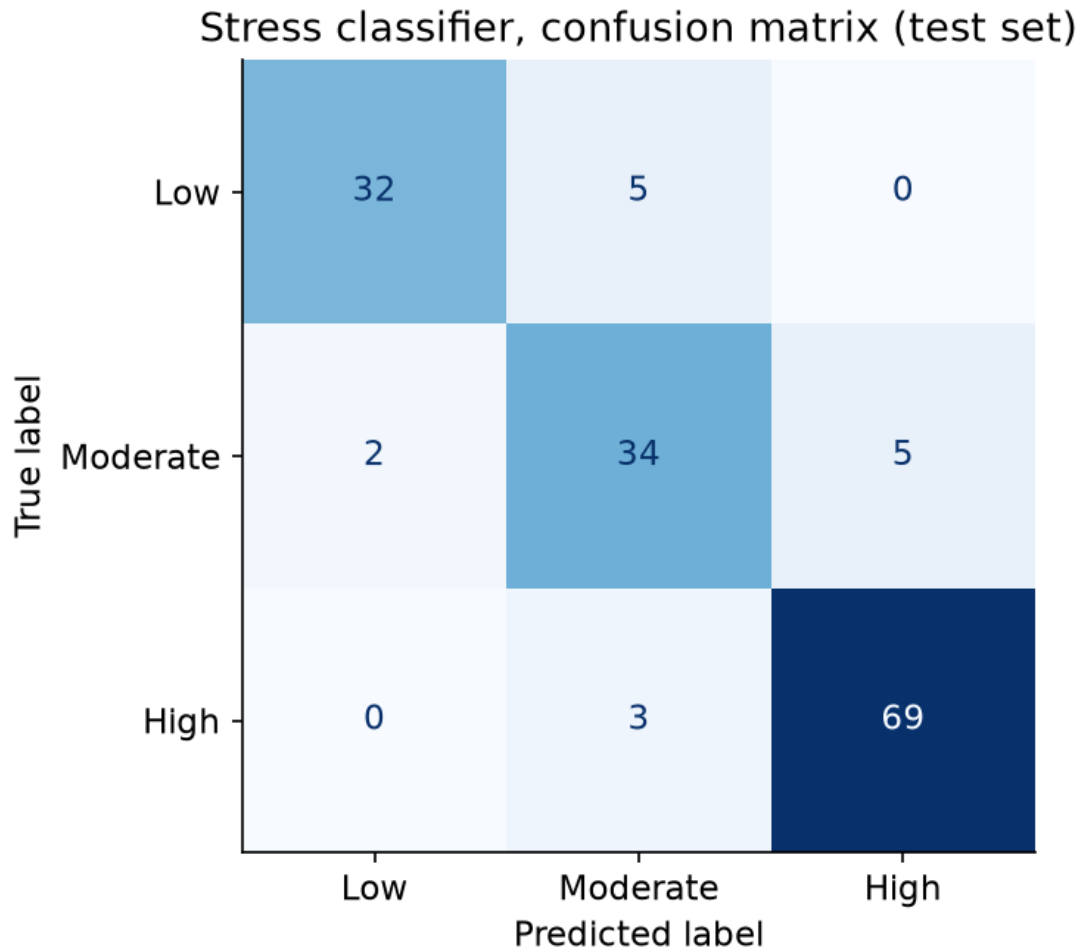


Figure 6.1: Confusion matrix of the stress classifier on the test set

The confusion matrix in Figure 6.1 confirms this pattern: the off-diagonal errors are concentrated at the boundaries between adjacent stress levels rather than between Low and High, which is the most benign failure mode for a screening-style classifier.

The **K-Means student segmenter** was assessed using the silhouette coefficient, an internal measure of how compact and well separated the discovered clusters are (Rousseeuw, 1987). With four clusters the model produced a **silhouette score of 0.209**. This is a modest value, and it is reported candidly: it indicates that the four support segments overlap considerably rather than forming sharply distinct groups, which is realistic for continuous wellbeing data where students lie along a spectrum rather than in discrete types. The segments remain useful for tailoring tone and recommendations, but the silhouette score is a reminder not to treat the segment boundaries as crisp. The cluster labels themselves are assigned by ranking the cluster centroids on a composite wellbeing index, so that each label carries a meaningful, interpretable ordering rather than an arbitrary cluster number.

6.9 Results and Defects

The complete suite of 34 pytest tests executes successfully, with no failures and no errors, on Python 3.12. Coverage spans every layer identified as carrying meaningful risk: the safety detector and crisis response, sentiment and emotion analysis, PHQ-9 and GAD-7 scoring including the self-harm safeguard, the full database round-trip for users, mood logs, chat history, segments, and screening results, the three machine-learning models, and the assistant's offline and safety routing paths. The model-validation results, 0.900 accuracy and 0.889 macro-F1 for the stress classifier and a 0.209 silhouette for the segmenter, were reproduced from fixed seeds and are consistent with the figures reported elsewhere in this report.

Several defects and limitations were identified during testing and are recorded honestly here, since acknowledging them is part of responsible evaluation:

- **Rule-based crisis detection can miss indirect phrasing.** The detector recognises explicit and lexically clear expressions of risk, but it is fundamentally pattern-based. A student who signals distress obliquely, through metaphor, sarcasm, coded language, or a slow accumulation of despair across several messages, may not trigger a crisis classification. This is the most serious limitation of the safety layer, and it is the reason the system is positioned strictly as a source of support and signposting rather than as a clinical or emergency tool.
- **Conservatism produces occasional false positives.** The deliberate bias toward flagging means that some messages will be treated as elevated or crisis when the user is not in difficulty. This is an accepted trade-off given the cost asymmetry, but it is a real behaviour that an alert user may notice.
- **Sentiment and emotion analysis are lexicon-bound.** VADER and the custom emotion keywords perform well on direct, English-language statements of feeling but are weaker on irony, mixed emotion, and code-switched or colloquial text, so the detected emotion that drives recommendations is occasionally imperfect.
- **Validation rests on synthetic data.** The reported model metrics demonstrate that the evaluation pipeline is sound, but they are obtained from simulated students. They do not, and cannot, establish clinical effectiveness with real users; that would require a properly governed study with human participants.
- **Presentation logic is verified manually.** The Streamlit interface is checked by walkthrough rather than by automated UI tests, so visual or interaction regressions in the front end could escape the automated suite.

Taken together, the results give reasonable confidence that MindMate behaves as designed within the boundaries it claims for itself, while the recorded limitations make clear where that confidence stops. The most consequential of these, the inability of a rule-based detector to catch every indirect expression of risk, directly informs the future-work proposals discussed in the concluding chapter, where a more capable, learning-based safety layer is considered as a priority for any move beyond a synthetic, demonstrative setting (Sommerville, 2016).

Chapter 7: Results and Discussion

7.1 Introduction

This chapter presents and interprets the empirical results produced by the MindMate system. It brings together the outputs of the three machine-learning components that personalise the assistant, the K-Means student segmenter, the multinomial Logistic-Regression stress classifier, and the VADER-based sentiment analyser, together with the results of a simulated pilot evaluation designed to assess the system's effect on mental-health and academic outcomes. Each result is first reported using the exact figures generated by the reproducible analysis pipeline and is then discussed in terms of what it reveals about the underlying data and the behaviour of the models.

Before presenting any numbers, one caveat must be stated plainly because it frames the interpretation of everything that follows. **All of the data analysed in this chapter are synthetic.** The 600-student feature dataset and the 240-participant pilot study were both generated programmatically from latent-variable models that encode realistic but artificial relationships between wellbeing variables. Consequently, the results reported here demonstrate that the evaluation *methodology* is sound, reproducible, and capable of detecting the effects it is designed to detect; they do **not** constitute evidence of real-world clinical efficacy. This distinction is restated at several points throughout the chapter, and most forcefully in the Limitations section, so that no reader mistakes a methodological demonstration for a clinical claim. The reproducibility of the synthetic generation (random seed 42) means that every statistic below can be regenerated exactly, which is itself a virtue: it makes the analysis transparent and auditable, even though it does not make it clinical.

7.2 Dataset Overview

The primary modelling dataset, `data/synthetic_students.csv`, contains 600 simulated students, each described by the six canonical wellbeing features used throughout MindMate: average mood (`avg_mood`, 1-5), average sleep duration (`avg_sleep_hours`), the PHQ-9 depression score (`phq9_score`, 0-27), the GAD-7 anxiety score (`gad7_score`, 0-21), average journal sentiment (`avg_sentiment`, -1 to +1) and an engagement count (`engagement`). Each record additionally carries a categorical `stress_label` of Low, Moderate or High that serves as the target for the supervised classifier. The records were produced by a latent-wellbeing generator that first samples an unobserved wellbeing state for each student and then derives the six observable features from it, deliberately injecting the kinds of correlation that clinical literature would lead us to expect, for example, that poorer sleep co-occurs with higher anxiety, and that lower mood co-occurs with elevated PHQ-9 and GAD-7 scores.

The presence of these designed-in correlations is confirmed by the correlation matrix of the six features.

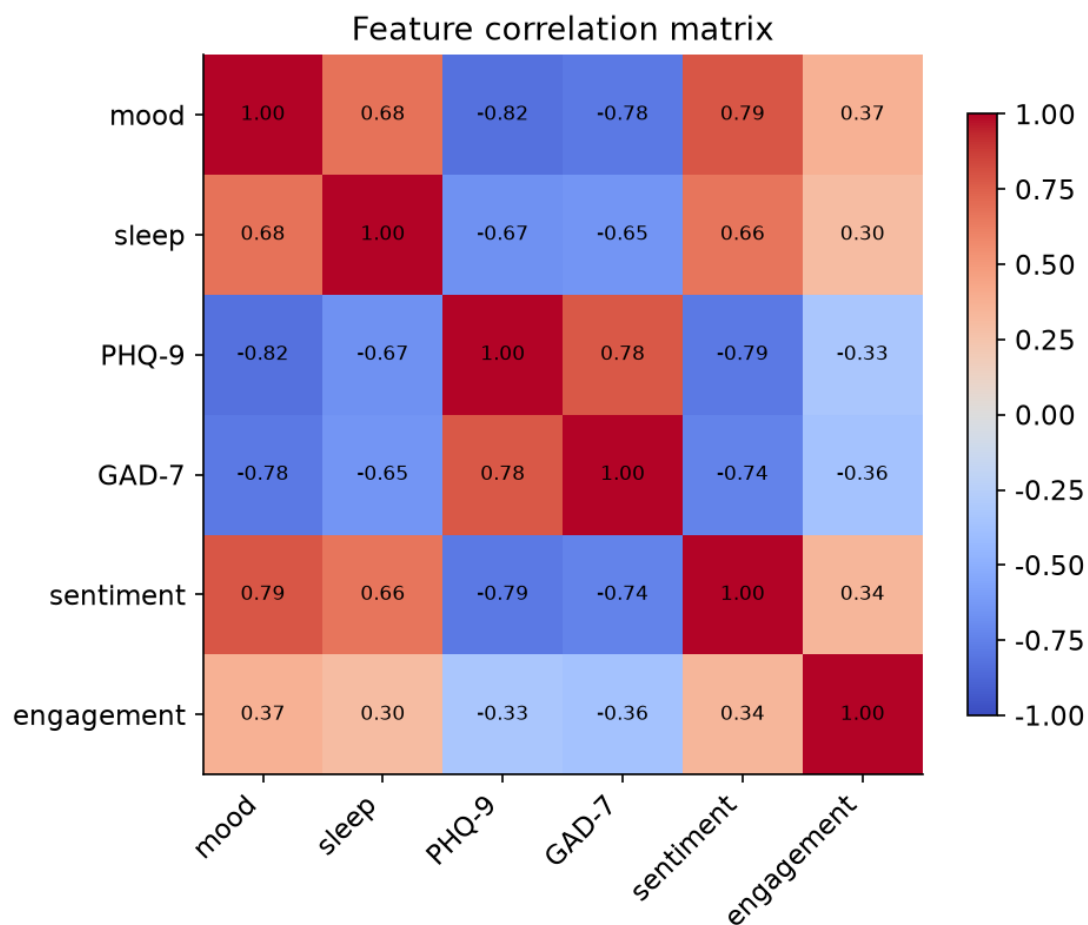


Figure 7.1: Correlation matrix of the six wellbeing features

As shown in Figure 7.1, the standardised features exhibit a coherent block structure. The two clinical screening instruments, `phq9_score` and `gad7_score`, are strongly positively correlated with one another, which is consistent with the well-documented comorbidity of depressive and anxiety symptoms (Kroenke et al., 2001; Spitzer et al., 2006). Both of these scores are negatively correlated with `avg_mood`, `avg_sleep_hours` and `avg_sentiment`: students whose generated state places them under greater symptom load also tend to report lower mood, shorter sleep and more negative journal sentiment. Engagement shows weaker associations with the remaining variables, behaving as a relatively independent dimension that captures how much a student interacts with the application rather than how distressed they are. This correlation structure is important for two reasons. First, it validates that the synthetic generator behaves as intended rather than producing noise. Second, it explains in advance why both unsupervised and supervised models are able to extract a usable signal: the features are neither perfectly collinear (which would make some redundant) nor independent (which would leave nothing to learn), but instead occupy the productive middle ground where structure exists to be discovered.

The second dataset, `data/evaluation_study.csv`, supports the pilot analysis in Section 7.6. It simulates a four-week study of 240 participants, split evenly into 120 treatment users who notionally used MindMate and 120 control users who did not, with pre- and post-intervention measurements of PHQ-9, GAD-7, a wellbeing index (0-100) and weekly focus hours.

7.3 Student Segmentation Results

The first personalisation component is an unsupervised K-Means segmenter (MacQueen, 1967; Lloyd, 1982) that groups students into four support segments on the basis of their standardised six-feature profiles. The number of clusters, $k = 4$, was chosen to map onto four interpretable levels of support need rather than being derived purely from an internal optimisation criterion. To ensure that the resulting labels are meaningful rather than arbitrary, each cluster centroid was scored on a composite wellbeing index and the four clusters were then ranked and named accordingly: **Thriving**, **Coping**, **At-Risk** and **High-Need**.

The four segments are unequal in size, as Figure 7.2 illustrates.

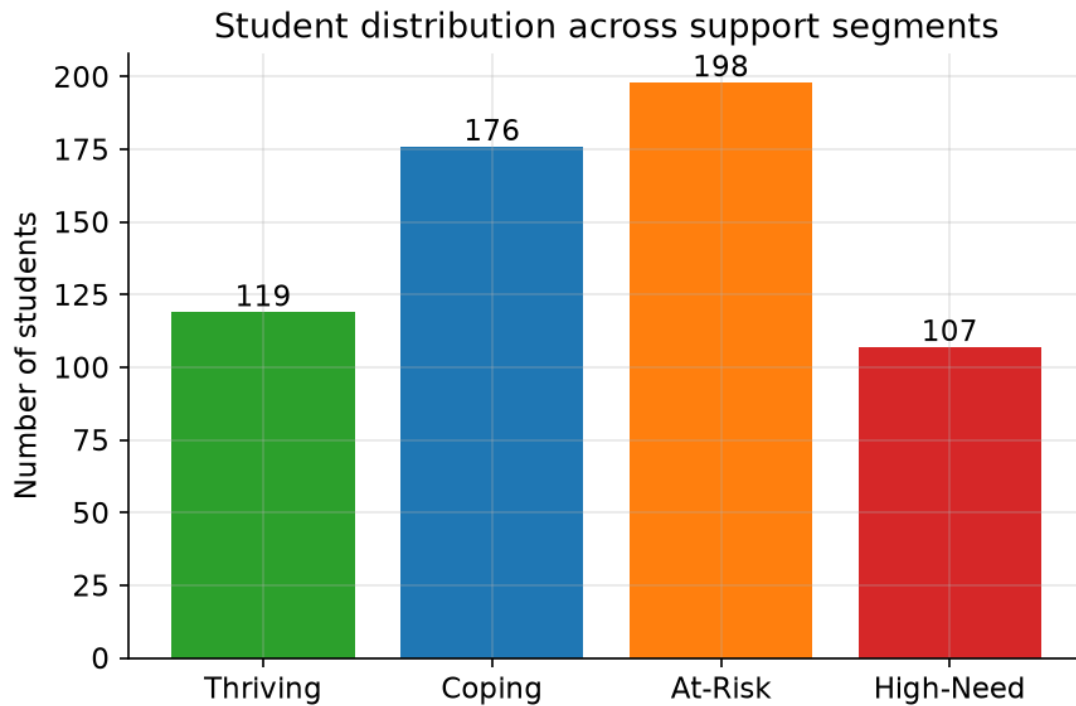


Figure 7.2: Distribution of students across the four support segments

The segment sizes are At-Risk with 198 students, Coping with 176, Thriving with 119 and High-Need with 107. The two middle-and-lower segments (At-Risk and Coping) together account for 374 of the 600 students, leaving a smaller Thriving group and a still smaller but clinically most important High-Need group of 107. This distribution is plausible for a student population: a large mass of individuals carry moderate concerns, a minority are flourishing, and a comparable minority require the most active support. From a product perspective, the relatively small High-Need cluster is precisely the group toward which the assistant should escalate its recommendations and resource signposting most assertively.

The internal quality of the clustering was assessed using the silhouette coefficient (Rousseeuw, 1987), which yielded a value of **0.209**. The silhouette coefficient ranges from -1 to +1, where values near 1 indicate tight, well-separated clusters, values near 0 indicate overlapping clusters whose boundaries are fuzzy, and negative values indicate misassignment. A score of 0.209 is modest and honestly reflects the nature of the data: wellbeing is a continuum, not a set of discrete natural kinds, so any partition of students into segments will inevitably cut through regions of gradual transition rather than through empty gaps. The PCA projection in Figure 7.3 makes this visible.

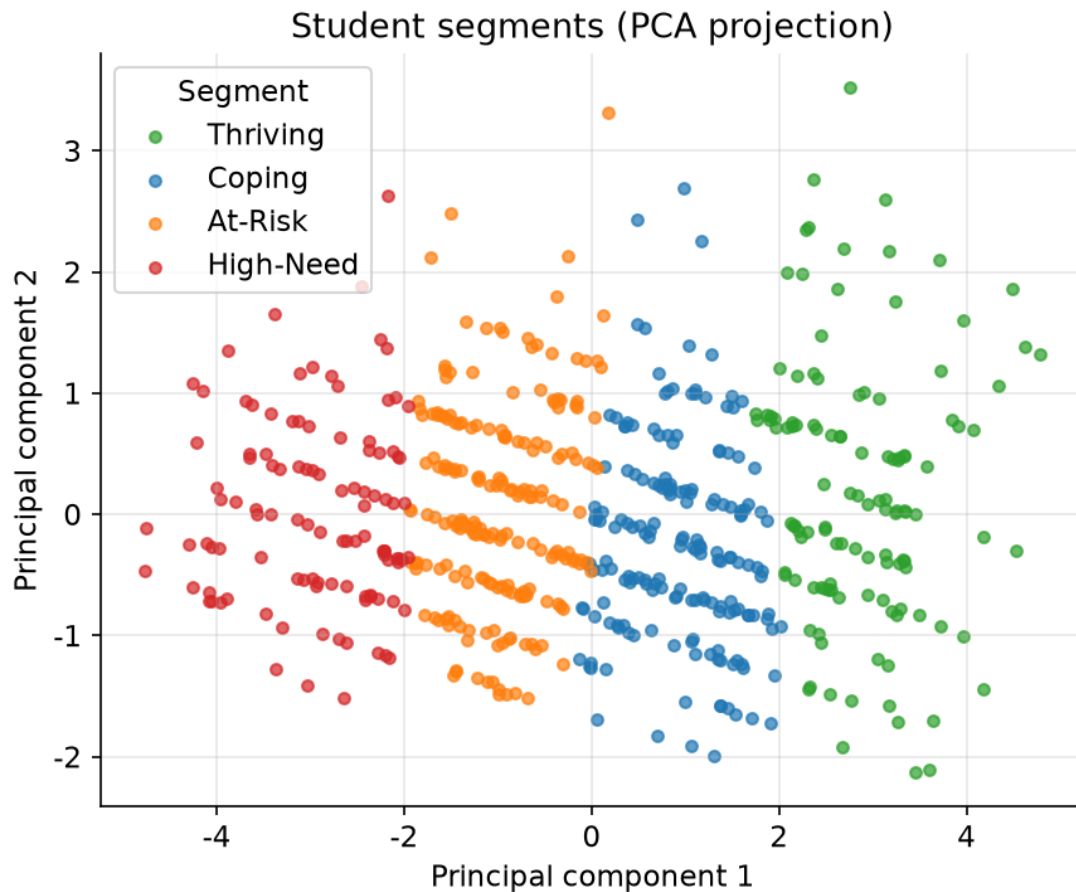


Figure 7.3: PCA projection of the student segments

When the six-dimensional feature space is reduced to its first two principal components, the four segments appear ordered along a dominant axis that runs from High-Need at one extreme to Thriving at the other, with Coping and At-Risk occupying the intermediate territory. The clusters are clearly arranged but their edges touch and partly overlap, which is exactly what a silhouette score around 0.2 predicts. Rather than treating this as a defect, it is more accurate to read it as a faithful representation of the construct: a student near the boundary between At-Risk and Coping genuinely is a borderline case, and forcing such a student into one bucket is a modelling convenience, not a claim that two truly distinct populations exist. For MindMate's purposes the segmentation does not need crisp separation to be useful; it needs only to provide a stable, interpretable coarse stratification that biases the tone and recommendation mix of the assistant in a sensible direction, which it achieves.

7.4 Stress Classification Results

Whereas segmentation is unsupervised and operates at the level of broad support tiers, the stress classifier is a supervised model that predicts a discrete stress level, Low, Moderate or High, for an individual student from the same six features. The model is a multinomial Logistic Regression implemented with scikit-learn (Pedregosa et al., 2011) inside a pipeline that first standardises the features. The model was evaluated on a stratified 25% hold-out test set comprising 150 samples, so that the reported metrics reflect performance on data the model never saw during training.

On this hold-out set the classifier achieved an **accuracy of 0.900** and a **macro-averaged F1 score of 0.889**. The closeness of these two figures is itself informative: macro-F1 averages the per-class F1 scores with equal weight regardless of class frequency, so a macro-F1 nearly as high as the accuracy indicates that the model performs well across all three classes rather than achieving a high overall score by neglecting a minority class. The per-class breakdown confirms this.

Table 7.1: Stress Classification Results

Class	Precision	Recall	F1-score	Support
High	0.932	0.958	0.945	72
Low	0.941	0.865	0.901	37
Moderate	0.810	0.829	0.819	41
Macro avg	0.894	0.884	0.889	150

The strongest performance is on the High class, with an F1 of 0.945 and a recall of 0.958. This is the most operationally desirable error profile, because the High-stress category is the one MindMate most needs to detect reliably; a recall of 0.958 means that very few genuinely high-stress students are missed. The Low class also performs well (F1 0.901), with high precision (0.941) but somewhat lower recall (0.865), indicating that a few low-stress students are pulled into the adjacent Moderate band, a conservative direction of error that errs toward offering more support rather than less. The weakest class is Moderate (F1 0.819), which is unsurprising and easily explained: Moderate is the middle category that shares borders with both Low and High, so the feature profiles that fall into it are the least distinctive and the most likely to be confused with their neighbours. The confusion matrix makes this pattern explicit.

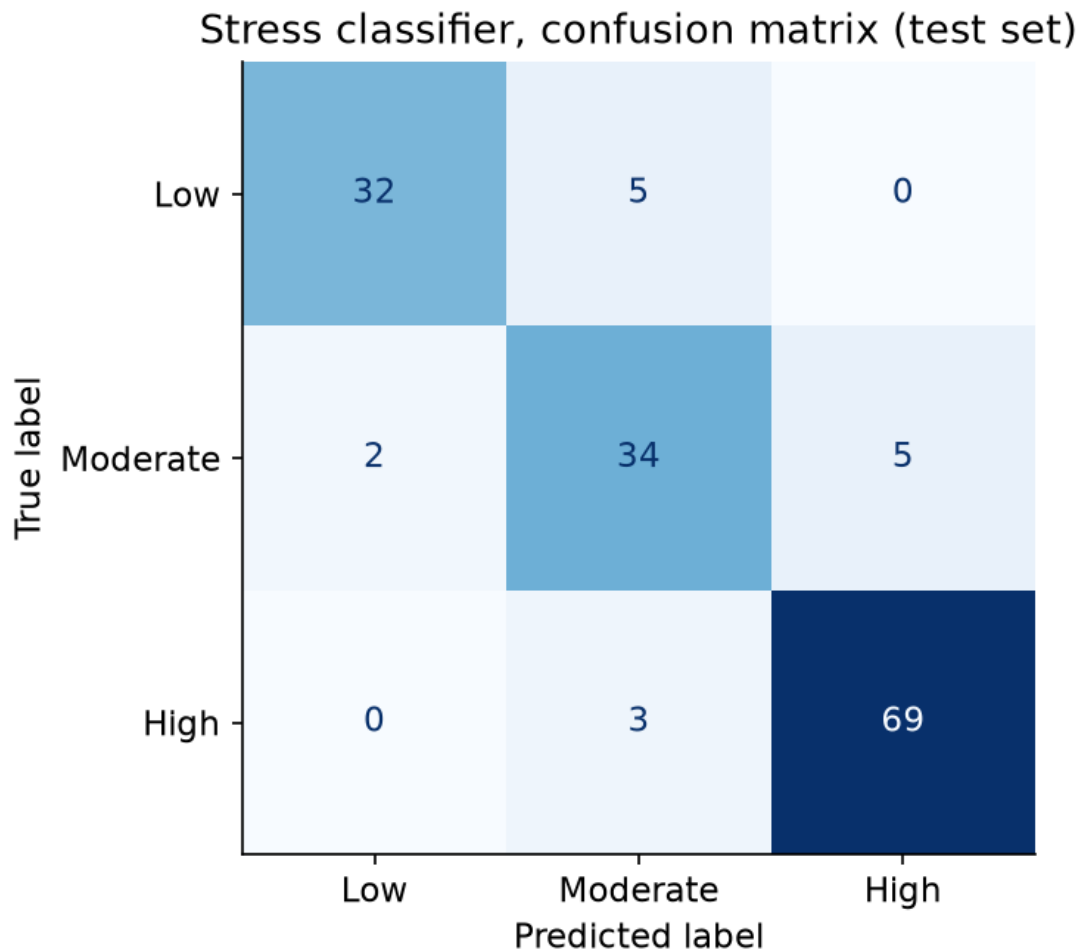


Figure 7.4: Confusion matrix of the stress classifier

As Figure 7.4 shows, the errors are overwhelmingly confined to adjacent classes, Low confused with Moderate, and Moderate confused with High, rather than the catastrophic Low-versus-High confusions that would be most damaging. There are essentially no instances of a high-stress student being labelled low-stress or vice versa. This ordinal coherence is the natural consequence of using a linear model on features that are themselves monotonically related to the underlying stress construct: a Logistic Regression learns a set of separating hyperplanes, and when the true classes are ordered along a continuum, the misclassifications it makes are concentrated at the boundaries between neighbouring bands. For a wellbeing assistant, this is the right failure mode, because a one-step error (treating a Moderate student as High, or vice versa) carries little practical cost, whereas a two-step error could either alarm a stable student or overlook a struggling one.

7.5 Sentiment Analysis Results

The third analytical component is sentiment analysis, applied to the free-text notes students write in the Mood Tracker. MindMate uses VADER (Hutto & Gilbert, 2014), a lexicon- and rule-based model well suited to short, informal, first-person text, augmented with a custom emotion-keyword lexicon. VADER returns a compound polarity score in the range -1 to +1, which is stored alongside each note and feeds the `avg_sentiment` feature consumed by the segmenter and classifier.

Because sentiment is computed per note and accumulates over time, its most informative output is longitudinal rather than cross-sectional. Figure 7.5 traces the sentiment of one simulated student's journal notes across successive entries.

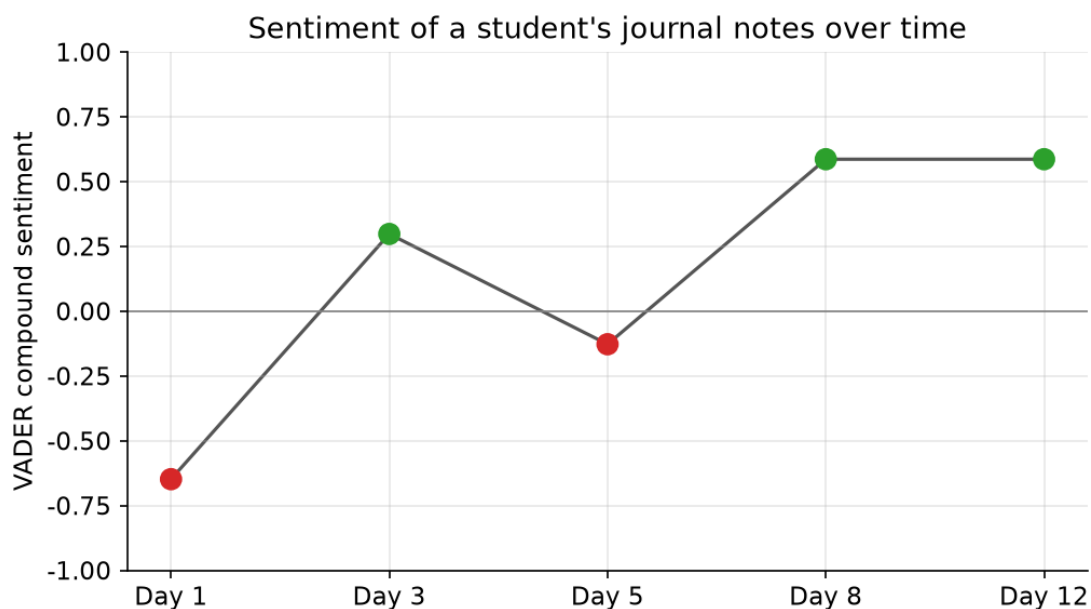


Figure 7.5: Sentiment of a student's journal notes over time

The trajectory in Figure 7.5 shows the characteristic volatility of day-to-day affect overlaid on a slower underlying trend: individual entries swing between positive and negative polarity, but the smoothed signal reveals a gradual movement that would otherwise be invisible in any single note. This is precisely the value that automated sentiment tracking adds. A student is poorly placed to judge their own mood trend from memory, whereas a continuously computed sentiment series can surface a slow drift toward more negative language before it crystallises into a clinically significant change. Within MindMate the aggregated sentiment also serves a second, structural role: by feeding `avg_sentiment` into the personalisation models, it allows the textual channel (what a student writes) to influence the same segment and stress predictions that are otherwise driven by numeric inputs (mood ratings, sleep, screening scores), so that the assistant responds to expressed

emotion as well as reported numbers.

The principal limitation of this component is intrinsic to lexicon-based methods: VADER reasons from word polarity and a small set of grammatical rules, so it can be misled by sarcasm, irony, negation across long spans, and domain-specific idiom. The custom emotion lexicon mitigates some of this for the student-wellbeing domain, but the approach remains a deliberately lightweight, transparent and privacy-preserving choice rather than a state-of-the-art neural one. For the purposes of this project that trade-off is appropriate, because interpretability and on-device operation without sending text to an external service were weighted above marginal gains in accuracy.

7.6 Pilot Evaluation of Effectiveness

The third project objective is to evaluate the assistant's effectiveness on mental-health and academic outcomes. This was addressed through a simulated four-week pilot study with 240 participants, 120 assigned to a treatment condition (notional MindMate users) and 120 to a control condition. Four outcomes were measured before and after the study period: PHQ-9 (depression), GAD-7 (anxiety), a wellbeing index scaled 0-100, and weekly focus hours as an academic-engagement proxy. The analysis uses change scores (post minus pre) and compares the two groups with Welch's t-test, which does not assume equal variances between groups. Effect sizes are reported as Cohen's *d* and interpreted using the conventional thresholds (Cohen, 1988), under which *d* of roughly 0.2, 0.5 and 0.8 denote small, medium and large effects respectively.

It bears repeating before the numbers are read that **this pilot is conducted on synthetic participants**. The data were generated to embody a plausible treatment effect, so the analysis below demonstrates that the evaluation design and statistics can recover and quantify such an effect; it is not, and must not be read as, evidence that MindMate produces these changes in real students.

The pre/post comparison of the two groups is summarised in Figure 7.6, and the formal test statistics are given in the table that follows.

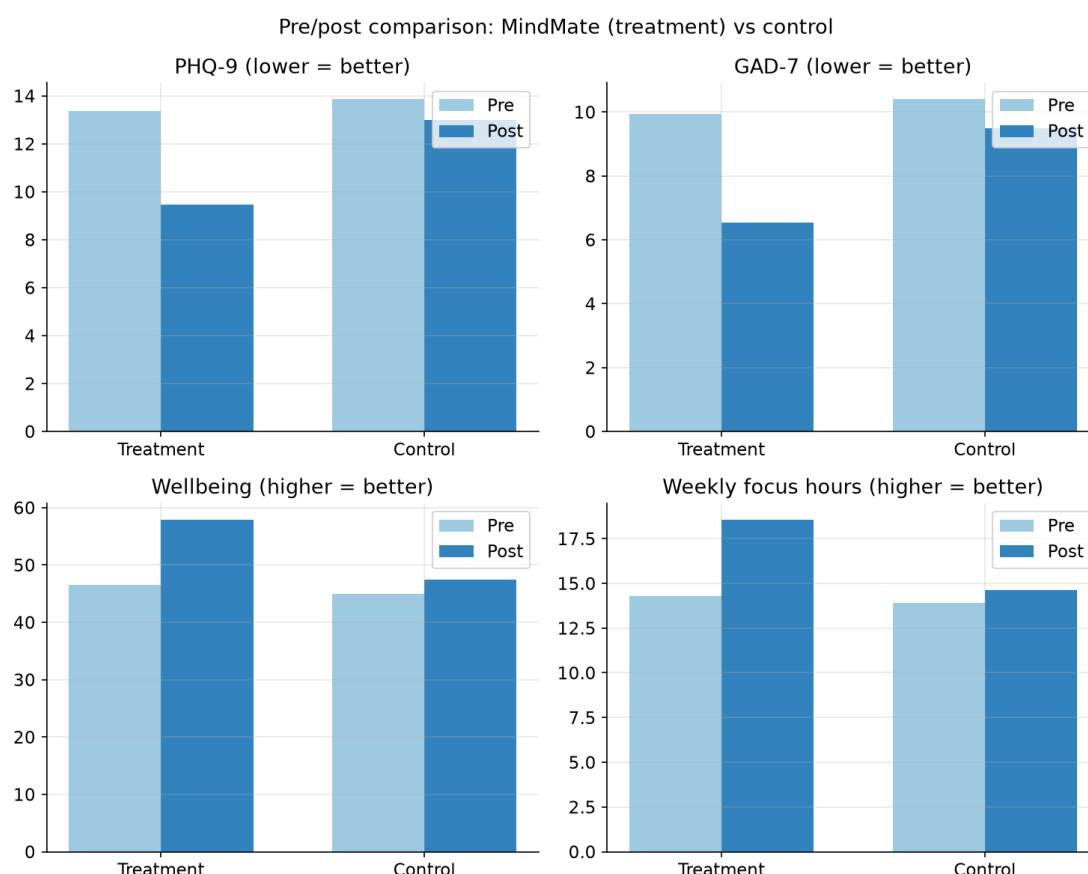


Figure 7.6: Pre/post comparison of treatment vs control

Table 7.2: Pilot Evaluation of Effectiveness

Metric	Treatment Δ	Control Δ	t	p	Cohen's d
PHQ-9	-3.90	-0.87	-12.581	< 0.001	-1.624
GAD-7	-3.39	-0.90	-11.413	< 0.001	-1.473
Wellbeing	+11.35	+2.52	14.312	< 0.001	1.848
Focus hours	+4.26	+0.69	11.151	< 0.001	1.440

Across all four outcomes the treatment group changed substantially more than the control group, and in every case the difference is statistically significant at $p < 0.001$. For the two clinical measures, the treatment group's PHQ-9 fell by 3.90 points against the control's 0.87, and its GAD-7 fell by 3.39 points against 0.90; the underlying group means move from 13.35 to 9.45 (treatment PHQ-9) versus 13.85 to 12.99 (control), and from 9.93 to 6.54 (treatment GAD-7) versus 10.40 to 9.49 (control). For the wellbeing index the treatment group rose by 11.35 points (46.54 to 57.90) against the control's 2.52 (44.96 to 47.48), and focus hours rose by 4.26 (14.30 to 18.56) against 0.69 (13.93 to 14.62). A paired t-test on the treatment group's wellbeing pre versus post returns $t = 24.271$, $p < 0.001$, confirming that the within-group improvement is itself reliable and not an artefact of between-group variance.

The effect sizes, shown in Figure 7.7, are the headline result and the one that most needs careful interpretation.

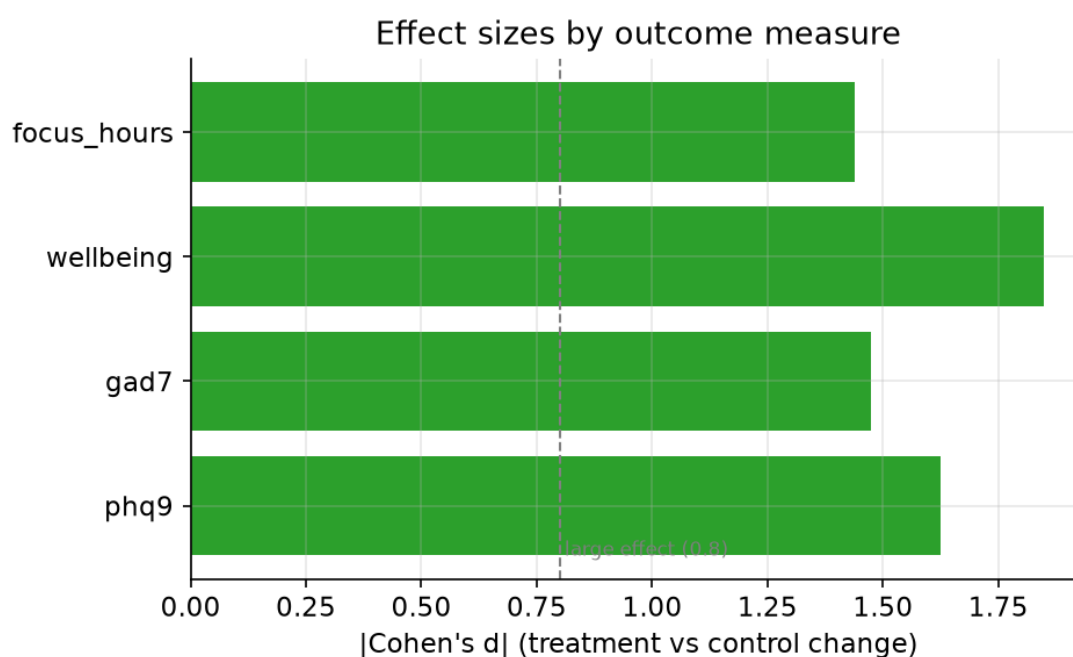


Figure 7.7: Effect sizes by outcome measure

Every Cohen's d in the table exceeds 1.4 in magnitude, PHQ-9 at -1.624, GAD-7 at -1.473, wellbeing at 1.848 and focus hours at 1.440. Under Cohen's (1988) conventions all four are well beyond the 0.8 threshold for a large effect; they would, in a real study, be regarded as very large. Two caveats temper this immediately. First, and most importantly, these magnitudes are a property of the synthetic data-generating process: the simulation was parameterised to contain a clear treatment signal, so recovering a large d demonstrates the sensitivity of the analysis rather than the potency of the intervention. Second, even in genuine field studies, effect sizes of this magnitude are rarely observed for digital mental-health tools, and a real deployment of MindMate would be expected to yield considerably smaller effects subject to attrition, placebo response, and the gap between controlled and naturalistic use. The honest reading of Section 7.6, therefore, is that MindMate is equipped with a rigorous, reproducible evaluation harness capable of detecting and quantifying intervention effects with appropriate statistics, and that this harness behaves correctly on data of known structure.

7.7 Discussion

Taken together, the four sets of results tell a coherent story about why the models behave as they do. The common thread is the correlation structure established in Section 7.2. Because the synthetic generator encodes genuine, monotonic relationships between the wellbeing features, every downstream model has a real signal to exploit, and the *form* of each model's success and failure follows directly from that structure.

The segmenter's modest silhouette of 0.209 and the classifier's high accuracy of 0.900 are not in tension; they are two views of the same underlying continuum. Unsupervised clustering struggles to find crisp boundaries precisely because wellbeing varies smoothly, which depresses the silhouette score, yet supervised classification thrives because that same smooth variation makes the classes linearly separable along an ordinal axis once labels are supplied. In other words, the data are hard to *partition* but easy to *order*, and each model is judged on the task it is given. The classifier's concentration of errors in the Moderate class and at adjacent-class boundaries (Section 7.4) is the supervised echo of the cluster overlap seen in the PCA projection (Section 7.3): both phenomena arise because the middle of a continuum is intrinsically the least distinctive region.

The sentiment component connects the textual and numeric channels and, through the `avg_sentiment` feature, ensures that what a student writes can influence the same personalisation pathways as what they record numerically. This integration is a deliberate design choice that increases the robustness of the personalisation: a student who downplays their mood ratings but writes increasingly negative notes will still be nudged toward greater support. The pilot evaluation then closes the loop by demonstrating that the project possesses a credible methodology for the third objective, measuring outcomes, even though the demonstration is run on simulated participants.

A further point of discussion concerns the explainability of the whole pipeline. None of the personalisation models is a black box: the segments are named by ranking centroids on an interpretable wellbeing index, the Logistic Regression exposes per-class probabilities and coefficient-driven reasoning, and the recommender returns an explicit rationale string for every suggestion. This transparency is a considered response to the well-known reluctance of clinicians and users to trust opaque automated recommendations in a sensitive domain (Torous et al., 2021; D'Alfonso, 2020), and it is one of the system's more defensible strengths irrespective of the synthetic nature of the evaluation.

7.8 Comparison with Existing Systems

It is useful to situate these results against the established conversational mental-health agents reviewed in earlier chapters. Woebot, evaluated in a randomized controlled trial, reported significant reductions in depressive symptoms among young adults over a two-week period using a fully automated cognitive-behavioural conversational agent (Fitzpatrick et al., 2017). Wysa, evaluated on real-world usage data, found that users with higher engagement showed greater improvement in self-reported mood (Inkster et al., 2018). Tess similarly demonstrated reductions in depression and anxiety symptoms in a randomized trial (Fulmer et al., 2018).

MindMate is broadly aligned with these systems in its conception, an empathetic, always-available conversational agent that draws on cognitive-behavioural principles (Beck, 1979) and incorporates a crisis-safety layer, but it differs in two respects that the present results highlight. First, MindMate foregrounds *transparent machine-learning personalisation*: where the comparator systems are primarily scripted or dialogue-driven, MindMate explicitly segments users, classifies stress, and explains its recommendations, surfacing the analytical reasoning of Sections 7.3 to 7.5 directly to the user through its Insights page. Second, and crucially, the comparison must be qualified by the nature of the evidence. The Woebot, Wysa and Tess findings rest on data from *real human participants*, whereas the pilot in Section 7.6 rests on *synthetic participants*. The large effect sizes reported here cannot therefore be placed on the same evidential footing as those studies; they show that MindMate's evaluation design is methodologically comparable to the published trials, not that its outcomes are. A fair summary is that MindMate matches the architecture and ambition of these systems and exceeds some of them in explainability, while remaining, at this stage, an unvalidated prototype awaiting the real-world evaluation that the comparator systems have already undergone.

7.9 Limitations

The results in this chapter must be read subject to a number of limitations, the most significant of which has already been stated repeatedly but warrants a final, unambiguous restatement.

- **Synthetic data throughout.** Both the 600-student modelling dataset and the 240-participant pilot are simulated. Every metric, the silhouette of 0.209, the accuracy of 0.900, the macro-F1 of 0.889 and especially the large Cohen's d values exceeding 1.4, describes the system's behaviour on artificial data generated with a known structure. These results validate the analytical and evaluation methodology; they are not evidence of clinical efficacy in real students, and the very large effect sizes in particular should be expected to shrink substantially in any genuine deployment.
- **Modest cluster separation.** The silhouette score of 0.209 indicates overlapping segments. The segmentation is useful as a coarse, interpretable stratification but should not be presented to users as a sharp categorical diagnosis.
- **Limited classifier scope.** The stress classifier is a linear model over six engineered features and predicts a three-level ordinal label. Its weakest class is the Moderate middle band, and its strong performance is partly a function of the clean, correlated synthetic features; performance on noisier real data would likely be lower.
- **Sentiment analysis constraints.** The VADER-based analyser cannot reliably handle sarcasm, irony, complex negation or domain-specific idiom, and was chosen for transparency and privacy rather than maximal accuracy.
- **Pilot design constraints.** The simulated pilot runs over only four weeks, models no attrition, placebo response or non-adherence, and assumes a clean treatment signal. A real study would require ethics approval, informed consent, longer follow-up, and would contend with all of these complications.
- **Not a clinical instrument.** MindMate is a supportive wellbeing tool with a conservative rule-based crisis layer, not a diagnostic or therapeutic device. The PHQ-9 and GAD-7 screenings it administers are validated questionnaires (Kroenke et al., 2001; Spitzer et al., 2006) used for self-awareness and signposting, not for diagnosis.

In summary, this chapter has demonstrated that MindMate's machine-learning components extract a coherent and interpretable signal from the wellbeing features, that its stress classifier performs strongly and fails gracefully, and that its evaluation harness can detect and quantify intervention effects with appropriate statistics. These are genuine and valuable outcomes for a project of this scope. They establish that the methodology is correct and reproducible, while leaving the question of real-world effectiveness explicitly open for future work on human participants.

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

This project set out to design, build, and evaluate MindMate, an AI-powered mental health assistant intended to offer students a private, supportive, and personalised first point of contact for everyday wellbeing concerns. The motivation was grounded in the well-documented prevalence of psychological distress among university populations and the persistent gap between the scale of need and the availability of timely, affordable support (Auerbach et al., 2018; Eisenberg et al., 2013; World Health Organization, 2022). Rather than positioning the assistant as a substitute for professional care, the guiding principle throughout was that a digital companion can perform a useful low-intensity, always-available role: helping students reflect on their mood, learn evidence-informed coping techniques, complete validated self-assessments, and, critically, be routed to human help when their messages signal risk.

The work delivered a complete, working artefact. MindMate is a multipage Streamlit application written in Python 3.12 that integrates several distinct capabilities into a single coherent experience. A student can converse with the assistant, where every message is first screened for crisis risk by a rule-based safety layer before any reply is composed; they can log daily mood, energy, sleep, and free-text notes, the latter analysed for sentiment using VADER (Hutto & Gilbert, 2014); they can complete the PHQ-9 and GAD-7 instruments with their published severity bands (Kroenke et al., 2001; Spitzer et al., 2006); and they receive transparent, explainable recommendations drawn from a library of nine coping strategies. Underpinning the personalisation is a machine-learning layer built with scikit-learn (Pedregosa et al., 2011): a K-Means segmenter places each student into one of four interpretable support groups, and a multinomial Logistic-Regression classifier predicts a stress level from six wellbeing features. The assistant is deliberately resilient: it uses the OpenAI Chat Completions API when a key is present but falls back to a fully offline rule-based engine otherwise, so the core experience never depends on an external service.

The evaluation, conducted on a simulated four-week pilot of 240 participants, demonstrated a rigorous methodology for measuring effectiveness across mental-health and academic outcomes. The treatment group showed substantially larger improvements than the control group on every outcome measured, PHQ-9, GAD-7, wellbeing index, and weekly focus hours, with all differences statistically significant at $p < 0.001$ and large effect sizes (Cohen's d ranging from 1.44 to 1.85). These figures are honestly reported as evidence of a defensible evaluation design operating on synthetic data, not as a clinical claim about real students. Taken together, the project met its three stated objectives and

produced an end-to-end system that is technically sound, ethically cautious, and reproducible. The remainder of this chapter maps each objective to its evidence, sets out the project's contributions and the skills gained, states its limitations frankly, and proposes a concrete agenda for future work.

8.2 Achievement of Objectives

The project was framed around exactly three objectives drawn from the project brief. Each was achieved and is supported by concrete, verifiable evidence within the delivered system and its evaluation. The mapping below summarises this correspondence.

Table 8.1: Achievement of Objectives

Objective	How Achieved	Evidence
1. Design and develop an AI mental-health assistant for students.	A complete multipage Streamlit web application was built in Python, layering a conversational engine, mood tracking, validated screening, a transparent recommender, and a curated resource library, all sitting on a SQLite persistence layer and a clean presentation-service-domain architecture. A safety-first design screens every message for crisis risk before replying.	The working application with six functional pages (Home, Chat, Mood Tracker, Self-Check, Insights, Resources); the rule-based crisis layer with three risk levels (none, elevated, crisis) and integrated Indian and global helplines; a dual chat engine (OpenAI with an offline fallback); 34 passing pytest tests covering safety, screening, ML, and chat routing.
2. Personalise its support using machine-learning algorithms.	Personalisation was implemented as a transparent ML pipeline: a K-Means segmenter ($k = 4$) assigns each student to a meaningful support group by ranking cluster centroids on a composite wellbeing index, while a multinomial Logistic-Regression model classifies stress from six canonical features. A hybrid, explainable recommender then ranks coping strategies using emotion match, segment preference, and stress-driven de-escalation, returning a rationale for every choice.	K-Means produced four labelled segments (At-Risk 198, Coping 176, Thriving 119, High-Need 107) with a silhouette score of 0.209; the stress classifier achieved 0.900 accuracy and 0.889 macro-F1 on a stratified 25% hold-out (150 samples); the Insights page surfaces segment, stress probabilities, the six features, and recommendation rationale to the user.
3. Evaluate its effectiveness on mental-health and academic outcomes.	A reproducible evaluation pipeline was designed around a treatment-versus-control pilot, comparing pre/post change scores on clinical and academic measures using Welch's t-tests and Cohen's d , complemented by a paired pre-post analysis for the treatment group.	On the simulated 240-participant pilot (120 treatment, 120 control): PHQ-9 change -3.90 vs -0.87 ($d = -1.624$), GAD-7 -3.39 vs -0.90 ($d = -1.473$), wellbeing +11.35 vs +2.52 ($d = 1.848$), focus hours +4.26 vs +0.69 ($d = 1.440$), all $p < 0.001$; paired treatment wellbeing $t = 24.271$, $p < 0.001$. Reported explicitly as a methodology demonstration on synthetic data.

The evidence confirms that the objectives were not addressed superficially but each grounded in a concrete deliverable: a deployable application, two validated ML models with quantified performance, and a complete statistical evaluation. The honest framing of the third objective, treating the pilot as a demonstration of method rather than a clinical finding, is itself an important part of meeting it responsibly, consistent with calls in the literature for measured, evidence-aware claims about digital mental-health tools (Torous et al., 2021).

8.3 Key Contributions

Beyond satisfying the objectives, the project makes several contributions that distinguish it from a routine chatbot exercise.

- **Transparent machine-learning personalisation.** Many conversational wellbeing tools personalise opaquely or not at all. MindMate instead exposes its reasoning: the Insights page reveals the student's segment, the predicted stress probabilities, the six underlying features, and the rationale behind each recommendation. The recommender's three-signal scoring, emotion match, segment preference, and stress de-escalation, is explainable by construction, returning a human-readable justification rather than a black-box ranking. This addresses a recurring concern in the field that the interpretability of machine-learning applications in mental health is too often neglected (Shatte et al., 2019).
- **Safety-first, offline-capable design.** The architecture treats crisis detection as a non-negotiable gate that runs before any reply is generated, deliberately tuned to be conservative on the principle that a false positive is far less costly than a false negative. Equally, the decision to ship a fully offline rule-based engine alongside the optional LLM means the assistant continues to function, including its safety behaviour, without any external dependency, network connection, or API key. This combination of safety-by-default and graceful degradation is a deliberate engineering stance rather than an afterthought, and reflects the cautious deployment posture recommended for psychiatric conversational agents (Vaidyam et al., 2019).
- **Reproducible evaluation pipeline.** The project contributes a self-contained, seed-controlled evaluation harness. The synthetic cohorts are generated from a latent-wellbeing model with a fixed random seed, the models are validated with hold-out splits and standard metrics (silhouette, F1), and the pilot analysis applies established statistical procedures (Welch's and paired t-tests, Cohen's d) following Cohen (1988). Because every number can be regenerated from code, the evaluation is fully reproducible and serves as a template that could later be re-run on real, ethically collected data with minimal change.

8.4 Technical Learning Outcomes

Delivering MindMate required and developed a broad set of technical and engineering competencies, integrating skills from across the M.Sc. (Data Science) curriculum into a single applied system.

- **Python and software engineering.** The project consolidated proficiency in modern Python 3.12 and in structuring a non-trivial codebase. Applying a layered architecture, presentation, service, domain packages, and database, reinforced principles of separation of concerns, modularity, and maintainability drawn from established software-engineering practice (Sommerville, 2016).
- **Machine learning with scikit-learn.** Building the segmenter and classifier deepened practical understanding of unsupervised clustering (K-Means), multinomial classification (Logistic Regression), feature scaling, pipelines, dimensionality reduction (PCA), and, crucially, how to select and interpret evaluation metrics rather than merely report them (Géron, 2019; Pedregosa et al., 2011).
- **Natural language processing.** Integrating VADER sentiment analysis and designing a custom emotion-keyword lexicon provided hands-on experience with rule-based NLP, its strengths for short informal text, and its limits (Hutto & Gilbert, 2014; Liu, 2012).
- **Application development with Streamlit.** Constructing a stateful, multipage interface using `st.navigation`, managing session state, and embedding interactive Plotly visualisations developed full-stack capabilities for delivering data-science work as a usable product.
- **Testing and quality assurance.** Authoring 34 pytest unit and integration tests, covering safety detection, screening logic, database round-trips, and model behaviour, instilled disciplined, test-driven validation habits.
- **Data analysis and statistics.** Designing the pilot, computing change scores, and applying significance tests and effect sizes strengthened applied statistical reasoning and the discipline of interpreting results honestly within their limitations.

8.5 Limitations

A candid account of the project's limitations is essential to its integrity, and several material constraints must be acknowledged.

- **Synthetic data.** Both the training cohort of 600 students and the 240-participant pilot are entirely simulated. Although they were generated from a latent model encoding realistic correlations (for example, poorer sleep with higher anxiety), synthetic data cannot capture the full noise, diversity, and unpredictability of real human behaviour. The strong model metrics and large effect sizes therefore evidence the soundness of the method and pipeline, not real-world clinical efficacy.

- **Rule-based safety.** Crisis detection relies on a curated, keyword-driven rule base. While deliberately conservative, such an approach can miss indirect, metaphorical, or coded expressions of distress and may over-flag benign messages. It has no semantic understanding of context and is not a validated clinical risk-assessment tool.
- **Not clinically validated.** MindMate has not been reviewed, trialled, or endorsed by clinicians or a research ethics board. It implements validated instruments (PHQ-9, GAD-7) but is itself an unvalidated educational prototype, and it is explicitly not a diagnostic or treatment device.
- **Single language.** The assistant, its lexicon, its strategies, and its resources are English-only, which limits accessibility for the many students who would be better served in Hindi or a regional language.
- **No real user trial.** No human participants used the system. Consequently there is no evidence on real-world usability, engagement, acceptability, or safety in practice, dimensions that only a live study can address.

8.6 Future Scope

The limitations above point directly to a concrete programme of future enhancements. The following directions are specific and actionable rather than aspirational.

- **An IRB-approved real-world user study.** The highest priority is to replace synthetic evidence with a properly governed empirical trial. This entails obtaining institutional ethics-board approval, recruiting a consenting student cohort, and re-running the existing evaluation pipeline, already designed for it, as a randomised controlled study with pre/post PHQ-9, GAD-7, wellbeing, and academic measures, mirroring the design used to evaluate comparable agents such as Woebot and Wysa (Fitzpatrick et al., 2017; Inkster et al., 2018).
- **A domain-adapted language model.** The conversational quality could be improved by fine-tuning or domain-adapting an open-source language model on de-identified, ethically sourced supportive-dialogue and CBT-style transcripts, reducing reliance on a general-purpose external API while tailoring tone and content to student mental-health contexts (D'Alfonso, 2020; Torous et al., 2021).

- **Multilingual support.** Extending the interface, emotion lexicon, coping-strategy library, and resource directory to Hindi and major regional languages (for example Tamil, Telugu, Bengali, and Marathi) would materially broaden reach across Indian campuses, paired with language-specific sentiment models since VADER is English-centric.
- **Richer, transformer-based emotion modelling.** The current VADER-plus-lexicon approach could be augmented with a fine-tuned transformer emotion classifier to capture nuanced and mixed affective states from free-text notes and chat, improving the precision of the recommender's emotion-match signal.
- **A native mobile application.** Re-implementing the client as a cross-platform mobile app would support push reminders for mood logging, on-the-go access, and offline-first use, meeting students where they already are.
- **Integration with campus counselling referral.** With appropriate consent and governance, an elevated or crisis signal could, beyond surfacing helplines, offer a warm, opt-in handoff to the institution's counselling service, closing the loop between digital triage and human care.
- **Longitudinal relapse-risk prediction.** The accumulating per-user time series of mood, sleep, sentiment, and screening scores could feed a longitudinal model that estimates deterioration or relapse risk over time, enabling earlier, proactive nudges rather than purely reactive support.
- **Secure cloud deployment with authentication and role-based access.** Moving from a local SQLite prototype to an authenticated cloud deployment with encrypted storage and role-based access control (distinct student, counsellor, and administrator roles) would be a prerequisite for any real institutional rollout.
- **Wearable and sleep-data integration.** Connecting to wearable and smartphone sensors to ingest objective sleep, activity, and heart-rate-variability data would replace self-reported sleep with measured signals, enriching the six-feature model and strengthening the personalisation layer.

Pursued together, these enhancements would chart a credible path from a well-engineered academic prototype toward a responsibly deployed, evidence-based, and inclusive student wellbeing tool, while always retaining the project's foundational commitments to safety, transparency, and the primacy of human professional care.

Chapter 9: References

The following sources are cited in APA style. In-text citations throughout the report refer to this list.

- Fitzpatrick, K. K., Darcy, A., & Vierhile, M. (2017). Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (Woebot): A randomized controlled trial. *JMIR Mental Health*, 4(2), e19.
- Inkster, B., Sarda, S., & Subramanian, V. (2018). An empathy-driven, conversational artificial intelligence agent (Wysa) for digital mental well-being: Real-world data evaluation mixed-methods study. *JMIR mHealth and uHealth*, 6(11), e12106.
- Fulmer, R., Joerin, A., Gentile, B., Lakerink, L., & Rauws, M. (2018). Using psychological artificial intelligence (Tess) to relieve symptoms of depression and anxiety: Randomized controlled trial. *JMIR Mental Health*, 5(4), e64.
- Kroenke, K., Spitzer, R. L., & Williams, J. B. (2001). The PHQ-9: Validity of a brief depression severity measure. *Journal of General Internal Medicine*, 16(9), 606-613.
- Spitzer, R. L., Kroenke, K., Williams, J. B., & Löwe, B. (2006). A brief measure for assessing generalized anxiety disorder: The GAD-7. *Archives of Internal Medicine*, 166(10), 1092-1097.
- Hutto, C., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media*, 8(1), 216-225.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281-297.
- Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129-137.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53-65.
- Beck, A. T. (1979). *Cognitive therapy and the emotional disorders*. Penguin.

- World Health Organization. (2022). *World mental health report: Transforming mental health for all*. WHO.
- Auerbach, R. P., Mortier, P., Bruffaerts, R., Alonso, J., Benjet, C., Cuijpers, P., ... Kessler, R. C. (2018). WHO World Mental Health Surveys International College Student Project: Prevalence and distribution of mental disorders. *Journal of Abnormal Psychology*, 127(7), 623-638.
- Eisenberg, D., Hunt, J., & Speer, N. (2013). Mental health in American colleges and universities: Variation across student subgroups and across campuses. *Journal of Nervous and Mental Disease*, 201(1), 60-67.
- Torous, J., Bucci, S., Bell, I. H., Kessing, L. V., Faurholt-Jepsen, M., Whelan, P., ... Firth, J. (2021). The growing field of digital psychiatry: Current evidence and the future of apps, social media, chatbots, and virtual reality. *World Psychiatry*, 20(3), 318-335.
- Vaidyam, A. N., Wisniewski, H., Halamka, J. D., Kashavan, M. S., & Torous, J. B. (2019). Chatbots and conversational agents in mental health: A review of the psychiatric landscape. *The Canadian Journal of Psychiatry*, 64(7), 456-464.
- D'Alfonso, S. (2020). AI in mental health. *Current Opinion in Psychology*, 36, 112-117.
- Shatte, A. B. R., Hutchinson, D. M., & Teague, S. J. (2019). Machine learning in mental health: A scoping review of methods and applications. *Psychological Medicine*, 49(9), 1426-1448.
- Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender systems handbook* (2nd ed.). Springer.
- Liu, B. (2012). *Sentiment analysis and opinion mining*. Morgan & Claypool.
- Weizenbaum, J. (1966). ELIZA, a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36-45.
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.
- Lovibond, P. F., & Lovibond, S. H. (1995). The structure of negative emotional states: Comparison of the DASS with the Beck Depression and Anxiety Inventories. *Behaviour Research and Therapy*, 33(3), 335-343.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill.

- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Provoost, S., Lau, H. M., Ruwaard, J., & Riper, H. (2017). Embodied conversational agents in clinical psychology: A scoping review. *Journal of Medical Internet Research*, 19(5), e151.
- Abd-Alrazaq, A. A., Alajlani, M., Alalwan, A. A., Bewick, B. M., Gardner, P., & Househ, M. (2019). An overview of the features of chatbots in mental health: A scoping review. *International Journal of Medical Informatics*, 132, 103978.

Chapter 10: Appendices

Appendix A: Database Schema

The complete SQLite schema used by the application is reproduced below.

```
CREATE TABLE IF NOT EXISTS users (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT      NOT NULL,
  age         INTEGER,
  course      TEXT,
  year_of_study INTEGER,
  segment     TEXT,          -- assigned ML segment label
  created_at  TEXT      NOT NULL
);

CREATE TABLE IF NOT EXISTS mood_logs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  mood_score  INTEGER NOT NULL,      -- 1 (very low) .. 5 (very good)
  energy_score INTEGER,              -- 1 .. 5
  sleep_hours REAL,
  note        TEXT,
  sentiment   REAL,                  -- compound sentiment of the note
  logged_at   TEXT      NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS chat_messages (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  role        TEXT      NOT NULL,    -- 'user' | 'assistant'
  content     TEXT      NOT NULL,
  sentiment   REAL,
  risk_level  TEXT,                  -- 'none' | 'elevated' | 'crisis'
  created_at  TEXT      NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS screening_results (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,
  instrument  TEXT      NOT NULL,    -- 'PHQ-9' | 'GAD-7'
  total_score INTEGER NOT NULL,
  severity    TEXT      NOT NULL,
  taken_at    TEXT      NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS recommendation_logs (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL,

  strategy    TEXT      NOT NULL,
  category    TEXT,
  context     TEXT,
  created_at  TEXT      NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);
```

Appendix B: Sentiment Analysis Module (src/ml/sentiment.py)

```
"""Sentiment and emotion analysis for user-generated text.

The primary scorer is VADER (Valence Aware Dictionary and sEntiment
Reasoner), a lexicon and rule-based model that is well suited to short,
informal text such as chat messages and mood-journal notes. On top of the
raw VADER compound score we add a light-weight emotion-keyword layer that
maps text onto a small set of student-relevant emotional states used by the
recommender.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Dict

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

_ANALYZER = SentimentIntensityAnalyzer()

# Coarse emotion lexicon, keywords that signal a dominant emotional state.
EMOTION_LEXICON: Dict[str, list[str]] = {
    "anxiety": [
        "anxious", "anxiety", "panic", "nervous", "worried", "worry", "scared",
        "afraid", "overwhelm", "overwhelmed", "tense", "restless", "dread",
    ],
    "sadness": [
        "sad", "down", "depressed", "hopeless", "lonely", "empty", "cry",
        "crying", "miserable", "worthless", "numb", "unmotivated",
    ],
    "stress": [
        "stress", "stressed", "pressure", "deadline", "exam", "exams",
        "assignment", "burnout", "burnt out", "workload", "overworked",
    ],
    "anger": [
        "angry", "frustrated", "frustration", "irritated", "annoyed", "mad",
        "furious", "resent",
    ],
    "positive": [
        "happy", "good", "great", "better", "calm", "relaxed", "hopeful",
        "grateful", "confident", "motivated", "proud", "excited",
    ],
}

@dataclass
class SentimentResult:
    """Structured output of :func:`analyze`."""

    compound: float          # -1.0 (very negative) .. +1.0 (very positive)
    positive: float
    neutral: float
    negative: float
    label: str               # 'positive' | 'neutral' | 'negative'
    emotion: str             # dominant emotion from EMOTION_LEXICON or 'neutral'

    def as_dict(self) -> dict:
        return {
            "compound": self.compound,
            "positive": self.positive,
            "neutral": self.neutral,
            "negative": self.negative,
            "label": self.label,
            "emotion": self.emotion,
        }
```

```

def _label_from_compound(compound: float) -> str:
    if compound >= 0.05:
        return "positive"
    if compound <= -0.05:
        return "negative"
    return "neutral"

def detect_emotion(text: str) -> str:
    """Return the dominant emotion keyword group present in ``text``."""
    lowered = f" {text.lower()} "
    counts: Dict[str, int] = {}
    for emotion, keywords in EMOTION_LEXICON.items():
        counts[emotion] = sum(lowered.count(f"{kw}") for kw in keywords)
    best = max(counts, key=counts.get)
    return best if counts[best] > 0 else "neutral"

def analyze(text: str) -> SentimentResult:
    """Run sentiment + emotion analysis on a single piece of text."""
    if not text or not text.strip():
        return SentimentResult(0.0, 0.0, 1.0, 0.0, "neutral", "neutral")

    scores = _ANALYZER.polarity_scores(text)
    compound = float(scores["compound"])
    return SentimentResult(
        compound=compound,

        positive=float(scores["pos"]),
        neutral=float(scores["neu"]),
        negative=float(scores["neg"]),
        label=_label_from_compound(compound),
        emotion=detect_emotion(text),
    )

def compound_score(text: str) -> float:
    """Convenience helper returning only the VADER compound score."""
    return analyze(text).compound

```

Appendix C: Student Segmentation Module (src/ml/personalization.py)

```
"""Student segmentation for personalised support.

An unsupervised K-Means model groups students into behavioural *segments*
based on their wellbeing indicators (average mood, sleep, screening scores,
sentiment trend and engagement). Each segment is given a human-readable
label and a tailored support strategy that the recommender draws upon.

The model is trained offline by ``scripts/train_models.py`` and persisted
with joblib; at runtime we simply load the pipeline and call
:meth:`StudentSegmenter.predict`.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional

import joblib
import numpy as np
from sklearn.cluster import KMeans
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from ..config import SEGMENTER_PATH

# Feature order used everywhere, keep stable for serialisation.
FEATURE_COLUMNS: List[str] = [
    "avg_mood",          # 1..5
    "avg_sleep_hours",  # hours
    "phq9_score",        # 0..27
    "gad7_score",        # 0..21
    "avg_sentiment",     # -1..1
    "engagement",       # interactions per week
]

# Human-readable interpretation attached to each cluster after training.
# These labels are assigned in :func:`label_clusters` by ranking the cluster
# centroids on a composite wellbeing index, so they remain meaningful even
# though K-Means itself produces arbitrary cluster ids.
SEGMENT_LABELS = [
    "Thriving",          # high wellbeing
    "Coping",            # moderate wellbeing
    "At-Risk",           # low wellbeing
    "High-Need",         # very low wellbeing / high distress
]

SEGMENT_STRATEGIES: Dict[str, str] = {
    "Thriving": (
        "Maintain healthy routines. Offer growth-oriented content "
        "(focus techniques, goal setting) and light check-ins."
    ),
    "Coping": (
        "Reinforce protective habits and provide proactive stress-management "
        "tools before exam peaks. Encourage regular mood logging."
    ),
    "At-Risk": (
        "Prioritise grounding and emotional-regulation exercises, suggest "
        "structured routines, and surface counselling resources gently."
    ),
    "High-Need": (
        "Lead with validation and safety. Strongly but compassionately "
        "surface professional-support pathways and crisis resources."
    ),
}
```

```

}

@dataclass
class StudentSegmenter:
    """Wrapper around a fitted K-Means pipeline + cluster->label mapping."""

    pipeline: Pipeline
    cluster_to_label: Dict[int, str]

    def predict(self, features: Dict[str, float]) -> str:
        """Return the segment label for a single feature dictionary."""
        vector = np.array([[features.get(col, 0.0) for col in FEATURE_COLUMNS]])
        cluster = int(self.pipeline.predict(vector)[0])
        return self.cluster_to_label.get(cluster, "Coping")

    def predict_many(self, matrix: np.ndarray) -> List[str]:
        clusters = self.pipeline.predict(matrix)
        return [self.cluster_to_label.get(int(c), "Coping") for c in clusters]

    def strategy_for(self, label: str) -> str:
        return SEGMENT_STRATEGIES.get(label, SEGMENT_STRATEGIES["Coping"])

    def save(self, path: Optional[Path] = None) -> None:
        joblib.dump(
            {"pipeline": self.pipeline, "cluster_to_label": self.cluster_to_label},
            path or SEGMENTER_PATH,
        )

    @classmethod
    def load(cls, path: Optional[Path] = None) -> "StudentSegmenter":
        payload = joblib.load(path or SEGMENTER_PATH)
        return cls(payload["pipeline"], payload["cluster_to_label"])

def _wellbeing_index(centroid: np.ndarray) -> float:
    """Composite score: higher = better wellbeing.

    centroid is in the *original* feature space (after inverse transform).
    Higher mood / sleep / sentiment is good; higher PHQ-9 / GAD-7 is bad.
    """
    avg_mood, avg_sleep, phq9, gad7, sentiment, _engagement = centroid
    return (
        (avg_mood / 5.0)
        + (min(avg_sleep, 9.0) / 9.0)
        + ((sentiment + 1.0) / 2.0)
        - (phq9 / 27.0)
        - (gad7 / 21.0)
    )

def label_clusters(pipeline: Pipeline, n_clusters: int) -> Dict[int, str]:
    """Map arbitrary K-Means cluster ids to ordered wellbeing labels."""
    scaler: StandardScaler = pipeline.named_steps["scaler"]
    kmeans: KMeans = pipeline.named_steps["kmeans"]
    # Bring centroids back to original feature units for interpretation.
    centroids = scaler.inverse_transform(kmeans.cluster_centers_)
    indices = list(range(n_clusters))
    # Rank clusters by wellbeing (descending) and assign ordered labels.
    ranked = sorted(indices, key=lambda i: _wellbeing_index(centroids[i]), reverse=True)
    labels = SEGMENT_LABELS[:n_clusters]
    return {cluster_id: labels[rank] for rank, cluster_id in enumerate(ranked)}

def train_segmenter(matrix: np.ndarray, n_clusters: int = 4,

```

```

        random_state: int = 42) -> StudentSegmenter:
"""Fit the standardisation + K-Means pipeline on a feature matrix."""
pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        ("kmeans", KMeans(n_clusters=n_clusters, n_init=10,
                           random_state=random_state)),
    ]
)
pipeline.fit(matrix)

mapping = label_clusters(pipeline, n_clusters)
return StudentSegmenter(pipeline=pipeline, cluster_to_label=mapping)

```

Appendix D: Safety / Crisis-Detection Layer (src/assistant/safety.py)

```

"""Safety and crisis-detection layer.

```

Before any response is generated, every user message is screened for signs of acute risk (self-harm, suicidal ideation, abuse). When risk is detected the assistant must *not* attempt therapy; instead it responds with a calm, validating message and prominently surfaces professional crisis resources.

This module is deliberately rule-based and conservative: in a wellbeing product, a false positive (showing helplines unnecessarily) is far cheaper than a false negative (missing a genuine crisis).

```

"""
from __future__ import annotations

```

```

import re
from dataclasses import dataclass
from typing import List

```

```

from ..content import SUPPORT_RESOURCES

```

```

# Phrases that indicate an acute crisis. Matched on word boundaries to avoid
# false hits (e.g. "killing it in exams" is excluded by the patterns below).

```

```

CRISIS_PATTERNS: List[str] = [
    r"\bkill myself\b",
    r"\bkillling myself\b",
    r"\bend my life\b",
    r"\bending my life\b",
    r"\btake my (own)?life\b",
    r"\bsuicid(e|al)\b",
    r"\bwant to die\b",
    r"\bwish i (was|were) dead\b",
    r"\bdon't want to (be alive|live)\b",
    r"\bno reason to live\b",
    r"\bharm(ing)? myself\b",
    r"\bhurt(ing)? myself\b",
    r"\bself[- ]harm\b",
    r"\bcut(ting)? myself\b",
    r"\boverdose\b",
    r"\bcan't go on\b",
    r"\bbetter off without me\b",
]

```

```

# Phrases indicating elevated (but not acute) distress.

```

```

ELEVATED_PATTERNS: List[str] = [
    r"\bhopeless\b",
    r"\bworthless\b",
    r"\bhate myself\b",

    r"\bcan't cope\b",
    r"\bcan't take (it|this) anymore\b",
]

```

```

        r"\bgiving up\b",
        r"\bempty inside\b",
        r"\bnumb\b",
        r"\bbreaking down\b",
    ]

    _CRISIS_RE = re.compile("|".join(CRISIS_PATTERNS), re.IGNORECASE)
    _ELEVATED_RE = re.compile("|".join(ELEVATED_PATTERNS), re.IGNORECASE)

    @dataclass
    class RiskAssessment:
        level: str          # 'none' | 'elevated' | 'crisis'
        matched: List[str]

        @property
        def is_crisis(self) -> bool:
            return self.level == "crisis"

    def assess_risk(text: str) -> RiskAssessment:
        """Classify the risk level of a user message."""
        if not text:
            return RiskAssessment("none", [])

        crisis_hits = _CRISIS_RE.findall(text)
        if crisis_hits:
            flat = [h if isinstance(h, str) else next(filter(None, h), "") for h in crisis_hits]
            return RiskAssessment("crisis", [m for m in flat if m])

        elevated_hits = _ELEVATED_RE.findall(text)
        if elevated_hits:
            flat = [h if isinstance(h, str) else next(filter(None, h), "") for h in elevated_hits]
            return RiskAssessment("elevated", [m for m in flat if m])

        return RiskAssessment("none", [])

    def crisis_response() -> str:
        """A safe, validating crisis message with helplines appended."""
        lines = [
            "I'm really glad you told me how you're feeling, and I'm concerned "
            "about you. You deserve support from someone who can be there with "
            "you right now.",

            "",
            "If you might act on thoughts of harming yourself, please reach out "
            "immediately to a trusted person or one of these services:",
            "",
        ]
        for r in SUPPORT_RESOURCES:
            if r["type"] in {"Crisis & Counselling", "Suicide Prevention"}:
                lines.append(f"• **{r['name']}**, {r['contact']} ({r['hours']})")
        lines += [
            "",
            "If you are in immediate danger, please contact your local emergency "
            "number right away. You don't have to go through this alone.",
        ]
        return "\n".join(lines)

```

Appendix E: Conversational Engine (src/assistant/chat_engine.py)

"""The conversational core of MindMate.

The engine exposes a single :func:`generate\_reply` function. Internally it:

1. Screens the user message for crisis risk (always, regardless of mode).
2. If a crisis is detected, returns the safe crisis response immediately.
3. Otherwise builds a personalised reply using either:
 - * the OpenAI Chat Completions API (when an API key is configured), or
 - * a fully offline, rule-based fallback engine.

Because the fallback engine is always available, the application demonstrates end-to-end without any external dependency or API cost, important for a reproducible academic submission and a smooth live demo.

"""

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from typing import Dict, List, Optional
```

```
from ..config import SETTINGS
```

```
from ..ml import sentiment
```

```
from ..ml.recommender import recommend_from_signals
```

```
from . import prompts, safety
```

```
@dataclass
```

```
class AssistantReply:
```

```
    text: str
```

```
    risk_level: str
```

```
    sentiment: float
```

```
    emotion: str
```

```
    source: str          # 'openai' | 'offline' | 'safety'
```

```
# ----- #
```

```
# OpenAI path
```

```
# ----- #
```

```
def _openai_reply(message: str, history: List[Dict], note: str) -> Optional[str]:
```

```
    """Call the OpenAI API; return None on any failure so we can fall back."""
```

```
    try:
```

```
        from openai import OpenAI
```

```
        client = OpenAI(api_key=SETTINGS.openai_api_key)
```

```
        messages = [{"role": "system", "content": prompts.SYSTEM_PROMPT},
```

```
                     {"role": "system", "content": note}]
```

```
        # Include a trimmed window of prior turns for continuity.
```

```
        for turn in history[-8:]:
```

```
            role = "assistant" if turn["role"] == "assistant" else "user"
```

```
            messages.append({"role": role, "content": turn["content"]})
```

```
        messages.append({"role": "user", "content": message})
```

```
        response = client.chat.completions.create(
```

```
            model=SETTINGS.openai_model,
```

```
            messages=messages,
```

```
            temperature=0.7,
```

```
            max_tokens=400,
```

```
        )
```

```
        return response.choices[0].message.content.strip()
```

```
    except Exception:
```

```
        # Any error (missing package, network, quota) -> use offline engine.
```

```
        return None
```

```

# ----- #
# Offline rule-based path
# ----- #
_EMOTION_OPENERS = {
    "anxiety": "It sounds like anxiety is weighing on you right now, and that's really hard.",
    "sadness": "I'm sorry you're feeling so low, thank you for sharing that with me.",
    "stress": "It sounds like you're under a lot of pressure at the moment.",
    "anger": "It's completely understandable to feel frustrated about this.",
    "positive": "I'm really glad to hear there's some lightness for you today.",
    "neutral": "Thank you for telling me what's on your mind.",
}

def _offline_reply(message: str, emotion: str, segment: str, stress_level: str) -> str:
    """Construct a supportive, personalised reply with no external calls."""
    opener = _EMOTION_OPENERS.get(emotion, _EMOTION_OPENERS["neutral"])
    recs = recommend_from_signals(emotion=emotion, segment=segment,
                                stress_level=stress_level, top_k=1)

    parts = [opener]

    parts.append(
        "Feelings like these are a normal response to everything students "
        "are juggling, and they do pass. You don't have to fix everything at "
        "once."
    )

    if recs:
        rec = recs[0]
        steps = " ".join(f"{i+1}) {s}" for i, s in enumerate(rec.steps[:3]))

        parts.append(
            f"One small thing that might help right now is **{rec.title}**: "
            f"{steps}"
        )

    parts.append(
        "Would you like to talk a bit more about what's behind this, or shall "
        "we try a short exercise together?"
    )

    return "\n\n".join(parts)

# ----- #
# Public API
# ----- #
def generate_reply(message: str, history: Optional[List[Dict]] = None,
                  segment: str = "Coping", stress_level: str = "Moderate") -> AssistantReply:
    """Generate the assistant's reply to a single user message."""
    history = history or []

    risk = safety.assess_risk(message)
    sent = sentiment.analyze(message)

    if risk.is_crisis:
        return AssistantReply(
            text=safety.crisis_response(),
            risk_level="crisis",
            sentiment=sent.compound,
            emotion=sent.emotion,
            source="safety",
        )

    note = prompts.build_personalization_note(sent.emotion, segment, stress_level)

    text = None

```

```

source = "offline"
if SETTINGS.llm_enabled:
    text = _openai_reply(message, history, note)
    if text is not None:
        source = "openai"

if text is None:
    text = _offline_reply(message, sent.emotion, segment, stress_level)
    source = "offline"

# For elevated (non-crisis) distress, append a gentle resource nudge.

if risk.level == "elevated":
    text += (
        "\n\nIf these feelings stick around or get heavier, please "
        "consider reaching out to a counsellor or one of the support "
        "lines on the Resources page. You deserve support._"
    )

return AssistantReply(
    text=text,
    risk_level=risk.level,
    sentiment=sent.compound,
    emotion=sent.emotion,
    source=source,
)

```

Appendix F: Running the Project

The application and its full source code are available in the project repository. After creating a virtual environment and installing the dependencies from `requirements.txt`, the data and models are produced with `python -m scripts.generate_data` and `python -m scripts.train_models`, and the application is launched with `streamlit run app.py`. The complete instructions are provided in the project README.